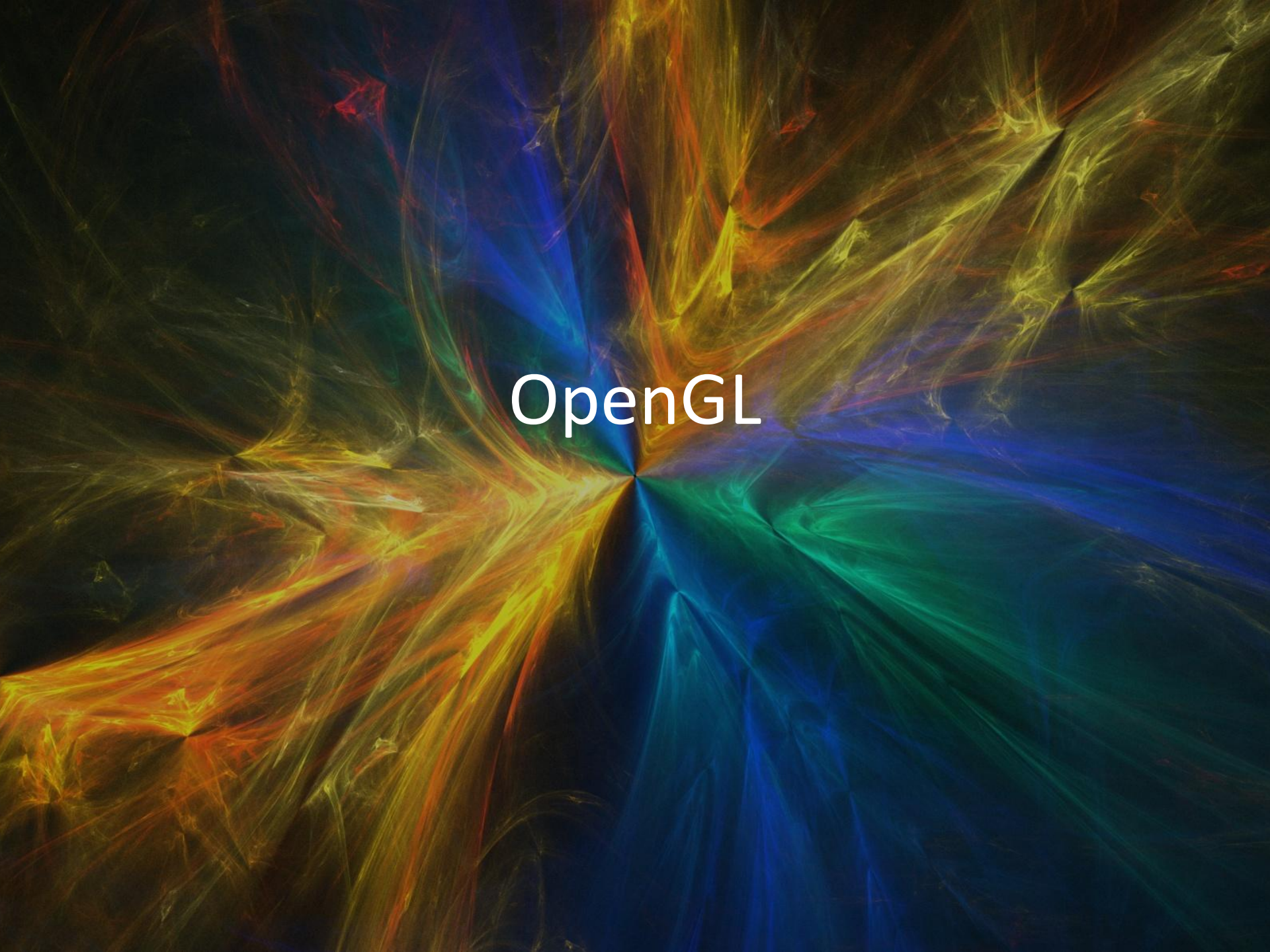


The background is a complex, abstract composition of numerous thin, overlapping lines and rays of light. These lines radiate from a central point, creating a starburst or nebula-like effect. The colors are primarily deep blues, vibrant yellows, and fiery oranges, with some hints of green and red. The overall texture is ethereal and dynamic, suggesting a digital or cosmic theme.

OpenGL

Czyszczenie buforów

```
glClearColor(r,g,b,a);
```

- $r, g, b, a \in [0, 1]$ (ułamki)
- Ustawia kolor czyszczenia bufora kolorów

```
glClear(maska);
```

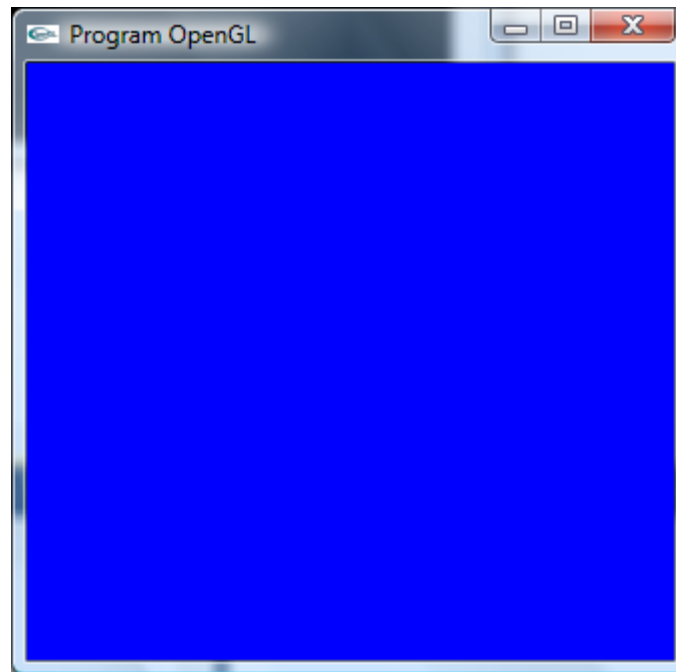
- `GL_COLOR_BUFFER_BIT` – czyści bufor kolorów
- `GL_DEPTH_BUFFER_BIT` – czyści z bufor
- Maski można łączyć za pomocą operatora `|`

```
glfwSwapBuffers(window);
```

- Przerzucenie tylnego bufora na przedni:

Zadanie

- Wyczyść zawartość okna na wybrany kolor (różowy jest zakazany).



Współrzędne jednorodne

$$\vec{a} = \begin{bmatrix} x \\ y \\ z \\ w \end{bmatrix} \equiv \begin{bmatrix} x/w \\ y/w \\ z/w \end{bmatrix}$$

Współrzędne jednorodne

Co jeśli $w=0$?

$$\vec{a} = \begin{bmatrix} x_a \\ y_a \\ z_a \\ 1 \end{bmatrix} \quad \vec{b} = \begin{bmatrix} x_b \\ y_b \\ z_b \\ 1 \end{bmatrix} \quad \vec{a} - \vec{b} = \begin{bmatrix} x_a - x_b \\ y_a - y_b \\ z_a - z_b \\ 0 \end{bmatrix}$$

Wniosek: jeżeli $w=0$ to współrzędna homogeniczna nie reprezentuje pozycji, a wektor (przesunięcie)

Transformacje geometryczne

- Reprezentowane przez macierze 4x4.
 - $R(kąt, \langle x, y, z \rangle)$ – macierz rotacji. Pomnożenie razy tę macierz obraca punkt o zadany kąt wokół osi zdefiniowanej przez wektor $\langle x, y, z \rangle$.
 - $a' = R(50^\circ, \langle 0, 0, 1 \rangle)a$
Obraca punkt a o 50 stopni wokół osi Z.
 - $T(\langle x, y, z \rangle)$ – macierz translacji. Pomnożenie razy tę macierz przesuwa punkt o zadany wektor.
 - $a' = T(\langle 1, 1, 2 \rangle)a$
Przesuwa punkt a o wektor $\langle 1, 1, 2 \rangle$
 - $S(\langle x, y, z \rangle)$ – macierz skalowania. Mnoży poszczególne składowe współrzędnej razy zadane współczynniki.
 - $a' = S(\langle 1, 1, 2 \rangle)a$
Skaluje punkt dwukrotnie wzdłuż osi Z.

Składanie transformacji

Można tak:

$$\vec{a}' = R(30^\circ, \langle 0,0,1 \rangle) S(2,1,2) R(10^\circ, \langle 0,1,0 \rangle) T(\langle 1,0,0 \rangle) \vec{a}$$

Ale mnożenie macierzy jest łączne, więc można i tak:

$$\vec{a}' = R(30^\circ, \langle 0,0,1 \rangle) S(2,1,2) R(10^\circ, \langle 0,1,0 \rangle) T(\langle 1,0,0 \rangle) \vec{a}$$

$$M = \left[R(30^\circ, \langle 0,0,1 \rangle) S(2,1,2) R(10^\circ, \langle 0,1,0 \rangle) T(\langle 1,0,0 \rangle) \right] \vec{a}$$

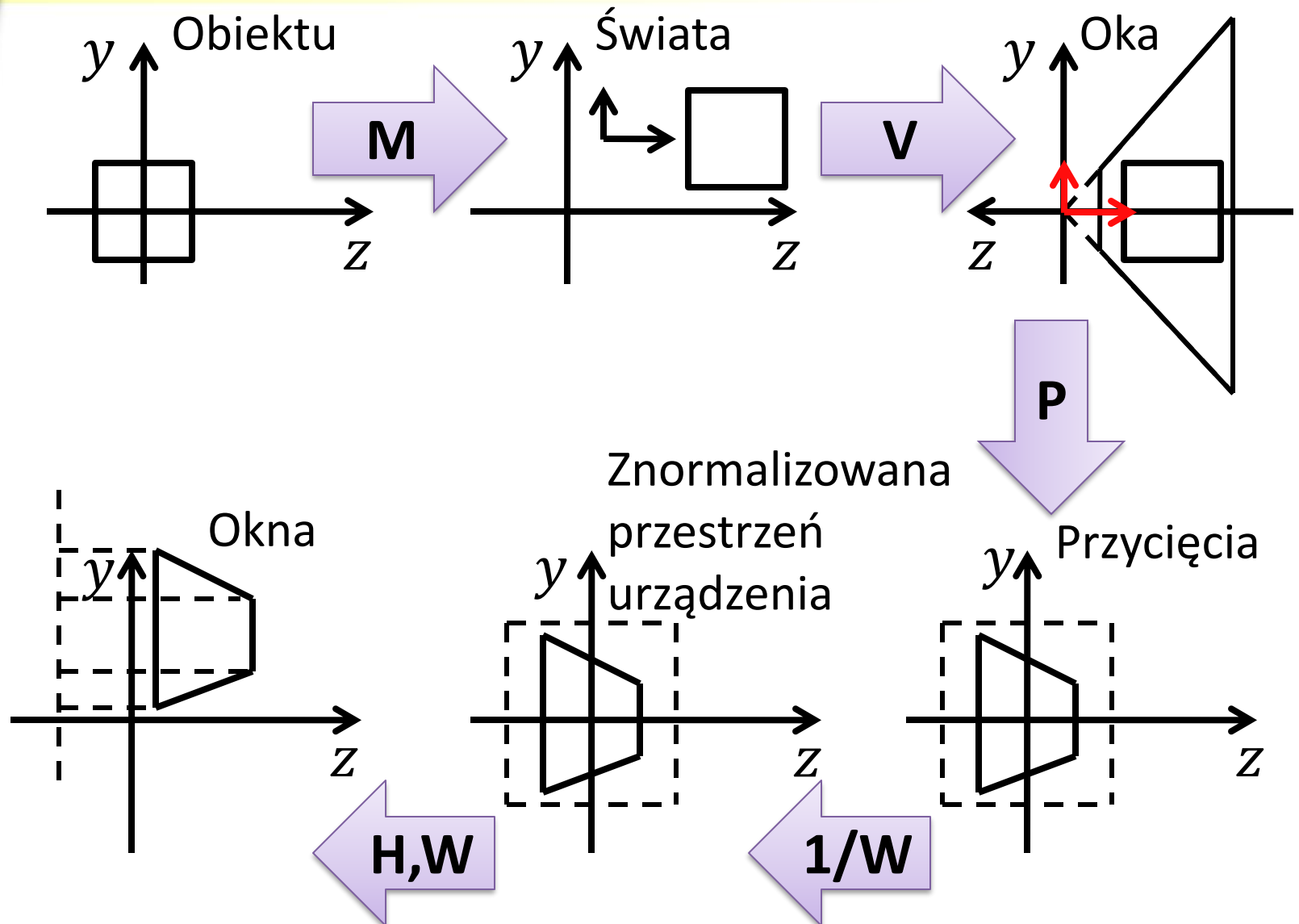
A zatem:

$$M = R(30^\circ, \langle 0,0,1 \rangle) S(2,1,2) R(10^\circ, \langle 0,1,0 \rangle) T(\langle 1,0,0 \rangle)$$

$$\vec{a}' = M \vec{a}$$

Macierz M jest złożeniem wszystkich transformacji!

Przestrzenie



Potok graficzny

- Macierz M (modelu)
 - nazywana również macierzą świata
 - złożenie transformacji geometrycznych,
- Macierz V (widoku)
 - reprezentuje kamerę w przestrzeni świata
- Macierz P (rzutowania)
 - reprezentuje sposób rzutowania na ekran

$$\vec{a'} = PVM\vec{a}$$

GLM – OpenGL Mathematics (1)

- Podstawowe typy:

```
glm::vec3 wektor3=glm::vec3(1.0f,2.0f,3.0f);

glm::vec4 wektor4=glm::vec4(1.0f,2.0f,3.0f,4.0f);

glm::mat3 macierz3x3=glm::mat3(
    vec3(1.0f,2.0f,3.0f),
    vec3(4.0f,5.0f,6.0f),
    vec3(7.0f,8.0f,9.0f),
);
```

$$\begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & 9 \end{bmatrix}$$
$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

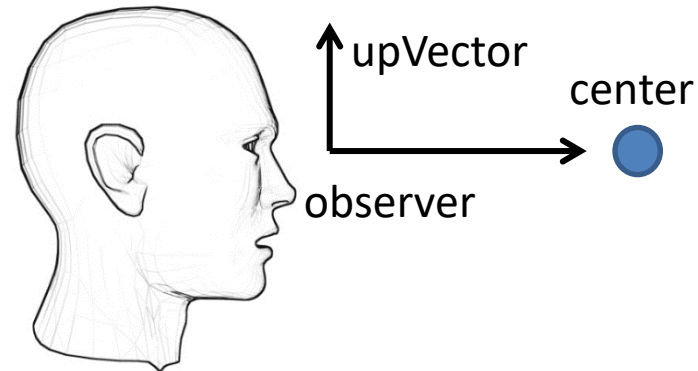
```
glm::mat4 macierz4x4=glm::mat4(1.0f);
```

- W typach zdefiniowano podstawowe operatory: +, -, *, /.
- Utwórz zmienną M zawierającą macierz jednostkową 4x4 w procedurze *drawScene*.

GLM – OpenGL Mathematics (2)

- **Macierz widoku**

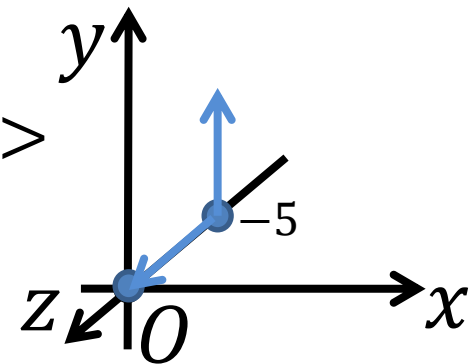
```
glm::mat4 glm::lookAt(  
    glm::vec3 observer,  
    glm::vec3 center,  
    glm::vec3 upVector);
```



- **Przykład:** kamera znajdująca się w

- punkcie $\langle 0, 0, -5 \rangle$
- patrząca na punkt $\langle 0, 0, 0 \rangle$
- z wektorem w górę kamery $\langle 0, 1, 0 \rangle$

```
glm::mat4 V=glm::lookAt(  
    glm::vec3(0.0f, 0.0f, -5.0f),  
    glm::vec3(0.0f, 0.0f, 0.0f),  
    glm::vec3(0.0f, 1.0f, 0.0f));
```

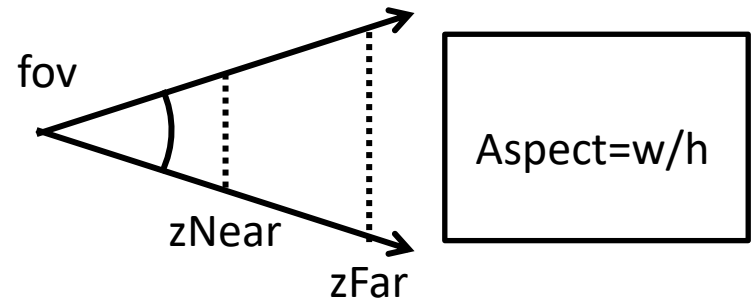


- **Skopiuj przykład do procedury *drawScene*!**

GLM – OpenGL Mathematics (3)

- Macierz rzutowania perspektywicznego

```
glm::mat4 glm::perspective(  
    float fovy,  
    float aspect,  
    float zNear,  
    float zFar);
```



- Przykład: kąt rozwarcia ostrosłupa 50° , bliska płaszczyzna odcinania 1, daleka 50, stosunek szerokości do wysokości okna 1:

```
glm::mat4 P=glm::perspective(  
    glm::radians(50.0f), 1.0f, 1.0f, 50.0f);
```

- Skopiuj przykład do procedury *drawScene*!
- Inne funkcje: frustum, infinitePerspective, ortho

OpenGL + GLM

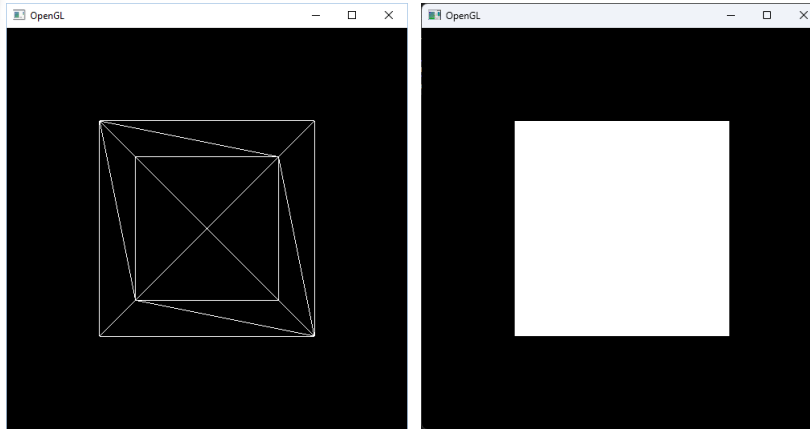
- Po zdefiniowaniu macierzy, trzeba aktywować program cieniujący i przekazać do niego macierze

```
spConstant->use(); //Aktywacja programu cieniującego  
glUniformMatrix4fv(spConstant->u("P"), 1, false, glm::value_ptr(P));  
glUniformMatrix4fv(spConstant->u("V"), 1, false, glm::value_ptr(V));  
glUniformMatrix4fv(spConstant->u("M"), 1, false, glm::value_ptr(M));
```

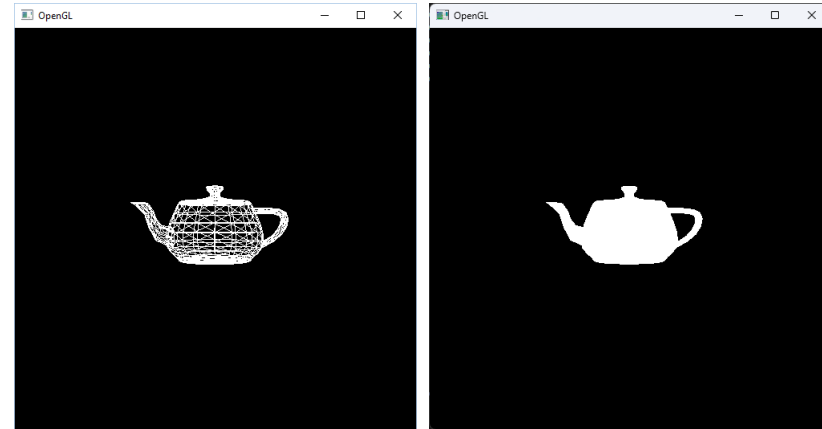
- Od tego momentu można rysować.

Gotowce (1/2)

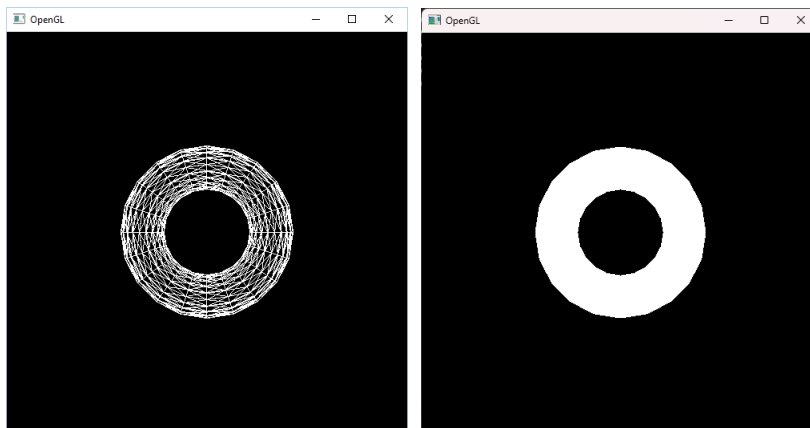
```
Models::cube.drawWire(); //lub  
Models::cube.drawSolid();
```



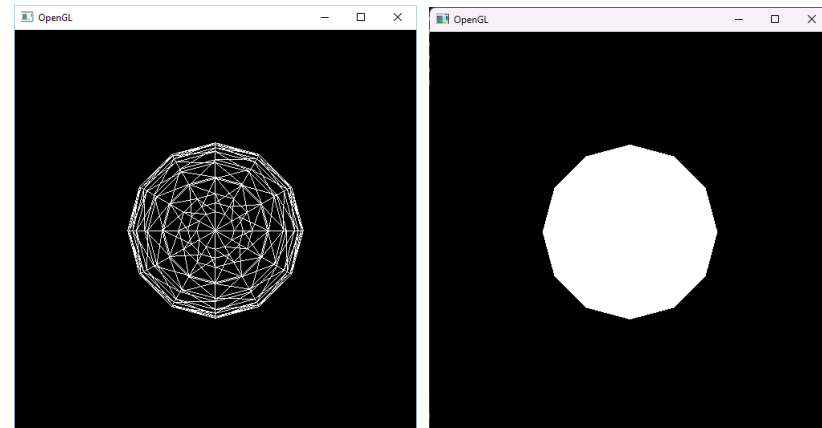
```
Models::teapot.drawWire(); //lub  
Models::teapot.drawSolid();
```



```
Models::torus.drawWire(); //lub  
Models::torus.drawSolid();
```



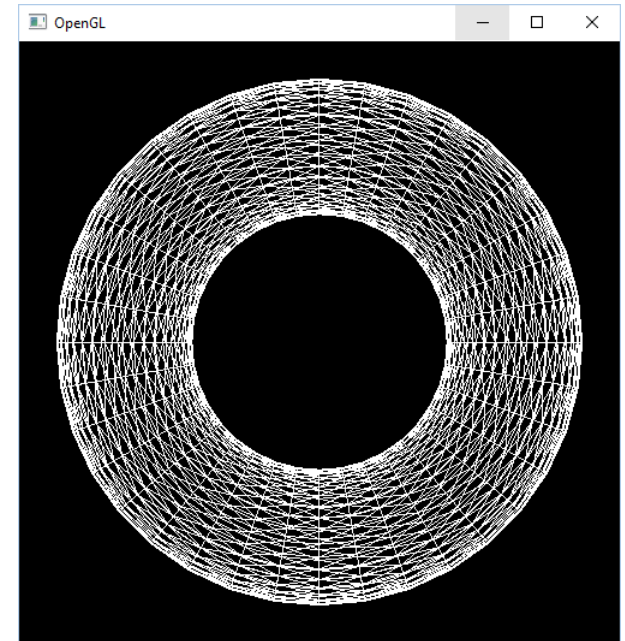
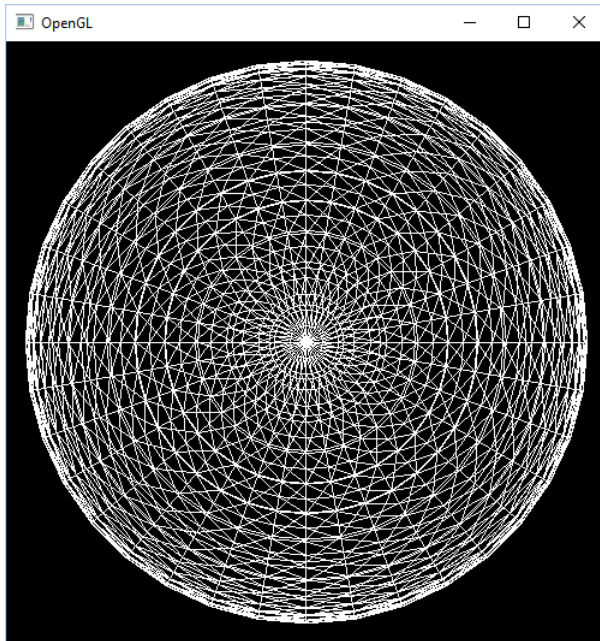
```
Models::sphere.drawWire(); //lub  
Models::sphere.drawSolid();
```



Gotowce (2/2)

```
//promień, liczba „południków”, liczba „równoleżników”  
static Models::Sphere mySphere(2,36,36); //Deklaracja poza drawScene!  
...  
mySphere.drawWire();
```

```
//promień zewnętrzny, promień wewnętrzny,  
// dokładność „pierścienia”, dokładność „tuby”  
static Models::Torus myTorus(1.5,0.5,36,36); //Deklaracja poza drawScene!  
...  
myTorus.drawWire();
```



Zadanie

- Narysuj wybrany obiekt.
- Nie zapomnij zdefiniować macierzy P , V i M oraz wysłać ich do programu cieniującego
- Wywołanie metody *drawWire* powinno być umieszczone w procedurze rysującej po wysłaniu macierzy do programu cieniującego, ale przed *glfwSwapBuffers*.

Składanie transformacji (1)

- $M = R(50, \langle 1, 0, 1 \rangle) T(\langle 1, 1, 0 \rangle) S(\langle 2, 2, 2 \rangle)$

```
glm::mat4 M;
```

```
//M=I;
```

```
M=glm::mat4(1.0f); //M zawiera macierz jednostkową
```

```
//M=IR(50, <1, 0, 1>) = R(50, <1, 0, 1>)
```

```
M=glm::rotate(M,  
    glm::radians(50.0f), glm::vec3(1.0f, 0.0f, 1.0f));
```

```
//M=R(50, <1, 0, 1>) T(<1, 1, 0>)
```

```
M=glm::translate(M, glm::vec3(1.0f, 1.0f, 0.0f));
```

```
//M=R(50, <1, 0, 1>) T(<1, 1, 0>) S(<2, 2, 2>)
```

```
M=glm::scale(M, glm::vec3(2.0f, 2.0f, 2.0f));
```

Składanie transformacji (2)

- Rozwiązanie alternatywne
- $M = R(50, \langle 1, 0, 1 \rangle) T(\langle 1, 1, 0 \rangle) S(\langle 2, 2, 2 \rangle)$

```
glm::mat4 M,R,T,S,I;
```

```
//zapisz do I macierz jednostkowa
```

```
I=glm::mat4(1.0f); //M zawiera macierz jednostkowa
```

```
//R=R(50,<1,0,1>)
```

```
R=glm::rotate(I,50.0f*PI/180.0f,glm::vec3(1.0f,0.0f,1.0f));
```

```
//T=T(<1,1,0>)
```

```
T=glm::translate(I,glm::vec3(1.0f,1.0f,0.0f));
```

```
//S=S(<2,2,2>)
```

```
S=glm::scale(I,glm::vec3(2.0f,2.0f,2.0f));
```

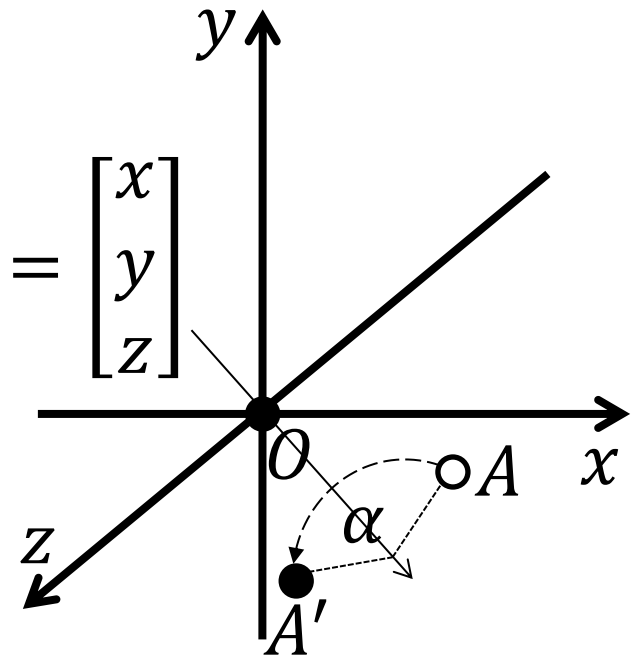
```
//M=R(50,<1,0,1>)T(<1,1,0>)S(<2,2,2>)
```

```
M=R*T*S;
```

glm::rotate

- Mnoży pierwszy argument razy macierz obrotu (**prawostronnie!**)
 - α – kąt obrotu
 - $\vec{o} = [x \ y \ z]^T$ - oś obrotu (przechodzi przez początek układu współrzędnych)

```
//M1=M*R(alfa, [x,y,z])  
  
glm::mat4 M1=glm::rotate(M,  
    alfa,  
    glm::vec3(x,y,z));
```



$$A' = R(\alpha, \vec{o}) \cdot A$$

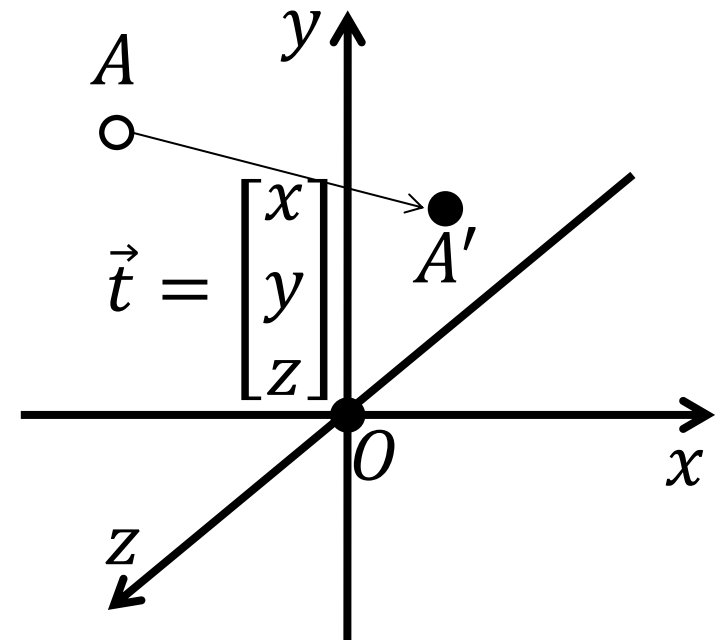
glm::translate

- Mnoży pierwszy argument razy macierz przesunięcia (**prawostronnie!**)

– $\vec{t} = [x \ y \ z]^T$ - wektor przesunięcia

```
//M1=M*T([x,y,z])
```

```
glm::mat4 M1=glm::translate(M,  
    glm::vec3(x,y,z));
```



$$A' = T(\vec{t}) \cdot A$$

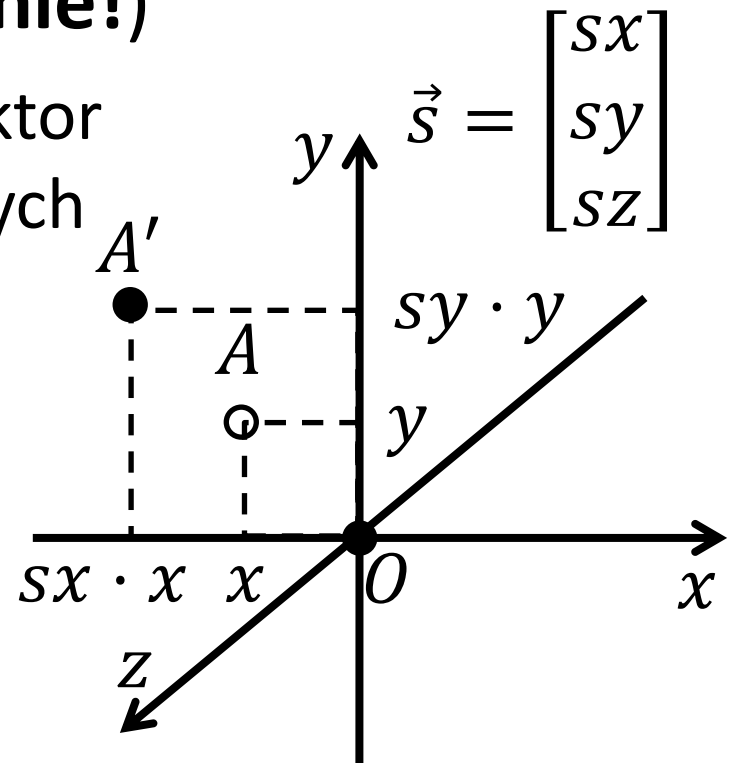
glm::scale

- Mnoży pierwszy argument razy macierz skalowania (**prawostronnie!**)

– $\vec{s} = [sx \quad sy \quad sz]^T$ - wektor współczynników skalujących

```
//M1=M*S([sx,sy,sz])
```

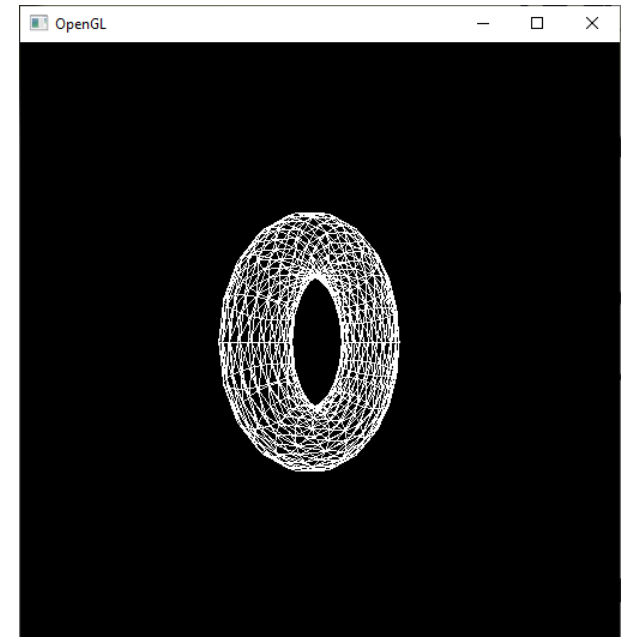
```
glm::mat4 M1=glm::scale(M,  
    glm::vec3(sx,sy,sz));
```



$$A' = S(\vec{s}) \cdot A$$

Zadanie

- Zaaplikuj dowolną transformację geometryczną do Twojego obiektu.
- Pamiętaj, że kolejność operacji w procedurze *drawScene* jest następująca:
 - Wyczyść ekran
 - Wylicz macierze
 - Prześlij do karty graficznej
 - Narysuj obiekt
 - Skopiuj bufor tylny do przedniego



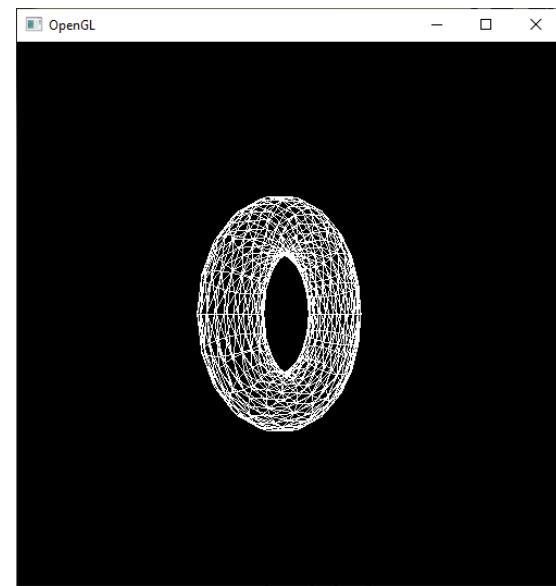
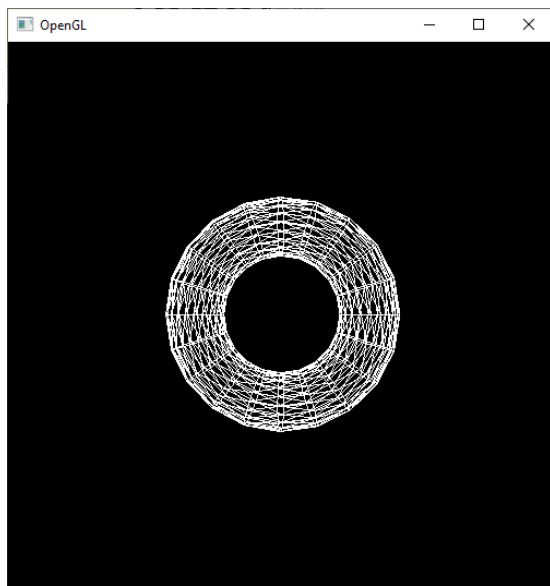
Animacja

```
float speed=3.14; // [radiany/s]
.....
void drawScene(GLFWwindow* window, float angle) {
.....
    glm::mat4 M=glm::mat4(1.0f);
    M=glm::rotate(M, angle, glm::vec3(0.0f, 1.0f, 0.0f));
.....
    Models::torus.drawWire();
.....
}

int main(void) {
.....
    float angle=0;
    glfwSetTime(0);
    while (!glfwWindowShouldClose(window)) {
        angle+=speed*glfwGetTime();
        glfwSetTime(0);
        drawScene(window, angle);
        glfwPollEvents();
    }
.....
}
```

Zadanie

- Zaimplementuj obracający się torus.
- Dodaj ewentualne inne transformacje zmieniające się w czasie.



Kolor rysowania

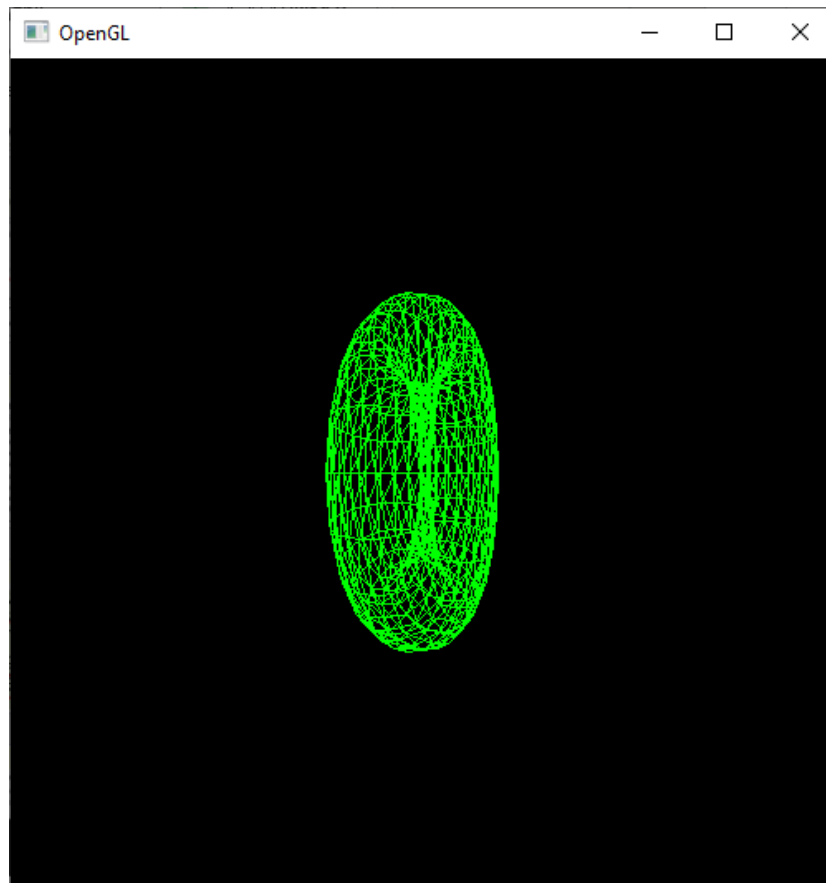
- Zmienia kolor rysowanego obiektu:

```
glUniform4f(spConstant->u("color"), r, g, b, a);
```

- Parametry aktualne to liczby ułamkowe z przedziału [0,1].
- Po aktywacji programu cieniującego, ale przed rysowaniem!

Zadanie

- Zmień kolor torusa na dowolny inny (ale nie różowy).



Zadanie

- Zastąp polecenie rysujące siatkę modelu obiektu poleceniem rysującym wypełniony obiekt (zamiast `drawWire...` użyj `drawSolid...`).
- Czy wynik jest zadowalający?

Cieniowanie

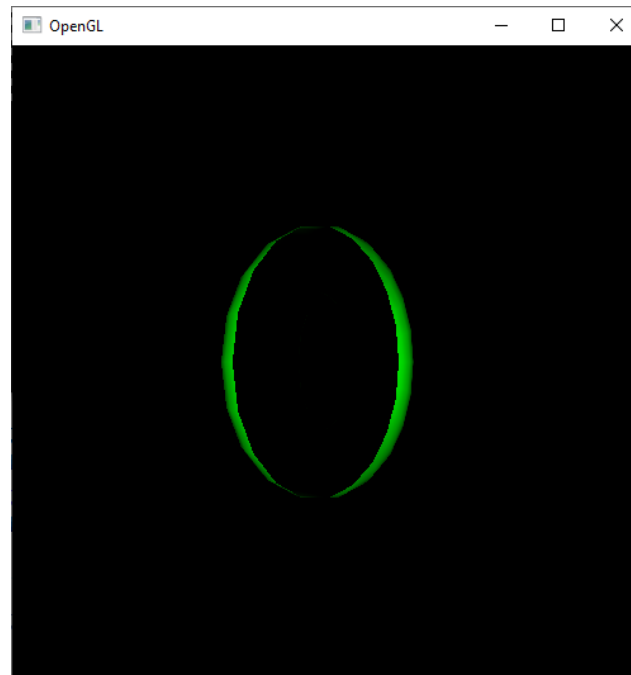
- Aby włączyć cieniowanie należy zmienić program cieniujący

```
spLambert->use();//Aktywacja programu cieniującego  
glUniform4f(spLambert->u("color"),0,1,0,1);  
glUniformMatrix4fv(spLambert->u("P"),1,false,glm::value_ptr(P));  
glUniformMatrix4fv(spLambert->u("V"),1,false,glm::value_ptr(V));  
glUniformMatrix4fv(spLambert->u("M"),1,false,glm::value_ptr(M));
```

- Domyślnie źródło światła ma kolor biały i znajduje się w nieskończoności, za obserwatorem.

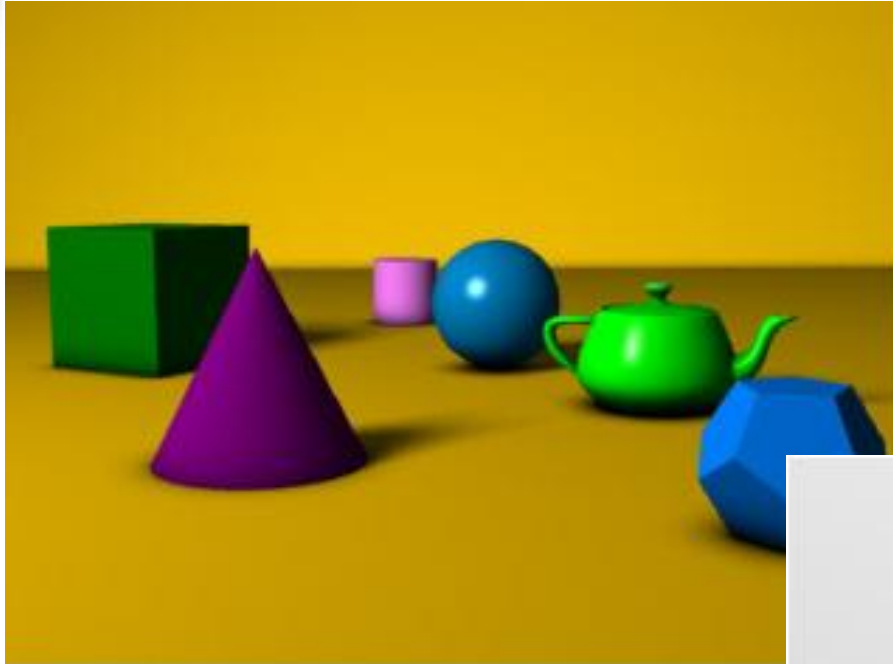
Zadanie

- Dodaj linijki włączające cieniowanie.
- Co się zmieniło?
- Czy efekt jest zadowalający?



Z-Bufor

Bufor Kolorów



Z-Bufor



Z-Bufor

1	1	1	1	1	1
1	1	1	1	1	1
1	1	1	1	1	1
1	1	1	1	1	1
1	1	1	1	1	1
1	1	1	1	1	1

0,7	0,7	0,7	0,7	0,7	0,7
0,7	0,7	0,7	0,7	0,7	
0,7	0,7	0,7	0,7		
0,7	0,7	0,7			
0,7	0,7				
0,7					
0,7					

0,7	0,7	0,7	0,7	0,7	0,7
0,7	0,7	0,7	0,7	0,7	1
0,7	0,7	0,7	0,7	1	1
0,7	0,7	0,7	1	1	1
0,7	0,7	1	1	1	1
0,7	1	1	1	1	1

0,7	0,7	0,7	0,7	0,7	0,7
0,7	0,7	0,7	0,7	0,7	1
0,7	0,7	0,7	0,7	1	1
0,7	0,7	0,7	1	1	1
0,7	0,7	1	1	1	1
0,7	1	1	1	1	1

0,8					
0,5	0,8				
0,3	0,5	0,8			
0,1	0,3	0,5	0,8		

0,7	0,7	0,7	0,7	0,7	0,7
0,7	0,7	0,7	0,7	0,7	1
0,5	0,7	0,7	0,7	1	1
0,3	0,5	0,7	1	1	1
0,1	0,3	0,5	0,8	1	1
0,7	1	1	1	1	1

Z-Bufor

- Z-Bufor służy do usuwania niewidocznych powierzchni.
- Bufor ten przechowuje informację w jakiej odległości od obserwatora znajduje się aktualnie narysowany piksel.
- Jeżeli kolejny rysowany w tym miejscu piksel jest dalej, to jest on pomijany, gdyż obecny przesłania go.
- Aby włączyć Z-Bufor należy wykonać instrukcję:

```
glEnable(GL_DEPTH_TEST); //w części inicjującej
```

Zadanie

- Włącz Z-Bufor.
- Czy teraz jest OK?
- Czego brakuje?

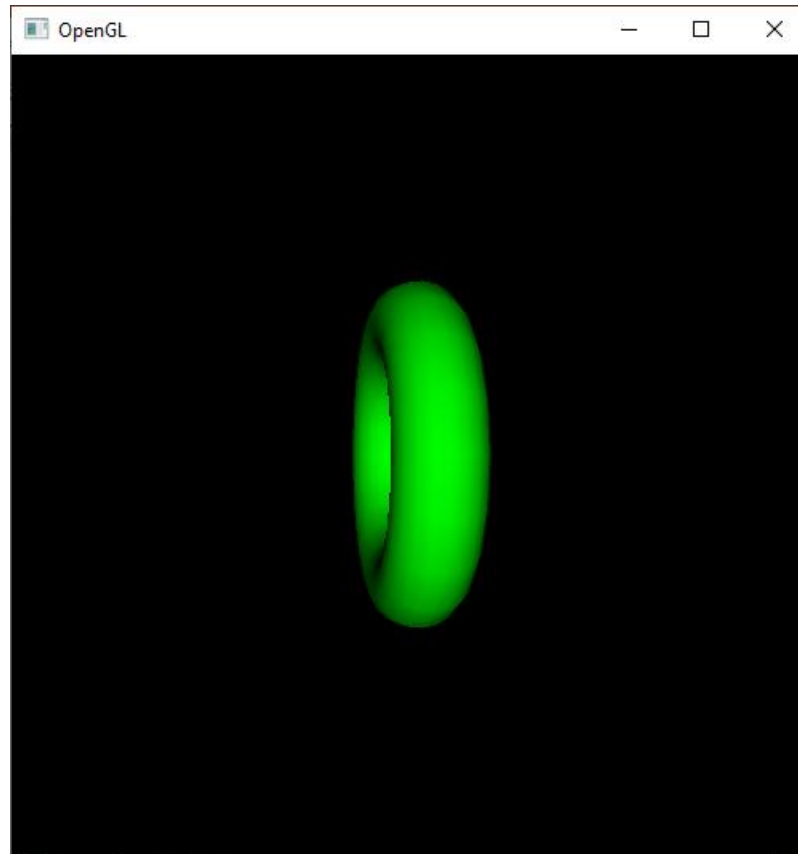
Czyszczenie Z-Bufora

- Z-Bufor, podobnie jak bufor kolorów musi być kasowany co klatkę.
- Wykorzystuje się do tego tę samą procedurę co dla bufora kolorów.

```
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
```

Zadanie

- Włącz czyszczenie Z-Bufora



Interakcja z użytkownikiem

- Biblioteka GLFW pozwala na zdefiniowanie procedury callback wywoływanej w momencie wciśnięcia klawisza.

Interakcja z użytkownikiem w GLFW

- Procedura *callback* obsługi klawiatury

```
void key_callback(  
    GLFWwindow* window,  
    int key,  
    int scancode,  
    int action,  
    int mod  
)
```

Aktywne okno

Wciśnięty klawisz:

- GLFW_KEY_XXX

Kod klawisza (zależny od platformy)

Mapa bitowa zbudowana z:

- GLFW_MOD_SHIFT = 0x01
- GLFW_MOD_CONTROL = 0x02
- GLFW_MOD_ALT = 0x04
- GLFW_MOD_SUPER = 0x08

Jedno z:

- GLFW_PRESS
- GLFW_RELEASE
- GLFW_REPEAT

//Rejestracja procedury *callback*:

```
glfwSetKeyCallback(window, key_callback);
```


Interakcja z użytkownikiem w GLFW

```
void key_callback(GLFWwindow* window, int key,
    int scancode, int action, int mods){

    if (action == GLFW_PRESS) {
        if (key == GLFW_KEY_A) printf("A \n");
        if (key == GLFW_KEY_D) printf("D \n");
        if (key == GLFW_KEY_W && (mods & GLFW_MOD_ALT) !=0)
            printf("ALT+W\n");
    }

    if (action == GLFW_RELEASE) {
        if (key == GLFW_KEY_W) printf("puszczone W\n");
    }

}
```

Zadanie

- Niech torus obraca się w reakcji na wciskanie strzałek w lewo i prawo.
- Kiedy żaden klawisz nie jest wciśnięty, torus powinien stać w miejscu.
- Można to zrealizować następująco:
 - Początkowo: $\text{speed}=0$
 - Wciśnięta $\leftarrow$ (GLFW_KEY_LEFT): $\text{speed}=-3.14$
 - Wciśnięta $\rightarrow$ (GLFW_KEY_RIGHT): $\text{speed}=3.14$
 - Klawisz puszczony: $\text{speed}=0$

Składanie transformacji (1)

- $M = R(50, \langle 1, 0, 1 \rangle) T(\langle 1, 1, 0 \rangle) S(\langle 2, 2, 2 \rangle)$

```
glm::mat4 M;
```

```
//M=I;
```

```
M=glm::mat4(1.0f); //M zawiera macierz jednostkowa
```

```
//M=IR(50, <1, 0, 1>) = R(50, <1, 0, 1>)
```

```
M=glm::rotate(M,  
              glm::radians(50.0f), glm::vec3(1.0f, 0.0f, 1.0f));
```

```
//M=R(50, <1, 0, 1>) T(<1, 1, 0>)
```

```
M=glm::translate(M, glm::vec3(1.0f, 1.0f, 0.0f));
```

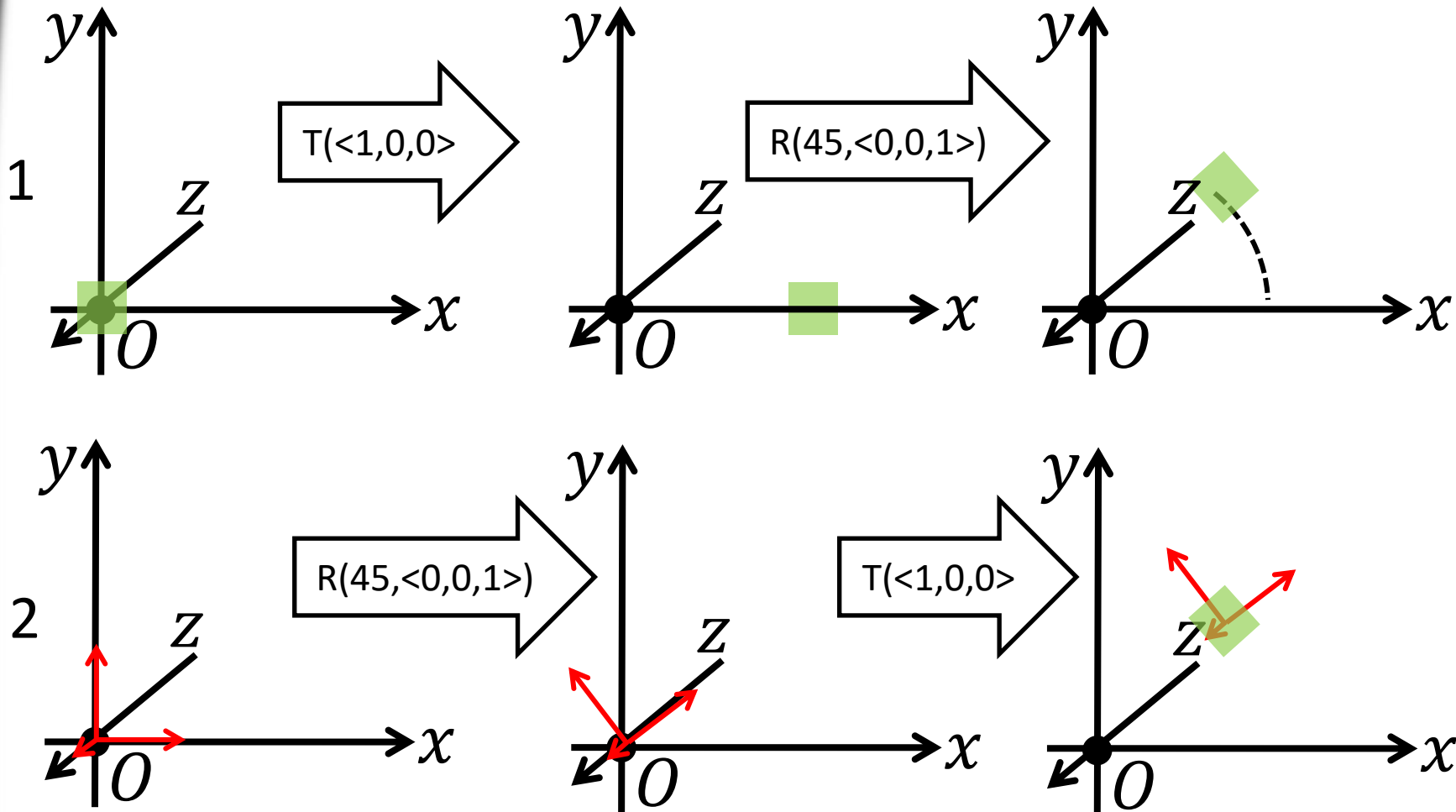
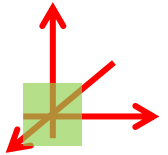
```
//M=R(50, <1, 0, 1>) T(<1, 1, 0>) S(<2, 2, 2>)
```

```
M=glm::scale(M, glm::vec3(2.0f, 2.0f, 2.0f));
```

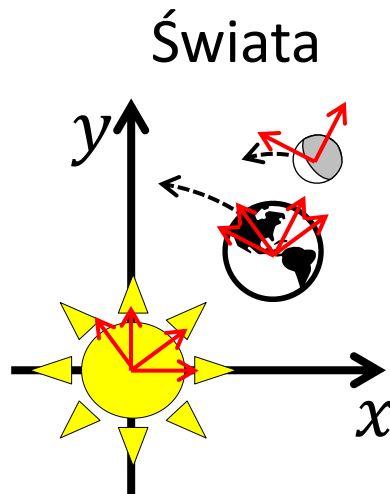
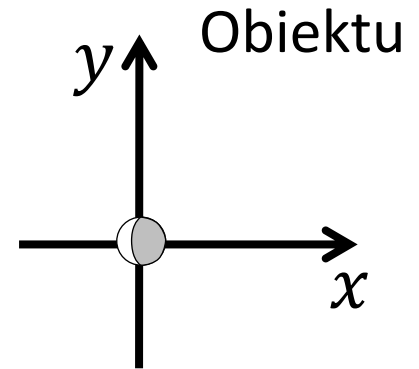
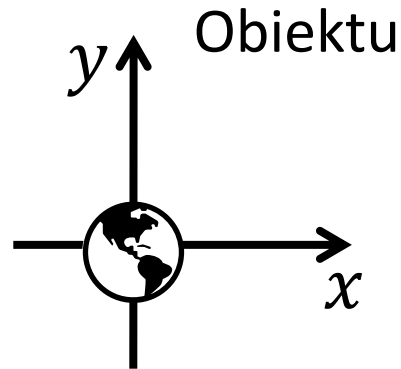
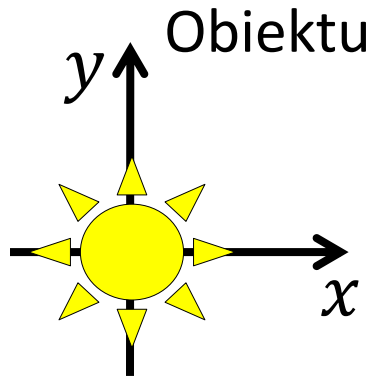
- $a' = PVMa = PVRTSa$

Wyliczanie macierzy modelu

$$\vec{a}' = M\vec{a} = R(45, \langle 0,0,1 \rangle)T(\langle 1,0,0 \rangle)\vec{a}$$



Wyliczanie macierzy modelu



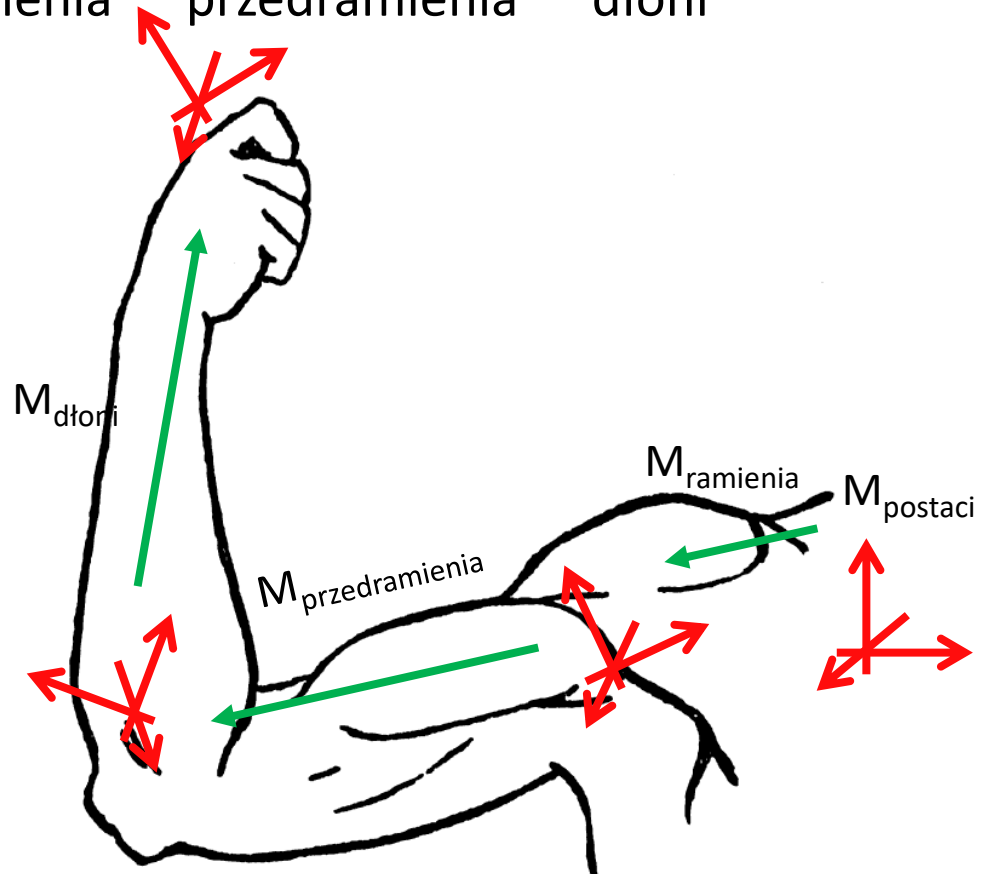
$$M_S = I$$

$$M_P = R_Z(angle) * T$$

$$M_K = M_P * R_Z(angle') * T'$$

Hierarchiczne modele

- Jeżeli rysujemy ramię, to $M = M_{\text{postaci}} M_{\text{ramienia}}$
- Jeżeli rysujemy dłoń, to:
- $M = M_{\text{postaci}} M_{\text{ramienia}} M_{\text{przedramienia}} M_{\text{dłoni}}$



Kolejność wykonywania poleceń

```
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

glm::mat4 P = glm::perspective(glm::radians(50.0f), 1.0f, 1.0f, 50.0f);
glm::mat4 V = glm::lookAt(
    glm::vec3(0.0f, 0.0f, -5.0f),
    glm::vec3(0.0f, 0.0f, 0.0f),
    glm::vec3(0.0f, 1.0f, 0.0f));

spLambert->use();
glUniformMatrix4fv(spLambert->u("P"), 1, false, glm::value_ptr(P));
glUniformMatrix4fv(spLambert->u("V"), 1, false, glm::value_ptr(V));

//---Poniższy kawałek kodu powtarzamy dla każdego obiektu
//Obliczanie macierzy modelu
glm::mat4 M = glm::mat4(1.0f);
M = glm::rotate(M, angle, glm::vec3(0.0f, 1.0f, 0.0f));

//Załadowanie macierzy modelu do programu cieniującego
glUniformMatrix4fv(spLambert->u("M"), 1, false, glm::value_ptr(M));
glUniform4f(spLambert->u("color"), 0, 1, 0, 1);

Models::torus.drawSolid(); //Narysowanie obiektu
//-----

glfwSwapBuffers(window); //Skopiowanie bufora ukrytego do widocznego.
```


Ćwiczenie (1/2)

- Zrób prosty układ planetarny
 - Słońce w początku układu współrzędnych

- Promień słońca 0.5

```
static Models::Sphere sun(0.5,36,36)
...
sun.drawSolid()
```

- Planeta z orbitującym dookoła księżycem:

- Promień planety 0.2

```
static Models::Sphere planet1(0.2,36,36)
...
planet1.drawSolid()
```

- Promień orbity planety 1.5 (translacja [1.5,0,0])

- Oś orbity planety <0,1,0>

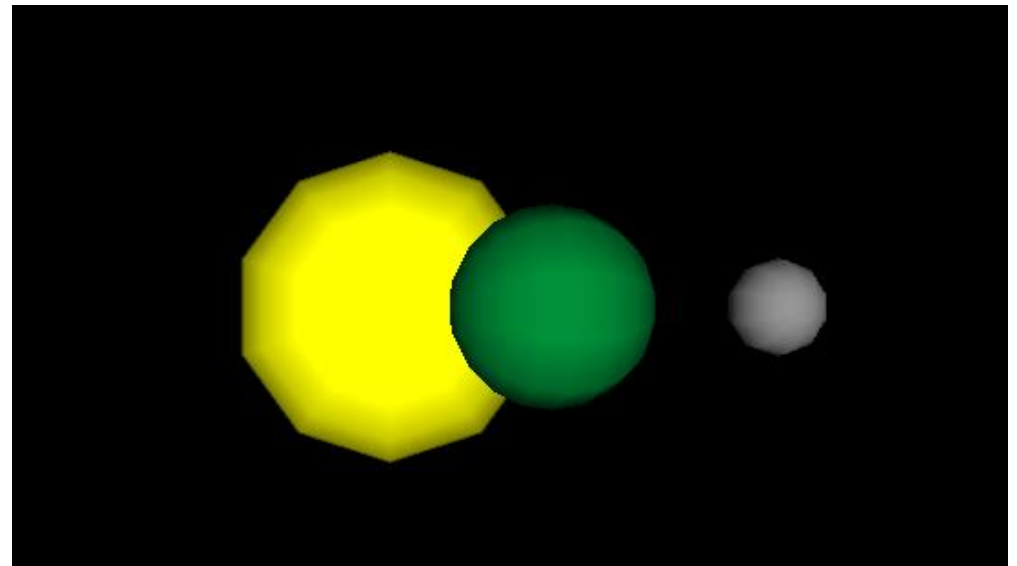
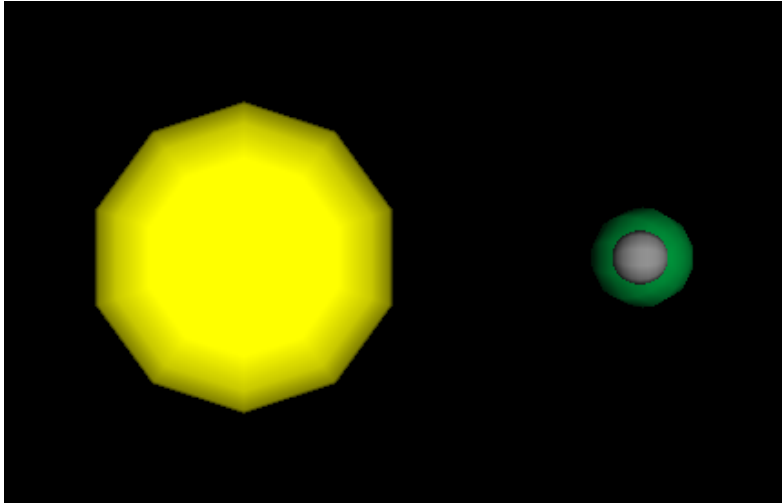
```
static Models::Sphere moon1(0.1,36,36)
...
moon1.drawSolid()
```

- Promień księżyca 0.1

- Promień orbity księżyca dookoła planety 0.5 (translacja [0.5,0,0])

- Oś orbity księżyca <0,1,0>

Ćwiczenie (2/2)



Ćwiczenie

- Dodaj drugą planetę z księżycem

- Promień planety 0.25

```
static Models::Sphere planet2(0.25,36,36)
...
planet2.drawSolid()
```

- Promień orbity planety 2 (translacja [2,0,0])

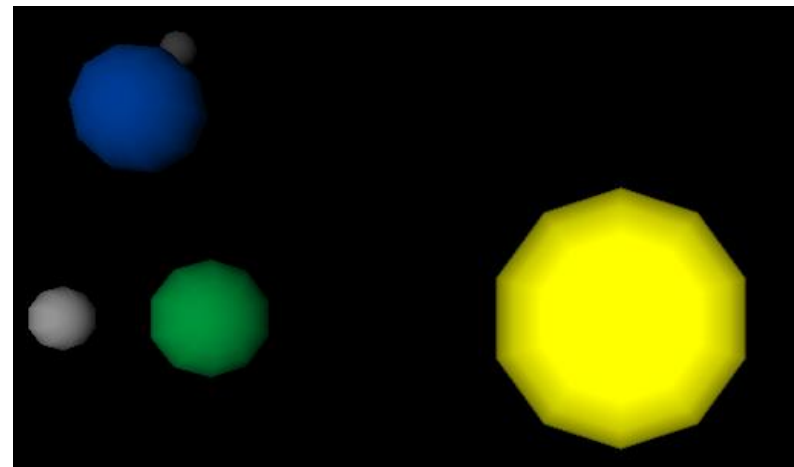
- Oś obrotu planety <0,0,1>

- Promień księżyca 0.07

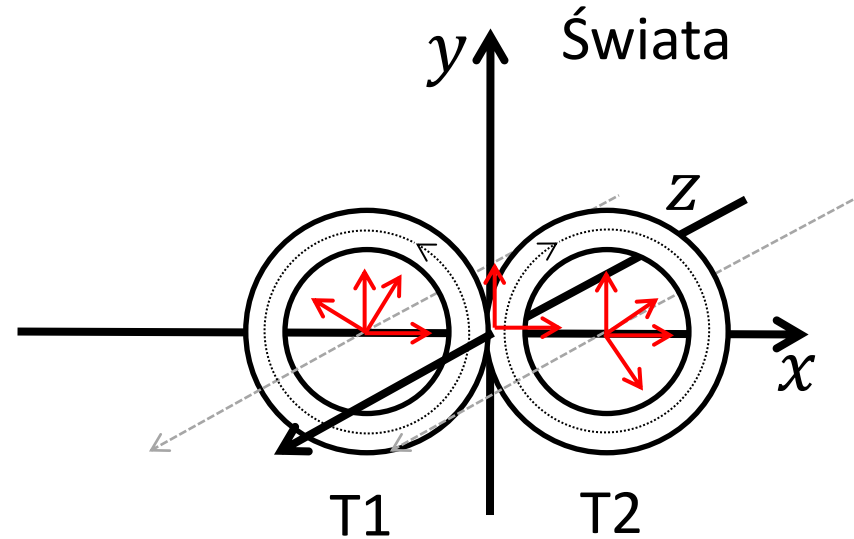
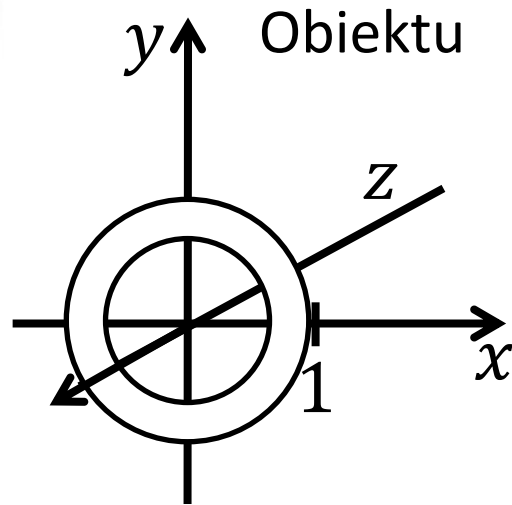
```
static Models::Sphere moon2(0.07,36,36)
...
moon2.drawSolid()
```

- Promień orbity księżyca dookoła planety 0.3 (translacja [0,0,0.3]!!)

- Oś obrotu księżyca <1,0,0>



Ćwiczenie

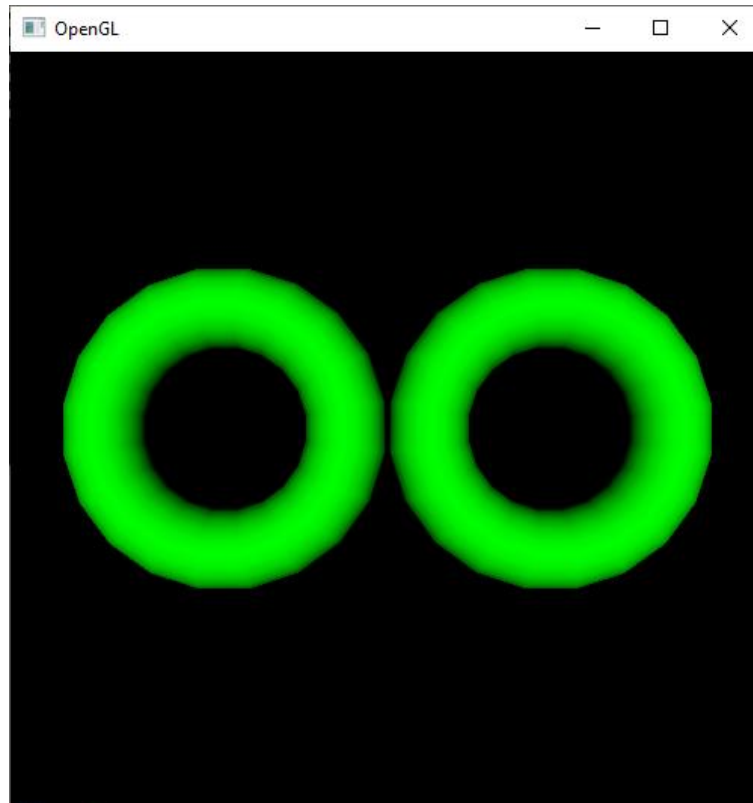


$$M_{T1} = T(< -1, 0, 0 >)R_Z(angle)$$

$$M_{T2} = T(< 1, 0, 0 >)R_Z(-angle)$$

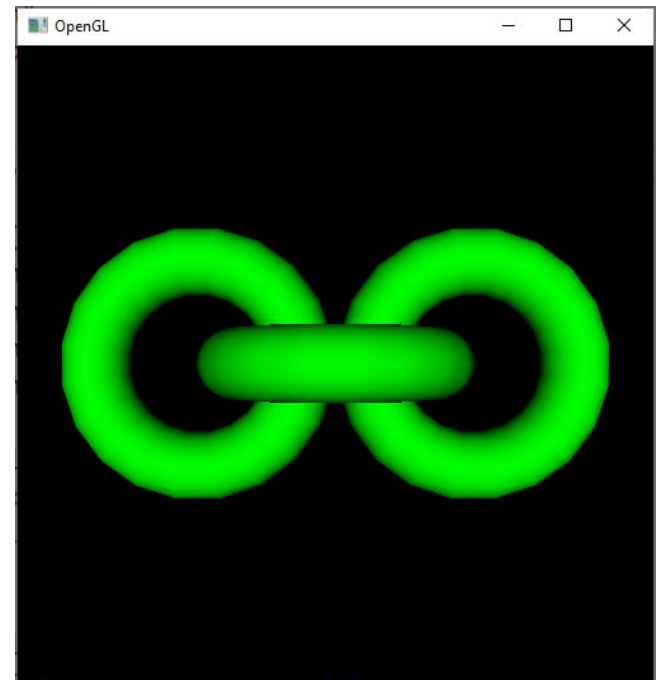
Ćwiczenie

- Ostatecznie powinno powstać coś podobnego do tego zrzutu ekranu:



Ćwiczenie opcjonalne

- Dodaj prostopadły torus łączący obecne dwa w łańcuszek
- Nowy torus powinien obracać się wokół osi przechodzącej przez jego środek. Kierunek dowolny.
- Usuń ten torus przed kolejnym ćwiczeniem

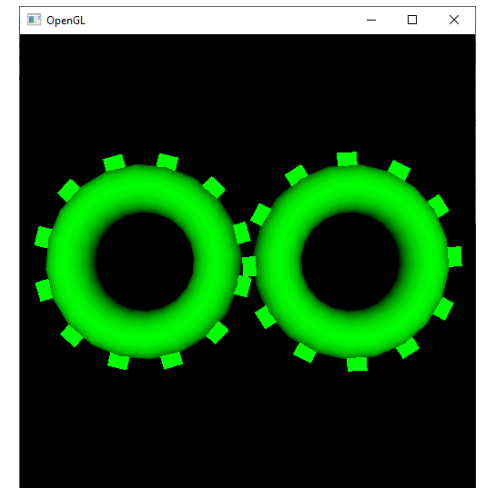


Ćwiczenie

- Dodaj kostki na obrzeżach torusów tak, aby tworzyły „ząbki” na trybach
- Rozmiar kostki 0.2 (normalnie kostka ma wymiary 2x2x2, więc należy ją przeskalować ze współczynnikiem 0.1 na każdym wymiarze)
- Umieść 12 kostek (co 30 stopni)
- Konieczne będzie rozsuniecie torusów.
- Konieczne będzie odpowiednie dobranie położeń kostek, żeby się nie nakładały
- Sposób obliczania macierzy kostki to:

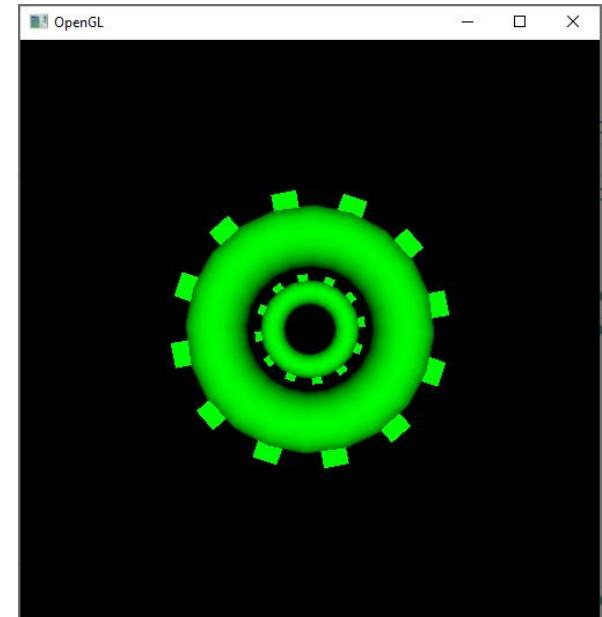
$$M_K = M_T * R_z(30x) * T(1,0,0) * S(0.1,0.1,0.1)$$

- Gdzie x to numer kostki, 30x to kąt w stopniach a M_T to macierz torusa (obojętnie którego).
- Kostki należy rysować w pętli (kolejne iteracje to kolejne wartości x)



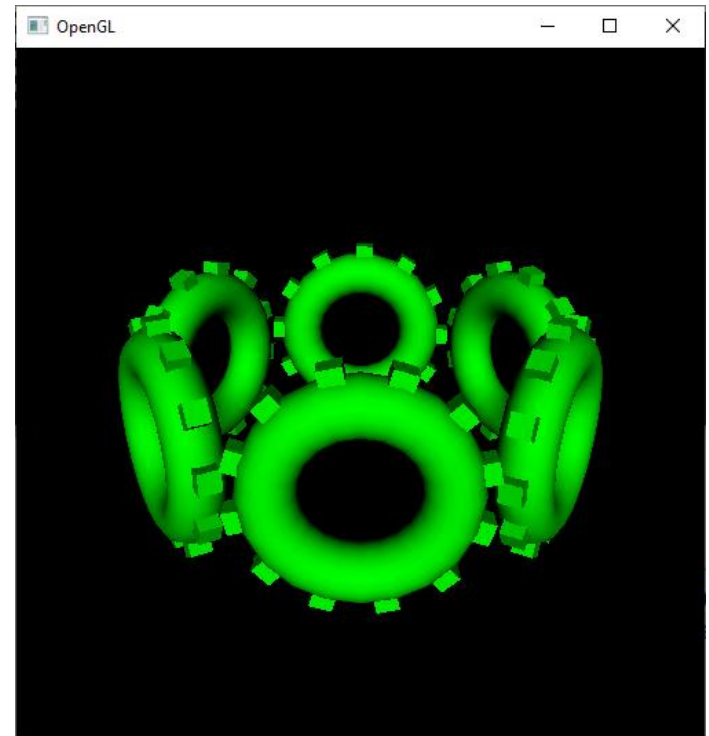
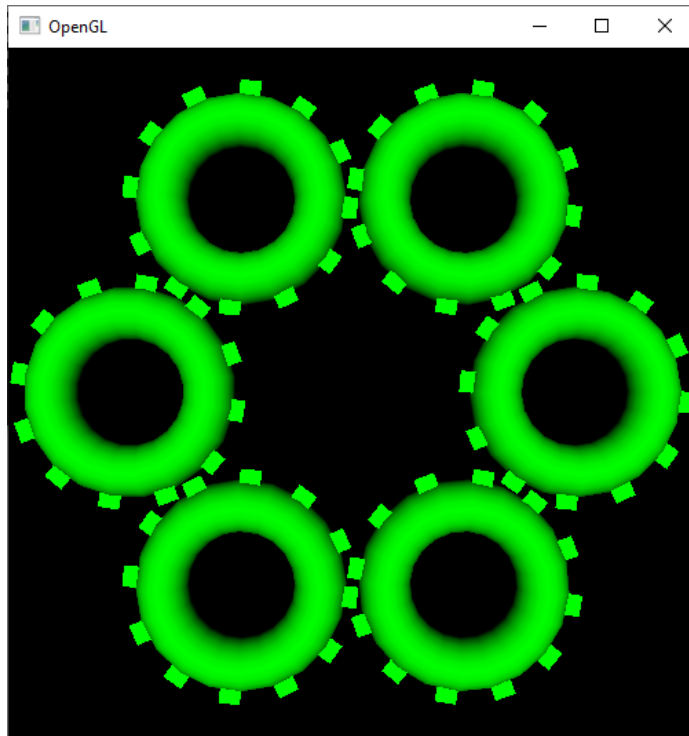
Ćwiczenie opcjonalne

- Pomniejsz i przesuń jeden z „trybów” do środka drugiego. Mają się dalej obracać w przeciwnych kierunkach.
- Aby ułatwić sobie realizację tego ćwiczenia najlepiej zrobić osobną procedurę rysującą „tryb”, której parametrem jest macierz modelu reprezentująca miejsce (i skalę) w którym ma zostać narysowany „tryb”.



Ćwiczenie

- Spróbuj zbudować następujące sceny (animowane odpowiednio oczywiście):

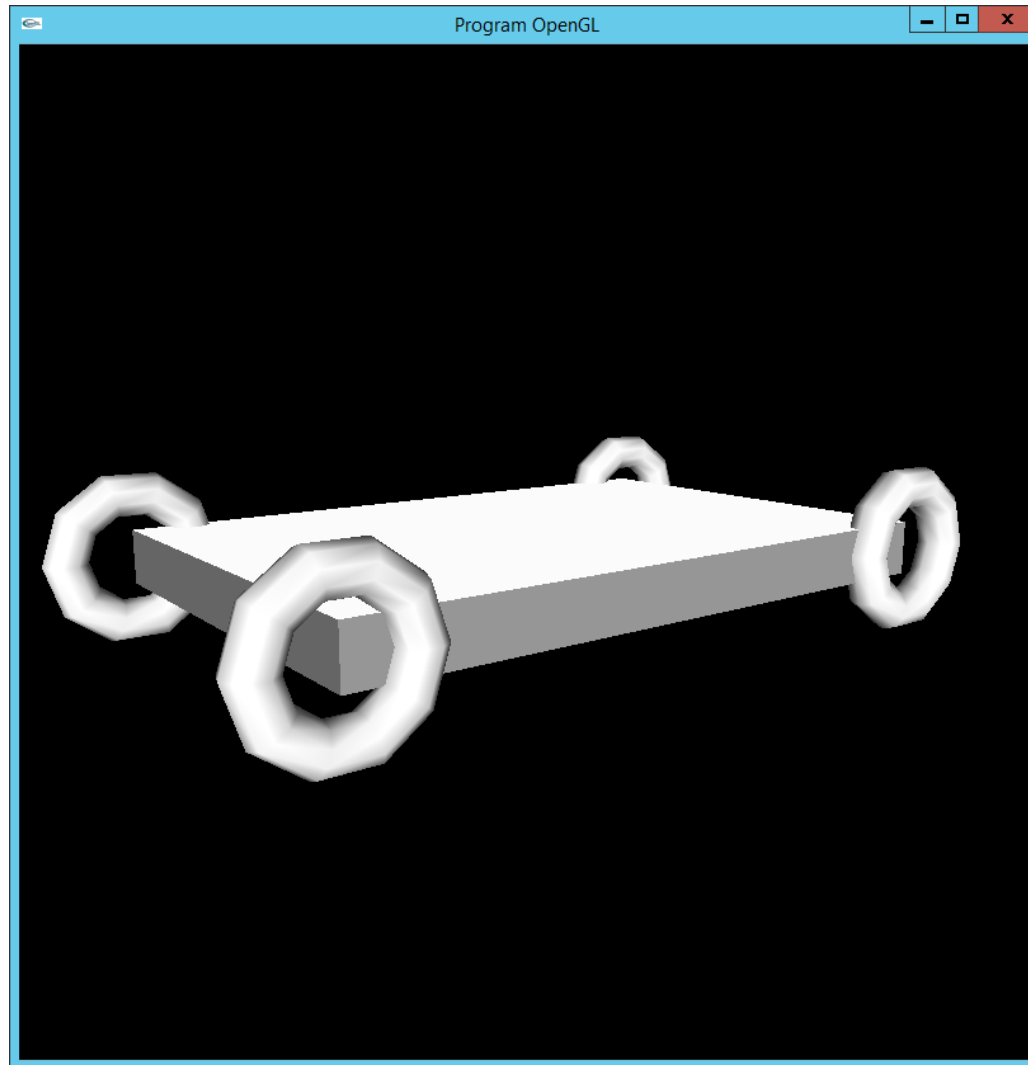


Ćwiczenie (1/2)

- Zamodeluj podwozie samochodu.
 - Podwozie składa się ze zdeformowanej kostki oraz 4 torusów tworzących koła
 - Zdeformowana kostka ma posiadać wymiary 3x0.25x2 (**Uwaga! Oryginalna ma 2x2x2**)
 - Każdy torus tworzący koło powinien posiadać promień wewnętrzny 0.1 i zewnętrzny 0.3.
- Podwozie można obracać dookoła za pomocą strzałek

```
static Models::Torus kolo(0.3,0.1,12,12)
...
kolo.drawSolid()
```

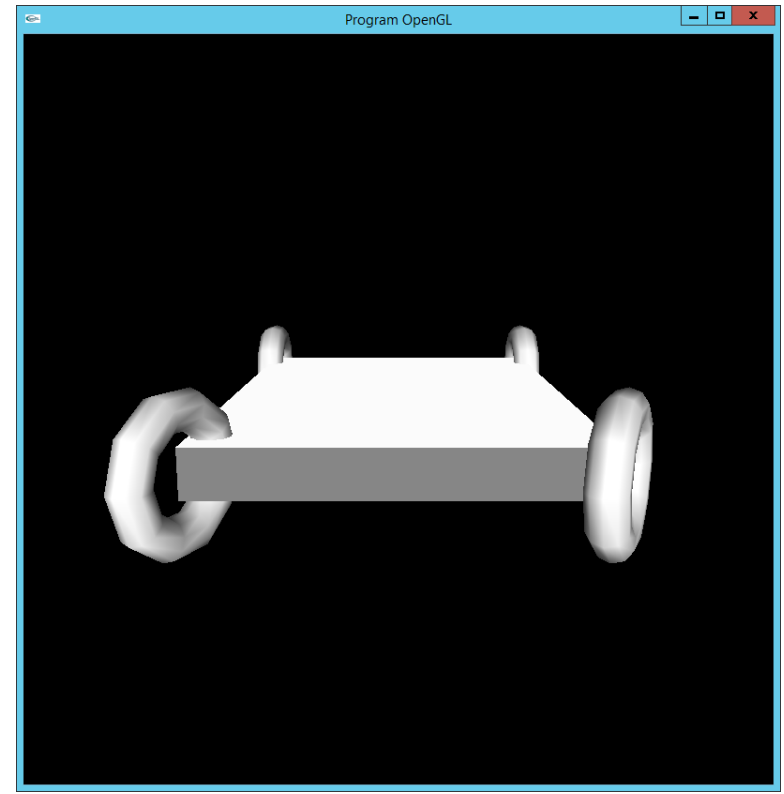
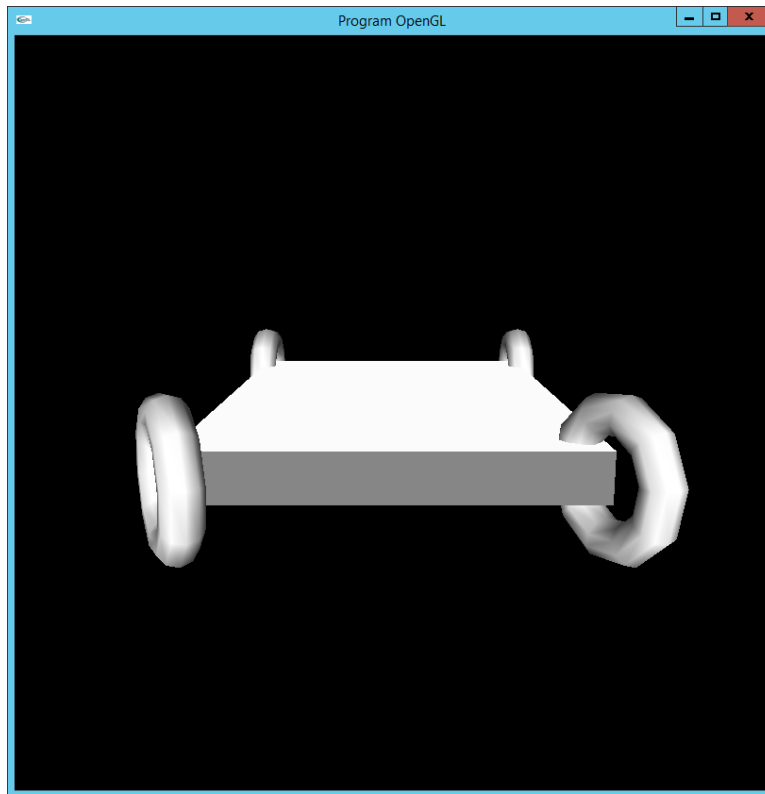
Ćwiczenie (2/2)



Ćwiczenie (1/2)

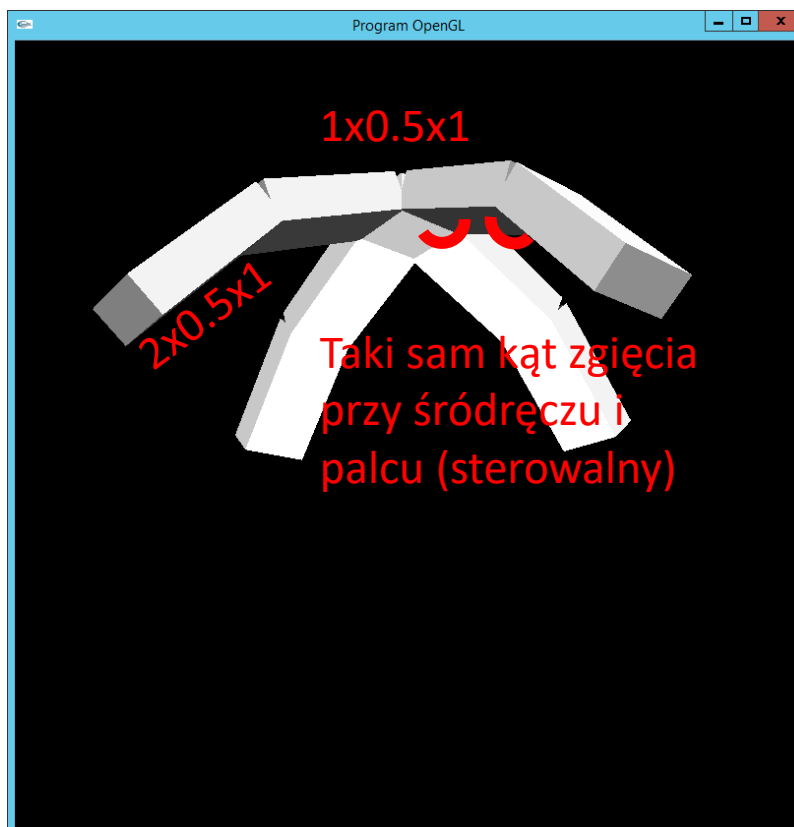
- Dodaj animację kół
 - Koła powinny cały czas wirować
 - Przednimi kołami można skręcać, np. klawiszami „a” i „d”. Puszczanie klawiszy przywraca normalny kierunek jazdy
 - Samo skręcanie przednich kół nie musi być animowane. Wciśnięcie „a” lub „d” powinno po prostu powodować, że koła od razu zmieniają swoją pozycję i obracają się w odpowiedni sposób

Ćwiczenie (2/2)



Ćwiczenie (1/3)

- Zrób dłoń robota zgodnie z rysunkiem.
 - Można sterować kątem zgięcia palców w stosunku do śródręcza

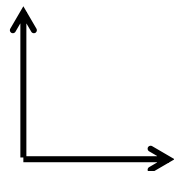


Ćwiczenie (2/3)

- W pierwszym kroku zdefiniuj procedurę rysującą prostopadłościan o dowolnych wymiarach. Zrób to w oparciu o `Models::cube` i skalowanie.
- Następnie zdefiniuj procedurę rysującą pojedynczy palec.
 - Procedura jako parametr powinna przyjmować:
 - Macierz reprezentującą układ współrzędnych u „nasady” palca
 - Kąt zgięcia palca
 - Sposób działania procedury jest pokazany na kolejnych slajdach

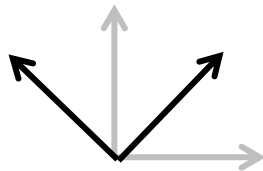
Kolejne kroki rysowania „palca”

Początkowy układ współrzędnych
przekazany jako parametr procedury



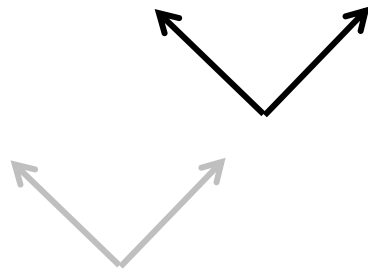
Kolejne kroki rysowania „palca”

Obróć układ o kąt zgięcia palca
(parametr) wokół osi Z



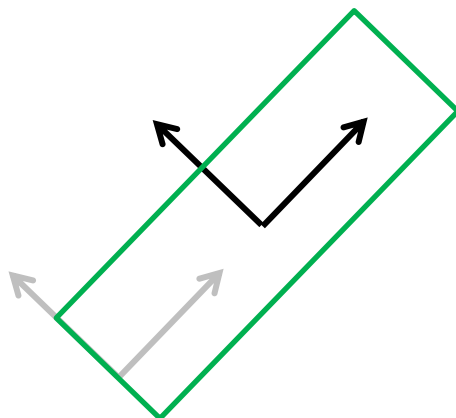
Kolejne kroki rysowania „palca”

Przesuń układ o vzdłuż osi X o 1 jednostkę



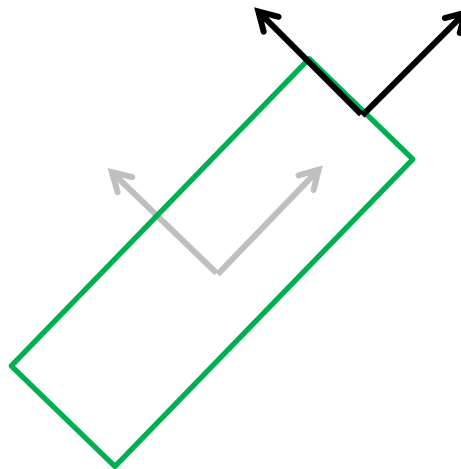
Kolejne kroki rysowania „palca”

Narysuj kostkę o wymiarach 2x0.5x1



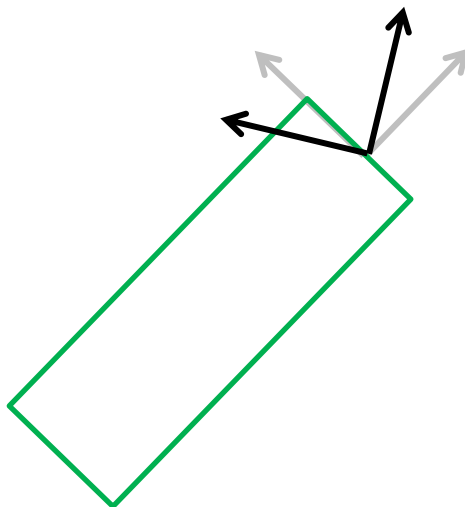
Kolejne kroki rysowania „palca”

Przesuń układ wzdłuż osi X o 1 jednostkę



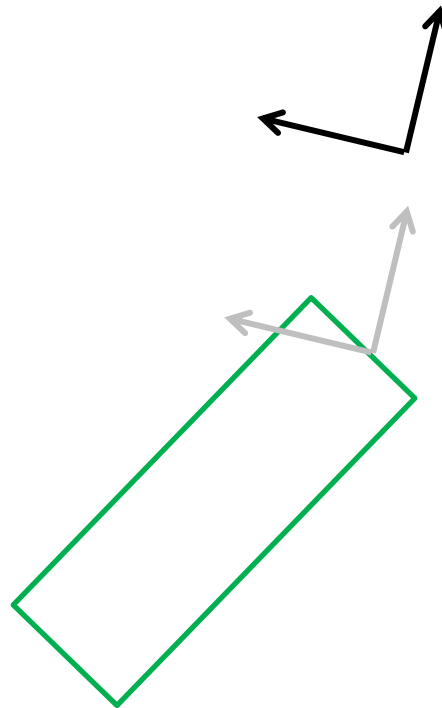
Kolejne kroki rysowania „palca”

Obróć układ o kąt zgięcia palca wokół osi Z



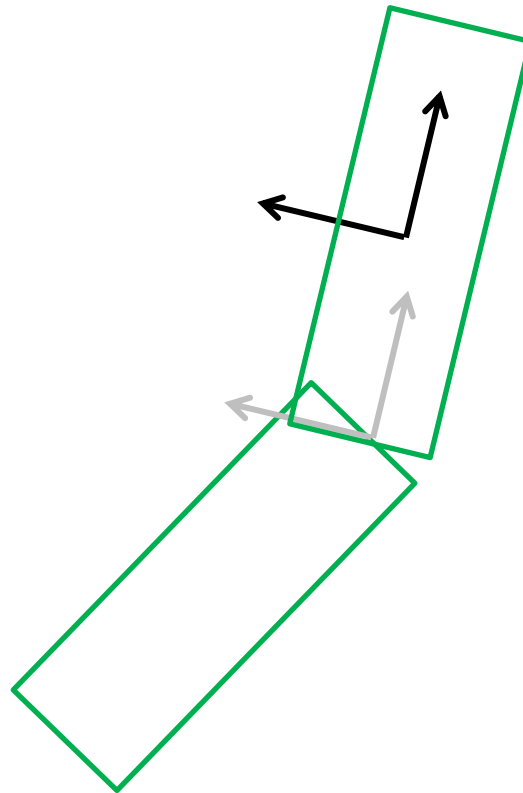
Kolejne kroki rysowania „palca”

Przesuń układ wzdłuż osi X o 1 jednostkę



Kolejne kroki rysowania „palca”

Narysuj kostkę o wymiarach 2x0.5x1



Ćwiczenie (3/3)

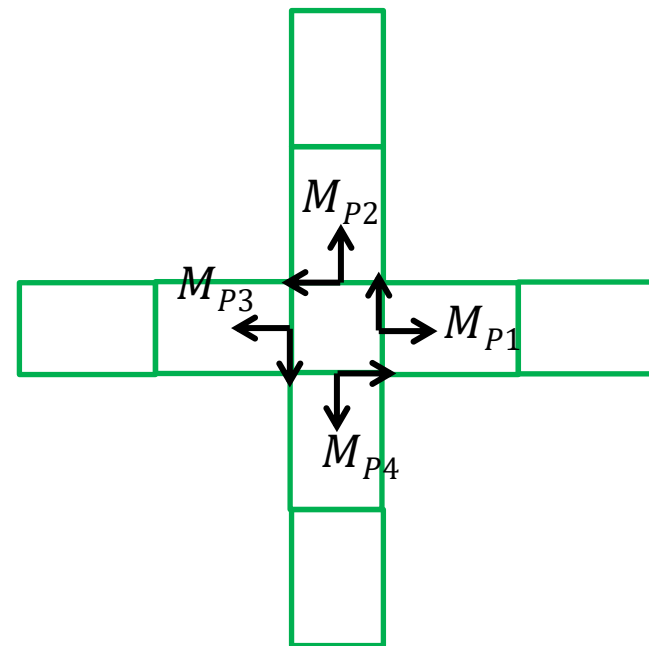
- W początku układu narysuj kostkę o wymiarach $1 \times 0.5 \times 1$
- Kolejne „palce” rysuj w układach współrzędnych wyliczanych następująco:

$$M_{P1} = T_x(0.5, 0, 0)$$

$$M_{P2} = R_Y(90)T_x(0.5, 0, 0)$$

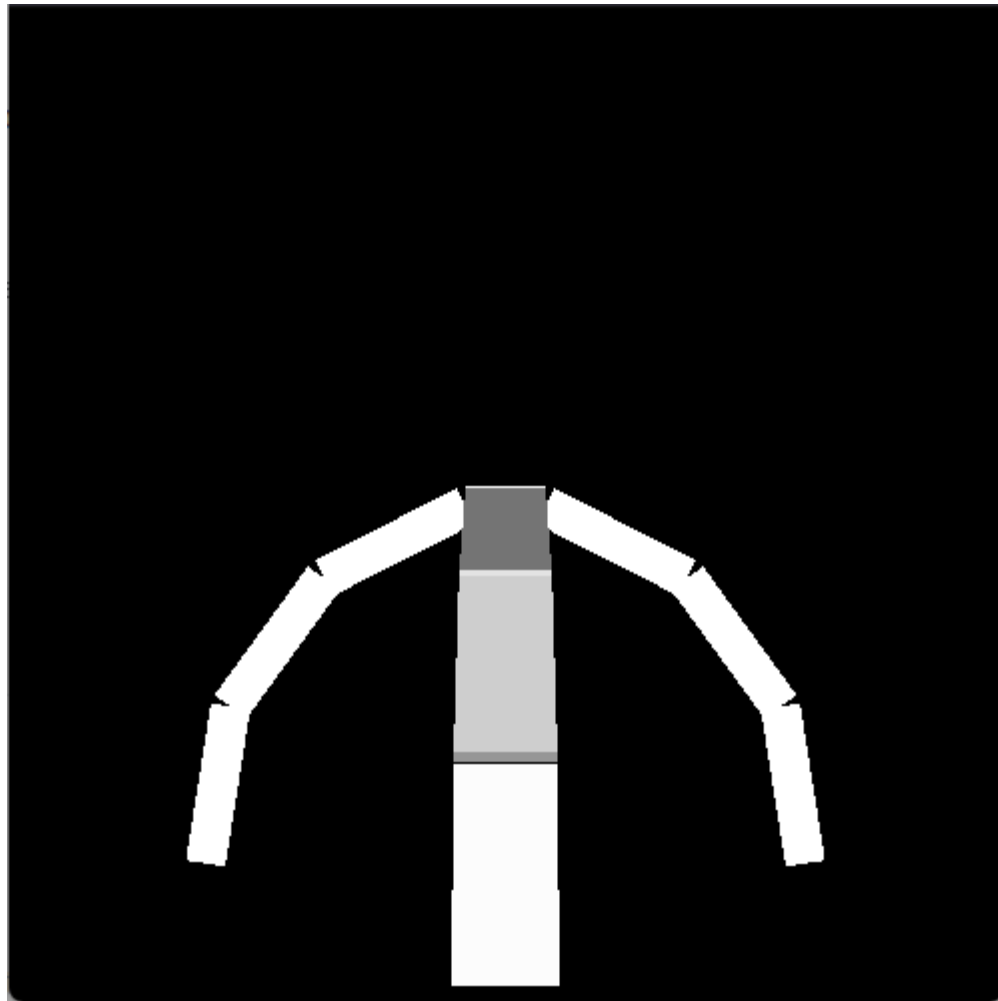
$$M_{P3} = R_Y(180)T_x(0.5, 0, 0)$$

$$M_{P4} = R_Y(270)T_x(0.5, 0, 0)$$



Ćwiczenie

- Dodaj dodatkowy paliczek do każdego palca.

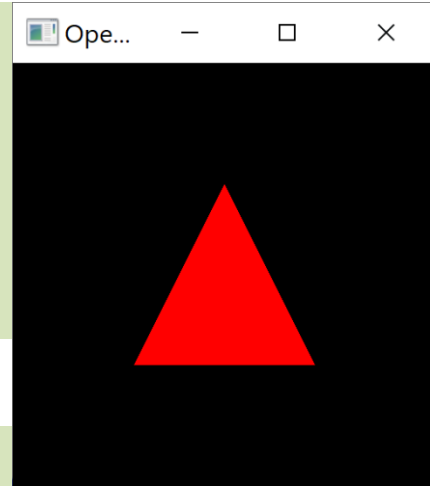


Rysowanie dowolnych obiektów

- Dotychczas korzystaliśmy z gotowych procedur, które rysowały całe obiekty – sześciany, torusy i inne.
- Można jednak narysować dowolny obiekt korzystając z takich składowych jak: punkty, linie i trójkąty.
- Aby narysować jakiś obiekt, można wykorzystać procedury *glDrawArrays* i *glDrawElements*.

Rysowanie z użyciem glDrawArrays

```
//Tablica współrzędnych wierzchołków
float verts[]={
    0, 1,0,1,
    -1,-1,0,1,
    1,-1,0,1
};
int vertexCount=3; //Liczba wierzchołków w tablicy
```



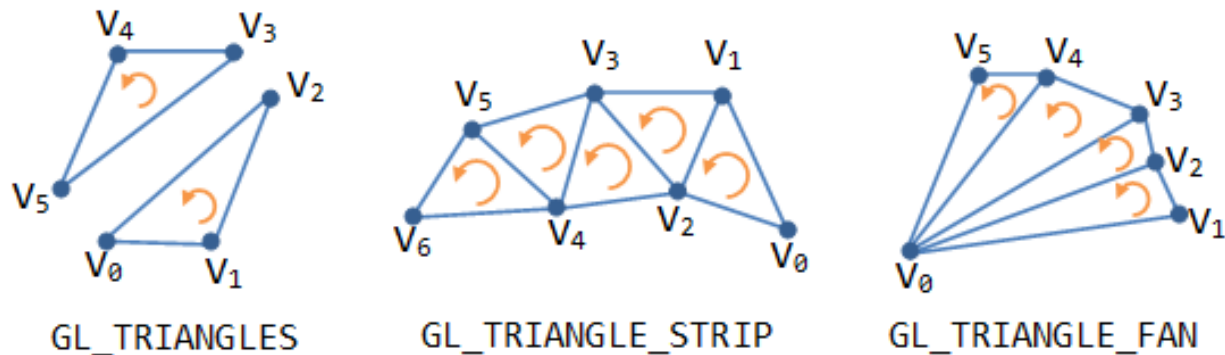
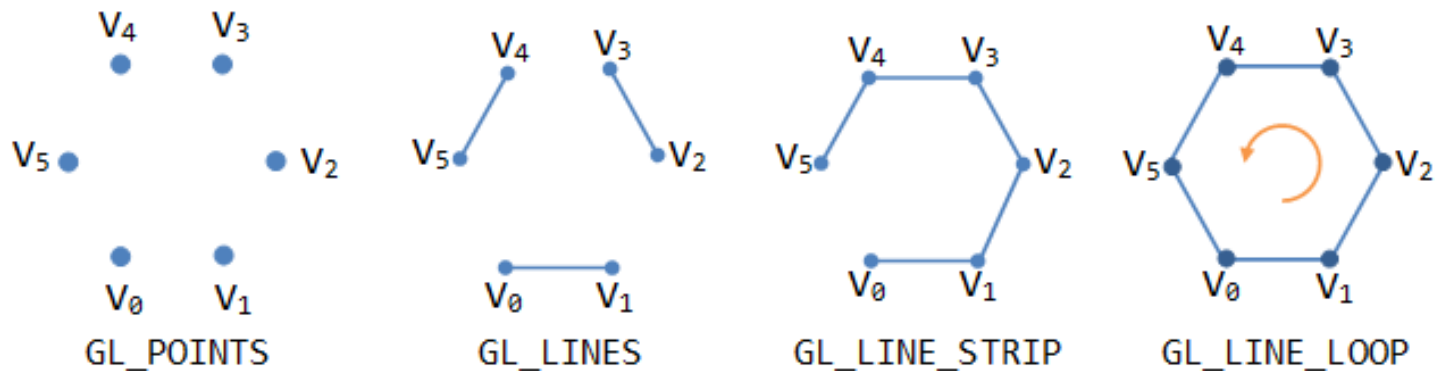
```
spConstant->use();
glUniformMatrix4fv(spConstant->u("P"),1,false,glm::value_ptr(P));
glUniformMatrix4fv(spConstant->u("V"),1,false,glm::value_ptr(V));
glUniformMatrix4fv(spConstant->u("M"),1,false,glm::value_ptr(M));
glUniform4f(spConstant->u("color"),1,0,0,1);

glEnableVertexAttribArray(spConstant->a("vertex"));
glVertexAttribPointer(spConstant->a("vertex"),4, GL_FLOAT, false, 0, verts);

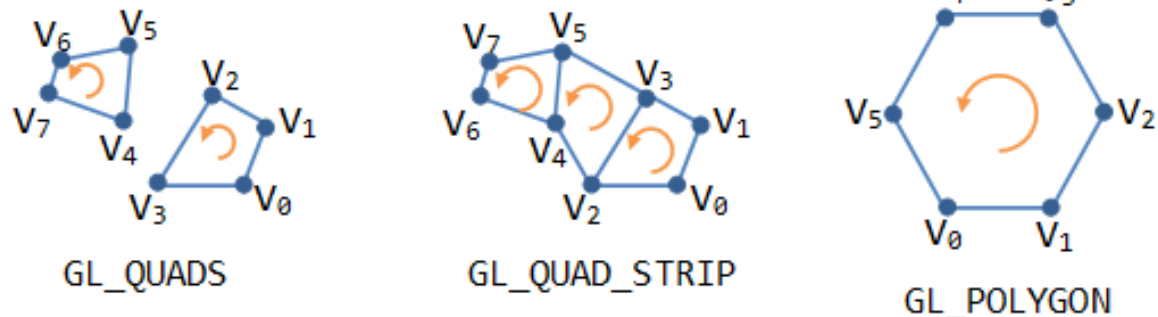
glDrawArrays( GL_TRIANGLES, 0, vertexCount );

glDisableVertexAttribArray(spConstant->a("vertex"));
```

Punkty i odcinki



Deprecated



Przykłady glDrawArrays

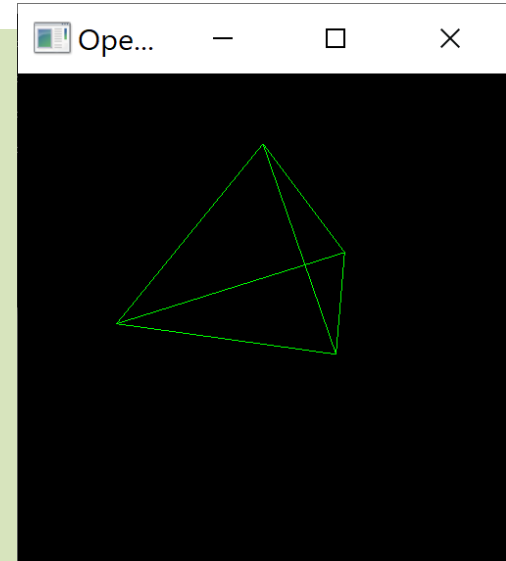
```
float verts[]={
    0,4.08,0,1,      0,0,2.88,1,
    0,4.08,0,1,      2.5,0,-1.44,1,
    0,4.08,0,1,      -2.5,0,-1.44,1,
    0,0,2.88,1,      -2.5,0,-1.44,1,
    0,0,2.88,1,      2.5,0,-1.44,1,
    -2.5,0,-1.44,1,  2.5,0,-1.44,1
};
int vertexCount=12;
.....

spConstant->use();
glUniformMatrix4fv(spConstant->u("P"),1,false,glm::value_ptr(P));
glUniformMatrix4fv(spConstant->u("V"),1,false,glm::value_ptr(V));
glUniformMatrix4fv(spConstant->u("M"),1,false,glm::value_ptr(M));
glUniform4f(spConstant->u("color"),0,1,0,1);

glEnableVertexAttribArray(spConstant->a("vertex"));
glVertexAttribPointer(spConstant->a("vertex"),4,GL_FLOAT,false,0,verts);

glDrawArrays( GL_LINES, 0, vertexCount );

glDisableVertexAttribArray(spConstant->a("vertex"));
```



Przykłady glDrawArrays

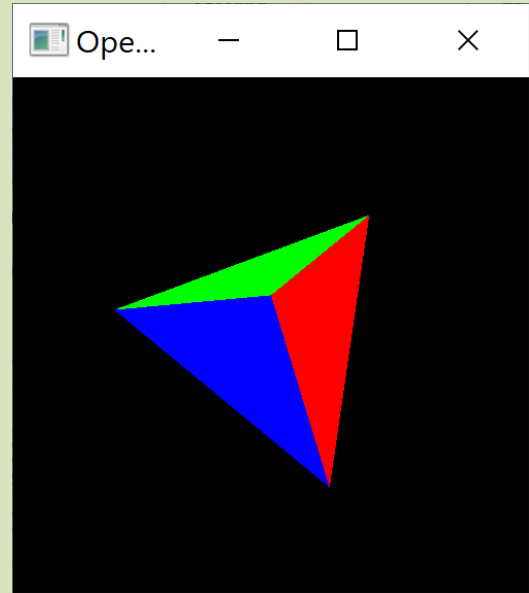
```
float verts[]={
    0,4.08,0,1,      0,0,2.88,1,      -2.5,0,-1.44,1,
    0,4.08,0,1,      0,0,2.88,1,      2.5,0,-1.44,1,
    0,4.08,0,1,      2.5,0,-1.44,1,  -2.5,0,-1.44,1,
    2.5,0,-1.44,1,  -2.5,0,-1.44,1,   0,0,2.88,1};
float colors[]={
    1,0,0,1,  1,0,0,1,  1,0,0,1,
    0,1,0,1,  0,1,0,1,  0,1,0,1,
    0,0,1,1,  0,0,1,1,  0,0,1,1,
    1,1,0,1,  1,1,0,1,  1,1,0,1};
int vertexCount=12;
... ..
spColored->use();
glUniformMatrix4fv(spColored->u("P"),1,false,glm::value_ptr(P));
glUniformMatrix4fv(spColored->u("V"),1,false,glm::value_ptr(V));
glUniformMatrix4fv(spColored->u("M"),1,false,glm::value_ptr(M));

glEnableVertexAttribArray(spColored->a("vertex"));
glVertexAttribPointer(spColored->a("vertex"),4,GL_FLOAT,false,0,verts);

glEnableVertexAttribArray(spColored->a("color"));
glVertexAttribPointer(spColored->a("color"),4,GL_FLOAT,false,0,colors);

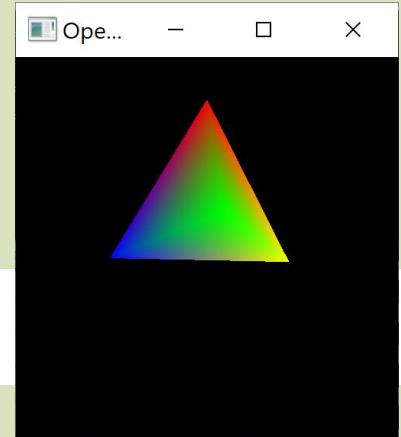
glDrawArrays( GL_TRIANGLES, 0, vertexCount );

glDisableVertexAttribArray(spColored->a("vertex"));
glDisableVertexAttribArray(spColored->a("color"));
```



Rysowanie z użyciem glDrawElements

```
float verts[]={
    0,4.08,0,1,      0,0,2.88,1,      -2.5,0,-1.44,1,      2.5,0,-1.44,1};
float colors[]={
    1,0,0,1,  0,1,0,1,  0,0,1,1,  1,1,0,1};
unsigned int indexes[] ={
    0,1,2,  0,1,3,  0,2,3,  3,2,1};
int indexCount=12;
int vertexCount=4;
```



```
spColored->use();
glUniformMatrix4fv(spColored->u("P"),1,false,glm::value_ptr(P));
glUniformMatrix4fv(spColored->u("V"),1,false,glm::value_ptr(V));
glUniformMatrix4fv(spColored->u("M"),1,false,glm::value_ptr(M));

glEnableVertexAttribArray(spColored->a("vertex"));
glVertexAttribPointer(spColored->a("vertex"),4,GL_FLOAT,false,0,verts);

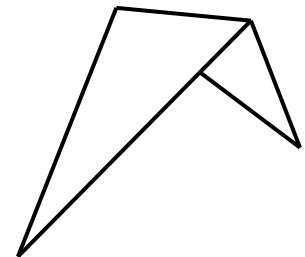
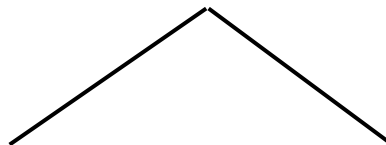
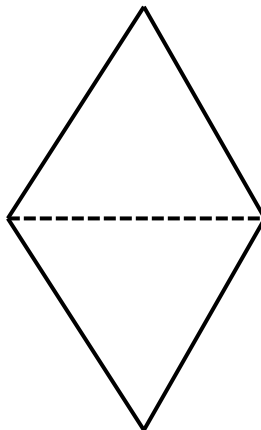
glEnableVertexAttribArray(spColored->a("color"));
glVertexAttribPointer(spColored->a("color"),4,GL_FLOAT,false,0,colors);

glDrawElements(GL_TRIANGLES,indexCount,GL_UNSIGNED_INT,indexes);

glDisableVertexAttribArray(spColored->a("vertex"));
glDisableVertexAttribArray(spColored->a("color"));
```

Zadanie

- Użyj programu cieniującego `spColored`
- Narysuj „ptaszka” nadając kształt taki jak poniżej i jakieś kolory



Zadanie

- Narysuj kostkę wykorzystując tablice `myCubeVertices` i `myCubeColors` z pliku `myCube.h`
- Użyj programu cieniującego `spColored`
- Użyj `glDrawArrays(GL_TRIANGLES,...`

Wczytanie obrazka

- Wczytanie obrazka do pamięci komputera.
 - Dowolna zewnętrzna biblioteka
- Zaimportowanie obrazka do OpenGL
 - Najczęściej do pamięci karty graficznej

LodePNG

- Malutka biblioteka do odczytu i zapisu plików PNG
- W najprostszej postaci wystarczy użyć jednej procedury, żeby odczytać plik:

```
std::vector<unsigned char> image;  
unsigned width, height;  
unsigned error = lodepng::decode(image, width, height, "plik.png");
```

- Domyślnie dekodowany do formatu RGBA, 1B na kanał.

Wczytanie obrazka do OpenGL

- Kiedy obraz zostanie wczytany do pamięci należy go zaimportować do OpenGL.
- Wykonuje się to za pomocą czterech funkcji OpenGL:
 - `glGenTextures()`
 - `glActiveTexture()`
 - `glBindTexture()`
 - `glTexImage2D()`

glGenTextures

- Służy do inicjowania uchwytów tekstur.

```
void glGenTextures(GLsizei n, GLuint *textures)
```

- n – liczba uchwytów do zainicjowania
- textures – tablica zmiennych typu GLuint, która ma przechowywać nowe uchwyty

- Przykłady

```
GLuint tex; //Uchwyt  
.....  
glGenTextures(1,&tex); //Zainicjuj jeden chwyt
```

```
GLuint tex[10]; //Uchwyt  
.....  
glGenTextures(10,tex); //Zainicjuj 10 uchwytów
```

glActiveTexture

- Wybór jednostki teksturującej, która ma być wykorzystywana przez kolejne polecenia
- Jednostka karty graficznej zajmująca się próbkowaniem i przetwarzaniem tekstur
- Różna liczba jednostek w zależności od modelu karty graficznej
- Identyfikowane przez stałe GL_TEXTUREx

```
void glActiveTexture(GLenum texture);
```

- Przykład

```
glActiveTexture(GL_TEXTURE0);
```

glBindTexture

- Wiąże uchwyt tekstury z wybraną jednostką teksturującą:

```
void glBindTexture(GLenum target, GLuint texture);
```

- target – typ tekstury, u nas GL_TEXTURE_2D
 - texture – uchwyt, który ma być aktywny
- Obrazek tekstury można skopiować do obszaru pamięci reprezentowanego przez uchwyt powiązany z wybraną jednostką teksturującą
- W trakcie rysowania używane są tekstury powiązane z jednostkami teksturującymi
- Przykład:

```
glBindTexture(GL_TEXTURE_2D, tex);
```

glTexImage2D

- Kopiuje obrazek z pamięci komputera do pamięci GPU reprezentowanej przez uchwyt
- Kopiuje dane z pamięci operacyjnej. Po wykonaniu można je usunąć.

```
void glTexImage2D( GLenum target, GLint level,  
                  GLint internalformat, GLsizei width, GLsizei height,  
                  GLint border, GLenum format, GLenum type,  
                  const GLvoid *pixels );
```

- target – na zajęciach zawsze GL_TEXTURE_2D
- level – poziom mipmapy, dla nas zawsze 0
- internalformat – reprezentacja wewnętrzna (format składowania w pamięci karty graficznej):
 - Dla obrazków 32 bitowych: GL_RGBA
 - Dla obrazków 24 bitowych: GL_RGB

glTexImage2D

- width – szerokość obr. (potęga 2 dla MIP mapping)
 - height – wysokość obr. (potęga 2 dla MIP mapping)
 - border – (**deprecated**) na zajęciach zawsze 0
 - format (wejściowy): np. GL_RGBA, GL_RGB
 - type – na zajęciach GL_UNSIGNED_BYTE
 - pixels – wskaźnik do obszaru pamięci, w którym zapisano dane obrazka
- Przykład:

```
glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA,  
width, height, 0,  
GL_RGBA, GL_UNSIGNED_BYTE,  
(unsigned char*) image.data());
```

glDeleteTextures

- Służy usuwania uchwytów tekstur i związanych z nimi obrazków.

```
void glDeleteTextures(GLsizei n, GLuint *textures)
```

- n – liczba uchwytów do usunięcia
- textures – tablica zmiennych typu GLuint, która przechowuje uchwyty do usunięcia

- Przykłady

```
GLuint tex; //Uchwyty
.....
glDeleteTextures(1,&tex); //Usuń jeden uchwyt
```

```
GLuint tex[10]; //Uchwyty
.....
glDeleteTextures(10,tex); //Usuń 10 uchwytów
```


Parametry teksturowania

- Aby móc używać tekstur przy rysowaniu obrazów należy jeszcze ustawić algorytm próbkowania kolorów

```
//Bilinear filtering  
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);  
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
```

- Polecenia te dotyczą tekstury powiązanej z uchwytem przypisanym (glBindTexture) do wybranej jednostki teksturującej (glActiveTexture)
 - Ustawienia są zapamiętywane razem z teksturą
 - Jeżeli wczytujemy więcej niż jeden obrazek, trzeba je wykonać dla każdego z nich.

Podsumowanie

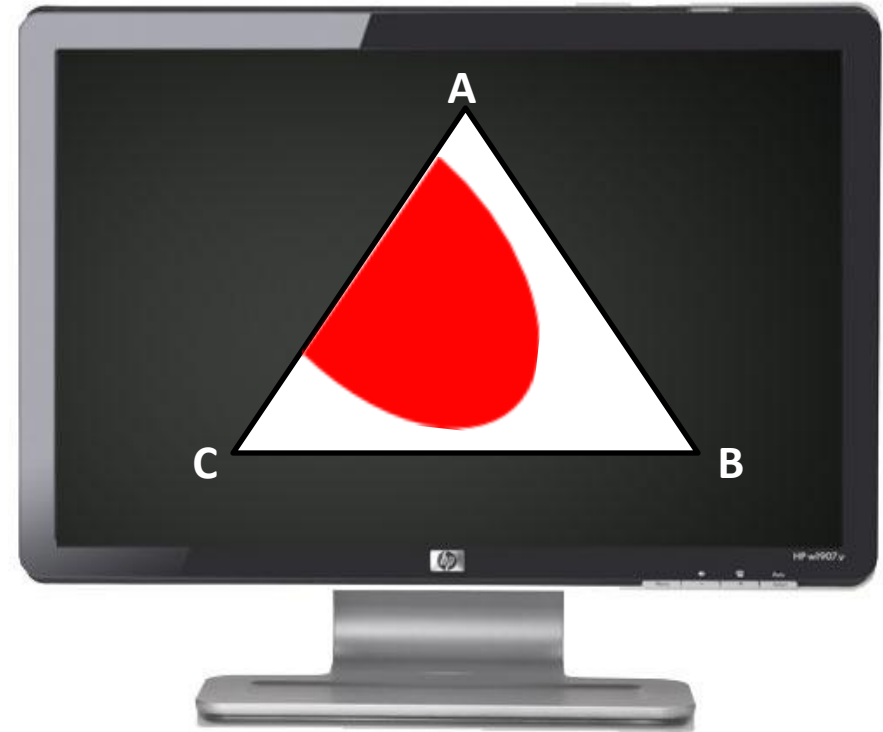
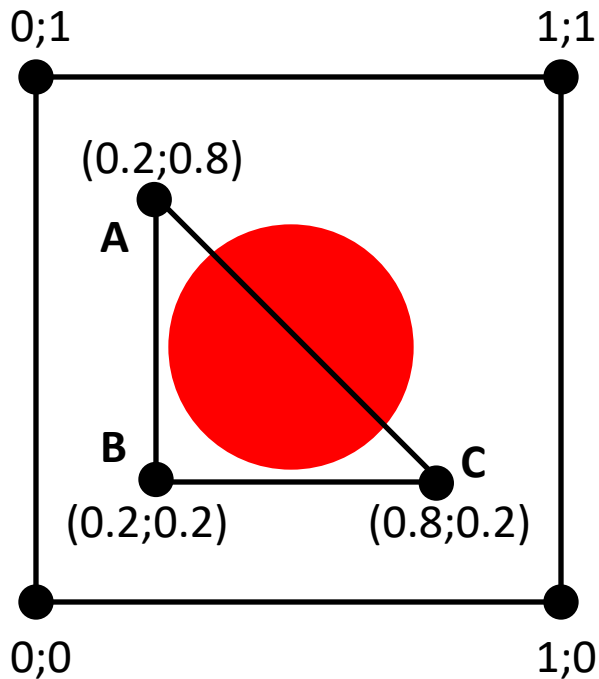
```
GLuint tex; //Uchwył - deklaracja globalna
```

```
GLuint readTexture(const char* filename) { //Deklaracja globalna  
    GLuint tex;  
    glActiveTexture(GL_TEXTURE0);  
  
    //Wczytanie do pamięci komputera  
    std::vector<unsigned char> image;    //Alokuj wektor do wczytania obrazka  
    unsigned width, height;    //Zmienne do których wczytamy wymiary obrazka  
    //Wczytaj obrazek  
    unsigned error = lodepng::decode(image, width, height, filename);  
  
    //Import do pamięci karty graficznej  
    glGenTextures(1,&tex); //Zainicjuj jeden uchwył  
    glBindTexture(GL_TEXTURE_2D, tex); //Uaktywnij uchwył  
    //Wczytaj obrazek do pamięci KG skojarzonej z uchwytem  
    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, width, height, 0,  
        GL_RGBA, GL_UNSIGNED_BYTE, (unsigned char*) image.data());  
  
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);  
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);  
  
    return tex;  
}
```

```
//Wczytanie i import obrazka - w initOpenGLProgram  
tex=readTexture("bricks.png");
```

```
//Usunięcie tekstury z pamięci karty graficznej - w freeOpenGLProgram  
glDeleteTextures(1,&tex);
```

Współrzędne teksturowania



Przykład

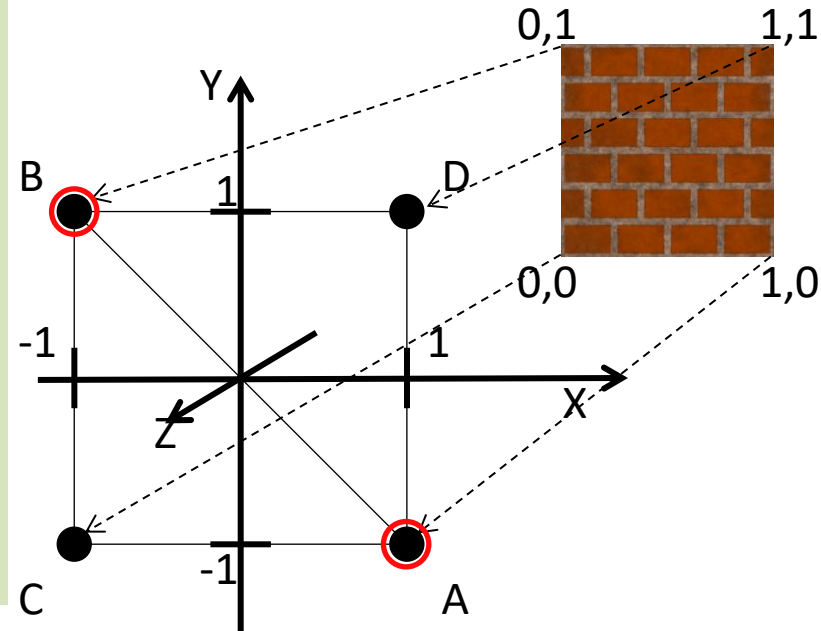
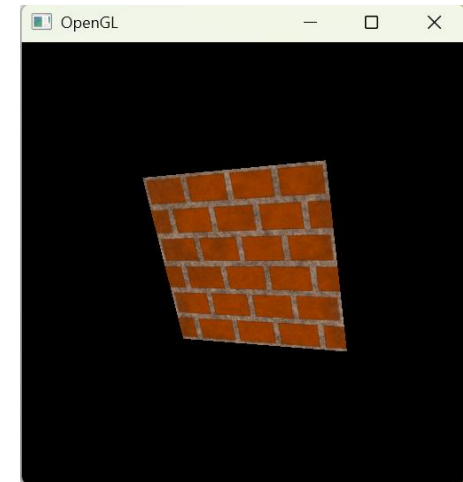
```
//Tablica współrzędnych wierzchołków
float verts[]={
    1.0f,-1.0f,0.0f,1.0f, //A
    -1.0f, 1.0f,0.0f,1.0f, //B
    -1.0f,-1.0f,0.0f,1.0f, //C

    1.0f,-1.0f,0.0f,1.0f, //A
    1.0f, 1.0f,0.0f,1.0f, //D
    -1.0f, 1.0f,0.0f,1.0f, //B
};
```

```
//Tablica współrzędnych teksturowania
float texCoords[]={
    1.0f, 0.0f, //A
    0.0f, 1.0f, //B
    0.0f, 0.0f, //C

    1.0f, 0.0f, //A
    1.0f, 1.0f, //D
    0.0f, 1.0f, //B
};
```

```
//Liczba wierzchołków w tablicy
int vertexCount=6;
```



Przykład

```
//Tablica współrzędnych wierzchołków
float verts[]={
    1.0f,-1.0f,0.0f,1.0f, //A
    -1.0f, 1.0f,0.0f,1.0f, //B
    -1.0f,-1.0f,0.0f,1.0f, //C

    1.0f,-1.0f,0.0f,1.0f, //A
    1.0f, 1.0f,0.0f,1.0f, //D
    -1.0f, 1.0f,0.0f,1.0f, //B
};
```

```
//Tablica współrzędnych teksturowania
float texCoords[]={
    1.0f, 0.0f,      //A
    0.0f, 1.0f,      //B
    0.0f, 0.0f,      //C

    1.0f, 0.0f,      //A
    1.0f, 1.0f,      //D
    0.0f, 1.0f,      //B
};
//Liczba wierzchołków w tablicy
int vertexCount=6;
```

```
spTextured->use();
glUniformMatrix4fv(spTextured->u("P"),1,false,glm::value_ptr(P));
glUniformMatrix4fv(spTextured->u("V"),1,false,glm::value_ptr(V));
glUniformMatrix4fv(spTextured->u("M"),1,false,glm::value_ptr(M));

glEnableVertexAttribArray(spTextured->a("vertex"));
glVertexAttribPointer(spTextured->a("vertex"),4,GL_FLOAT,false,0,verts);

glEnableVertexAttribArray(spTextured->a("texCoord"));
glVertexAttribPointer(spTextured->a("texCoord"),2,GL_FLOAT,false,0,texCoords);

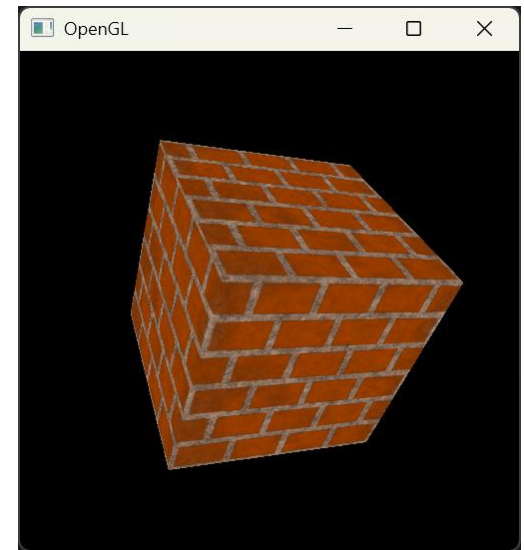
glActiveTexture(GL_TEXTURE0); glBindTexture(GL_TEXTURE_2D,tex);
glUniform1i(spTextured->u("tex"),0);

glDrawArrays( GL_TRIANGLES, 0, vertexCount );

glDisableVertexAttribArray(spTextured->a("vertex"));
glDisableVertexAttribArray(spTextured->a("texCoord"));
```

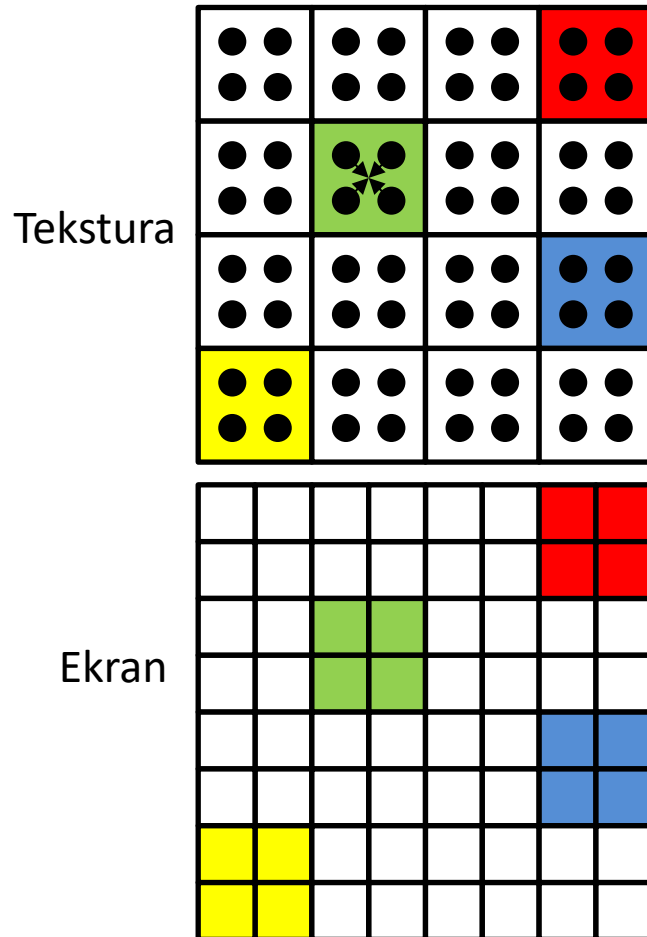
Zadanie

- Włącz plik nagłówkowy myCube.h
 - Plik zawiera tablicę współrzędnych wierzchołków oraz liczbę wierzchołków w tej tablicy
- Usuń polecenia dotyczące ustawiania kolorów kostki
- Dodaj tablicę współrzędnych teksturowania
- Wczytaj obrazek „bricks_diffuse.png”
- Wyświetl oteksturowaną kostkę.

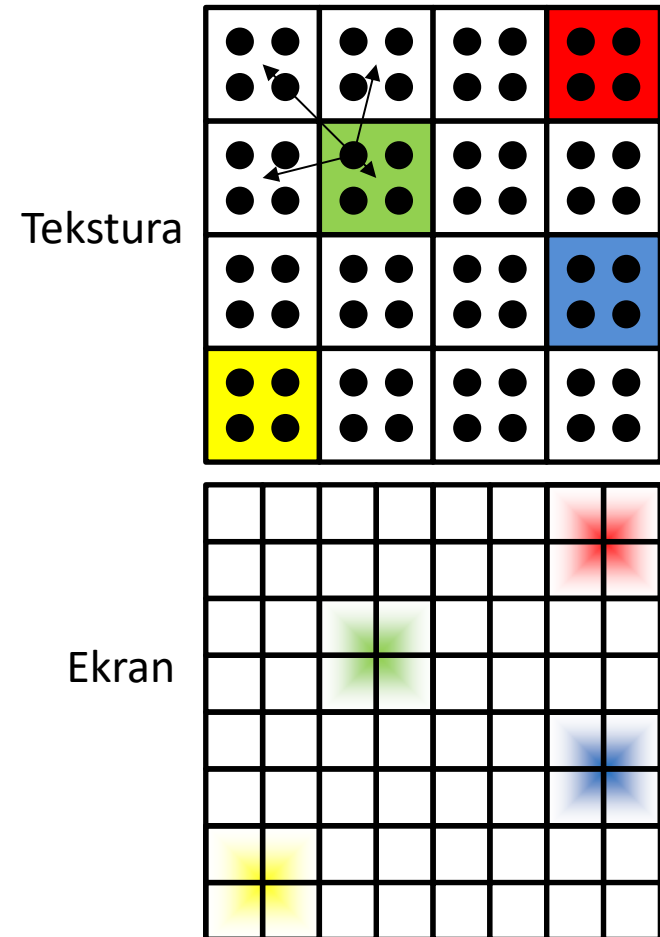


Powiększanie tekseli

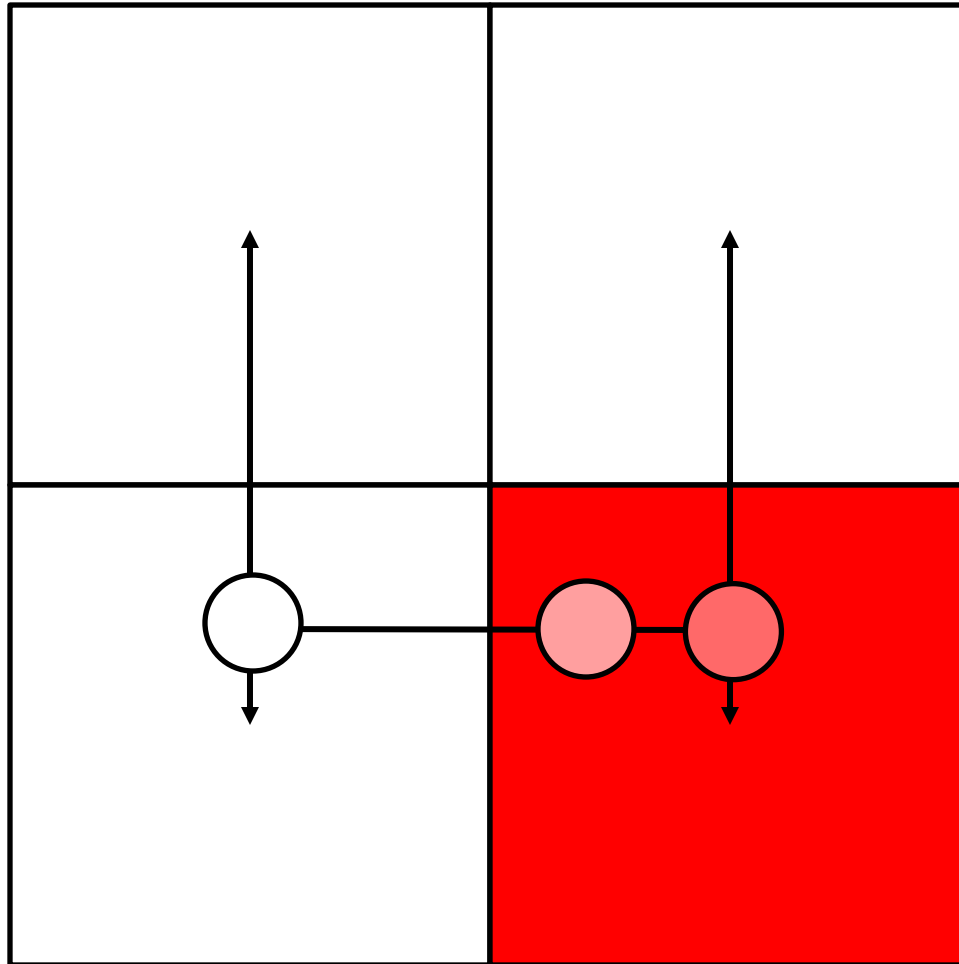
```
glTexParameteri(GL_TEXTURE_2D,  
GL_TEXTURE_MAG_FILTER,  
GL_NEAREST);
```



```
glTexParameteri(GL_TEXTURE_2D,  
GL_TEXTURE_MAG_FILTER,  
GL_LINEAR);
```



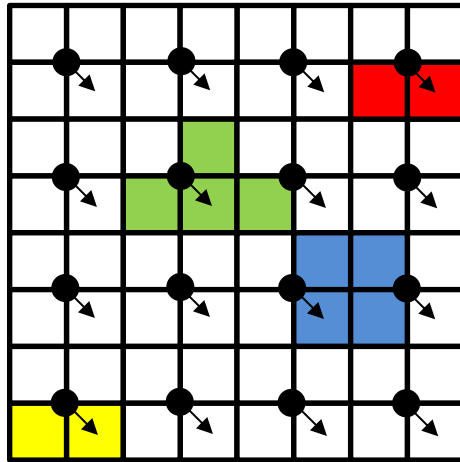
Bilinear filtering



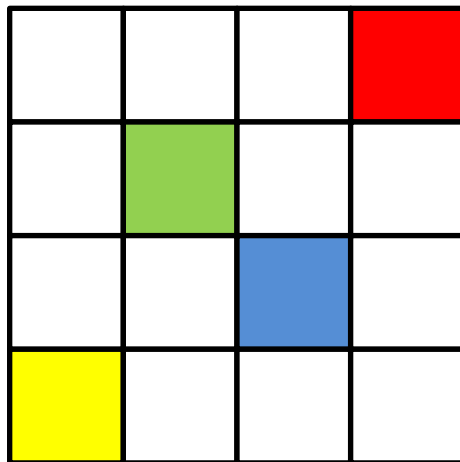
Pomniejszanie tekseli

```
glTexParameteri(GL_TEXTURE_2D,  
GL_TEXTURE_MIN_FILTER,  
GL_NEAREST);
```

Tekstura

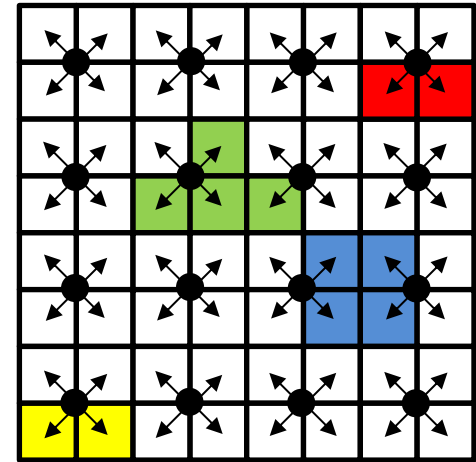


Ekran

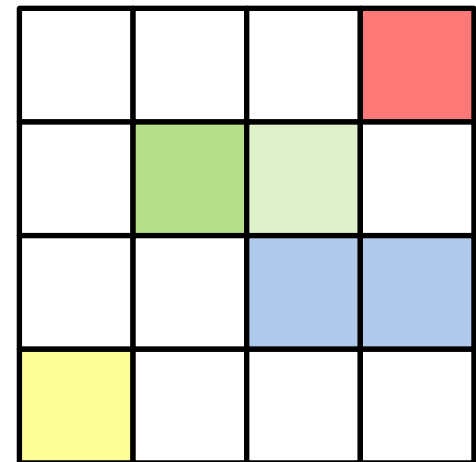


```
glTexParameteri(GL_TEXTURE_2D,  
GL_TEXTURE_MIN_FILTER,  
GL_LINEAR);
```

Tekstura



Ekran



Zadanie

- Zapamiętaj obecny stan projektu – będziemy chcieli do niego wrócić.
- Wczytaj teksturę tester.png
- Wykonaj następujące zmiany:
 - Położenie kamery: [0, 1.5, -1.5], punkt na który kamera patrzy: [0,1,0].
 - Bliska płaszczyzna odcinania w odległości 0.01 (fun. glm::perspective)
 - Zmień linijkę rysującą kostkę na `glDrawArrays(GL_TRIANGLES,18,6)`;
- Oprogramuj 4 klawisze tak, aby wciśnięcie każdego z nich wykonywało jedno z poleceń przedstawionych poniżej.

```
glTexParameteri(GL_TEXTURE_2D,  
    GL_TEXTURE_MAG_FILTER,  
    GL_NEAREST);
```

```
glTexParameteri(GL_TEXTURE_2D,  
    GL_TEXTURE_MAG_FILTER,  
    GL_LINEAR);
```

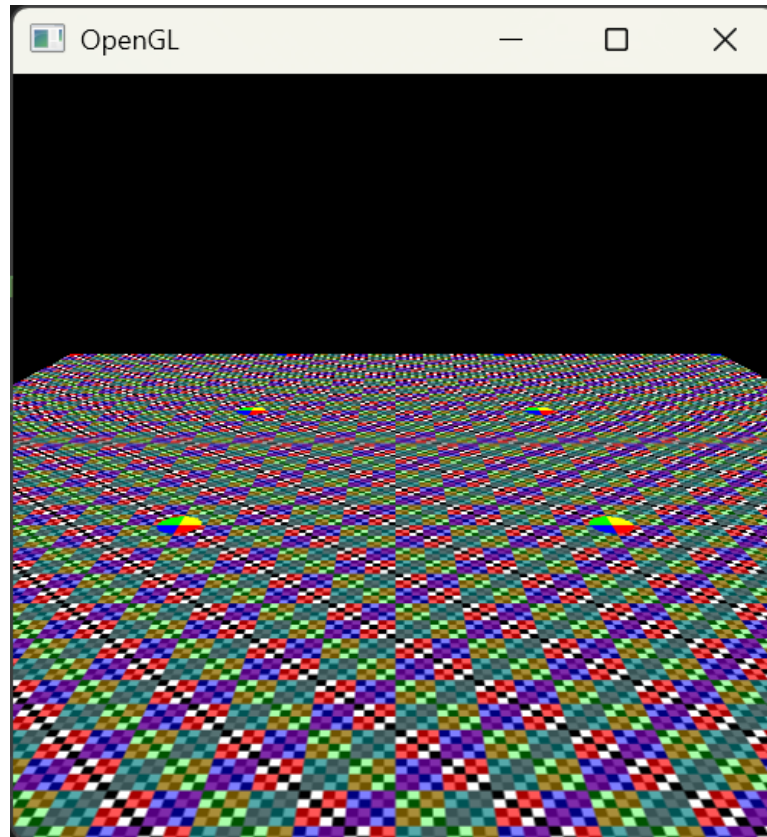
```
glTexParameteri(GL_TEXTURE_2D,  
    GL_TEXTURE_MIN_FILTER,  
    GL_NEAREST);
```

```
glTexParameteri(GL_TEXTURE_2D,  
    GL_TEXTURE_MIN_FILTER,  
    GL_LINEAR);
```

- Obejrzyj różnice w rysowanym kwadracie w zależności od włączonych algorytmów powiększania/pomniejszania tekstei.

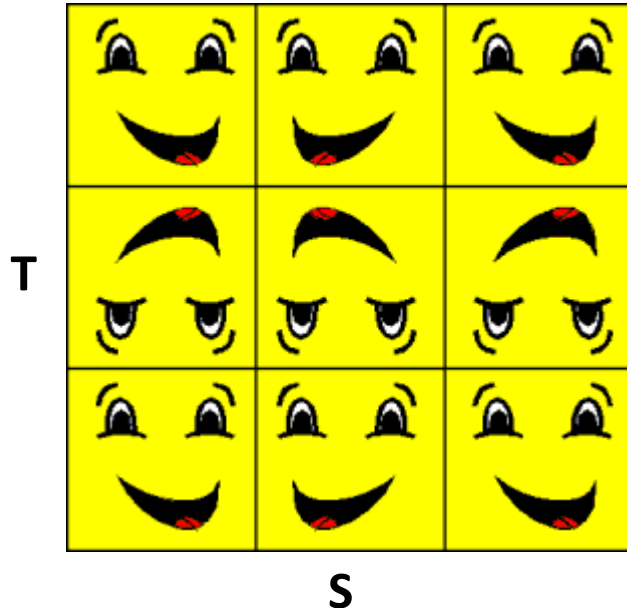
Zadanie

- Zmodyfikuj tablicę współrzędnych teksturowania tak, aby każda jedynka w tej tablicy była zamieniona na 3.



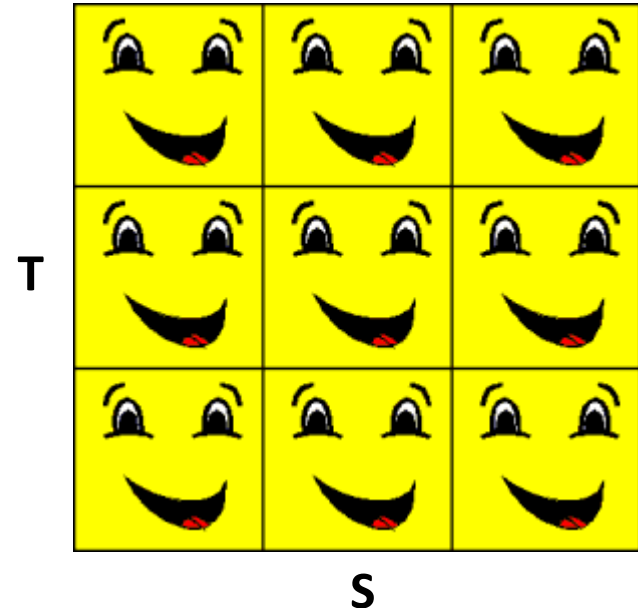
Adresowanie tekstur (1/2)

Mirror



```
glTexParameteri(GL_TEXTURE_2D,  
    GL_TEXTURE_WRAP_S,  
    GL_MIRRORED_REPEAT);  
  
glTexParameteri(GL_TEXTURE_2D,  
    GL_TEXTURE_WRAP_T,  
    GL_MIRRORED_REPEAT);
```

Repeat



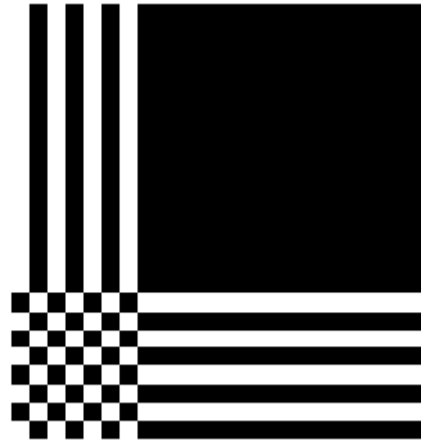
```
glTexParameteri(GL_TEXTURE_2D,  
    GL_TEXTURE_WRAP_S,  
    GL_REPEAT);  
  
glTexParameteri(GL_TEXTURE_2D,  
    GL_TEXTURE_WRAP_T,  
    GL_REPEAT);
```

Adresowanie tekstur (2/2)

Clamp to Edge



Texture



Clamped texture applied to primitive

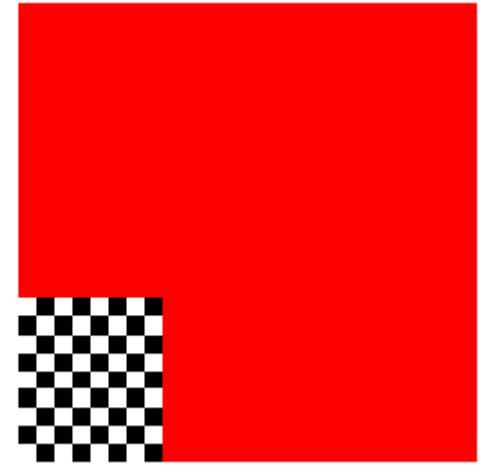
```
glTexParameteri(GL_TEXTURE_2D,  
    GL_TEXTURE_WRAP_S,  
    GL_CLAMP_TO_EDGE);
```

```
glTexParameteri(GL_TEXTURE_2D,  
    GL_TEXTURE_WRAP_T,  
    GL_CLAMP_TO_EDGE);
```

Clamp to Border



Texture



Texture with red border applied to primitive

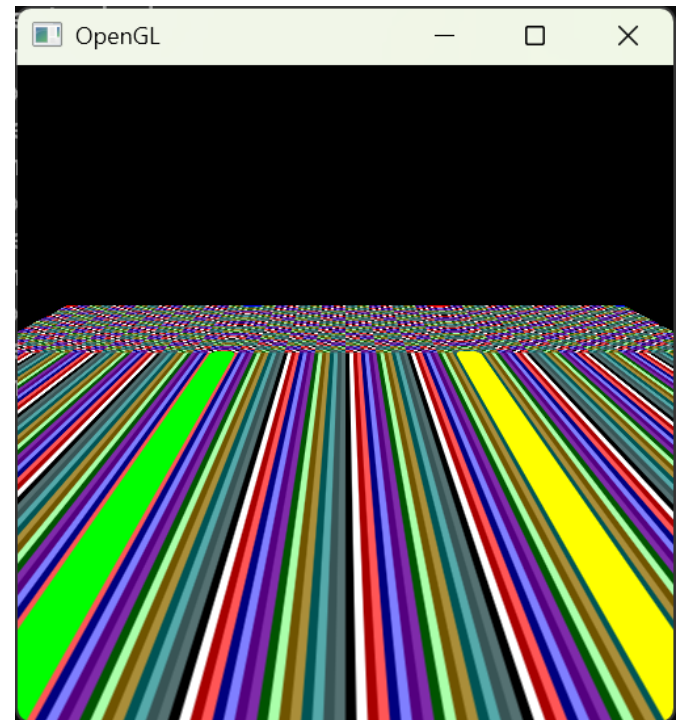
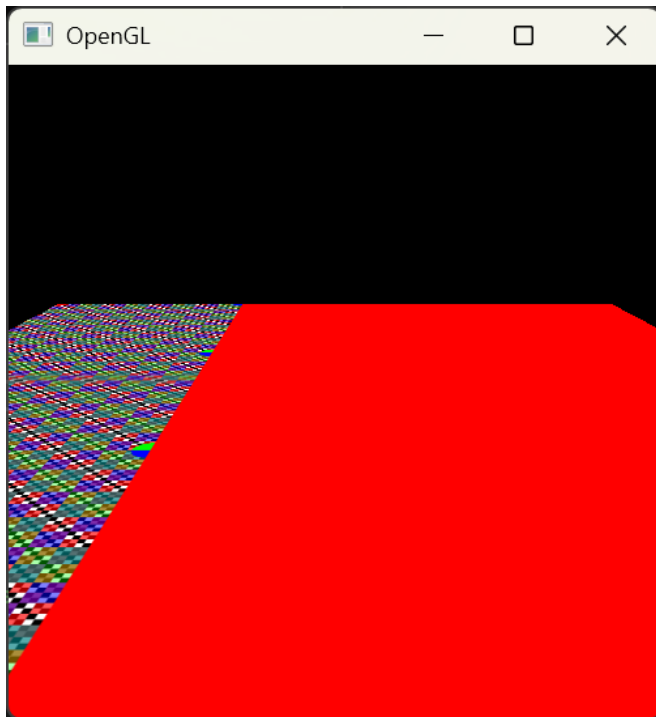
```
float col[]={1.0f,0.0f,0.0f,1.0f};  
glTexParameterfv(GL_TEXTURE_2D,  
    GL_TEXTURE_BORDER_COLOR,col);
```

```
glTexParameteri(GL_TEXTURE_2D,  
    GL_TEXTURE_WRAP_S,  
    GL_CLAMP_TO_BORDER);
```

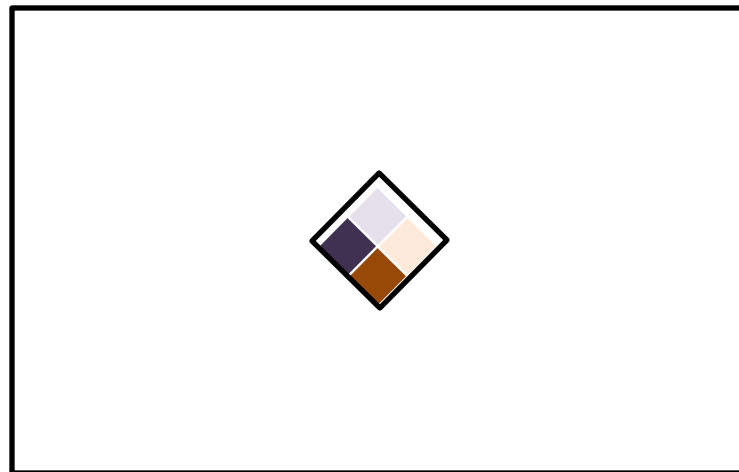
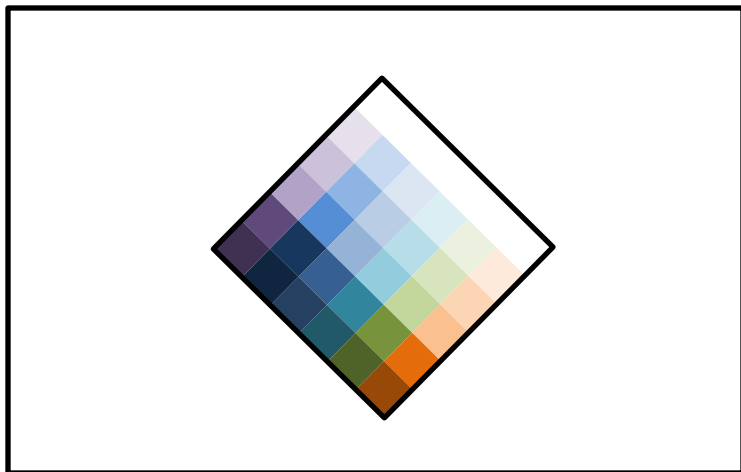
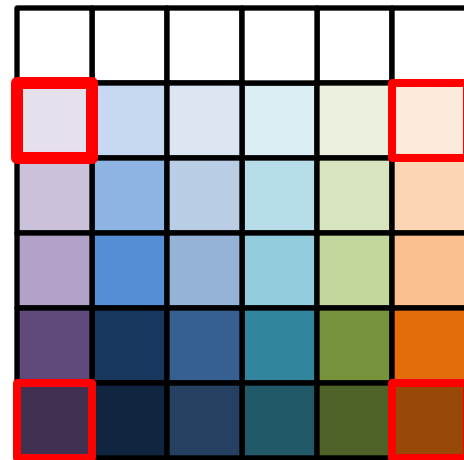
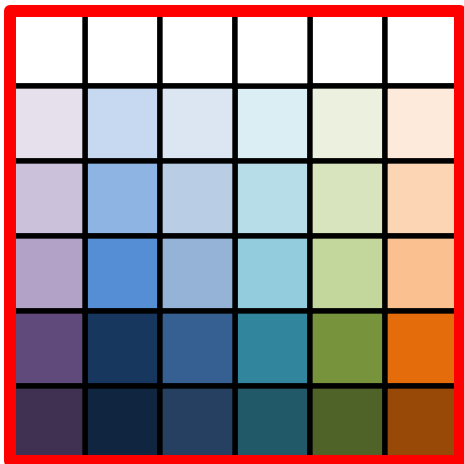
```
glTexParameteri(GL_TEXTURE_2D,  
    GL_TEXTURE_WRAP_T,  
    GL_CLAMP_TO_BORDER);
```


Zadanie

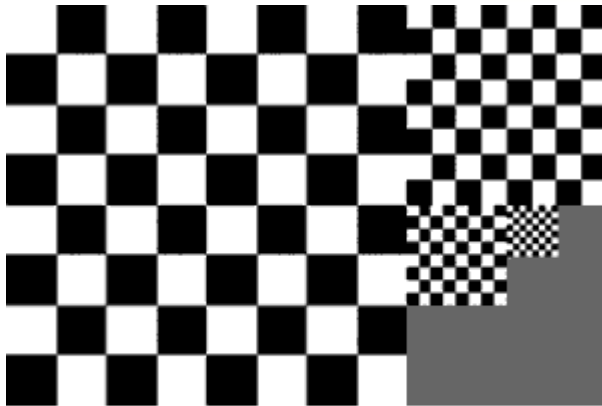
- Przetestuj różne metody adresowania tekstur.
- Zastosuj różne metody dla osi S i T.



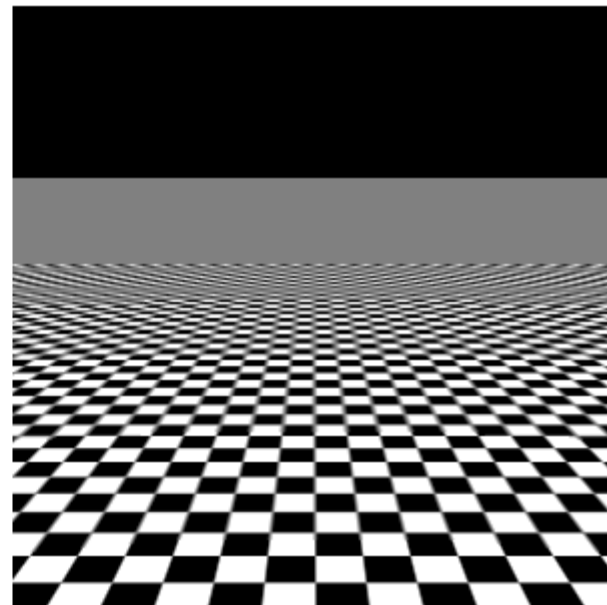
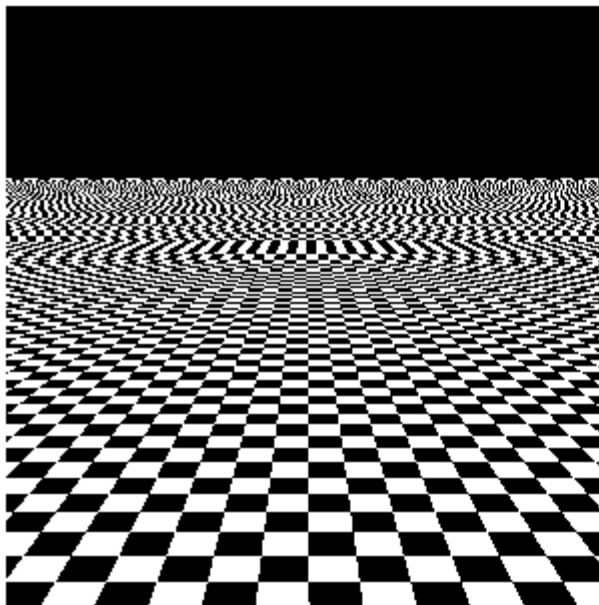
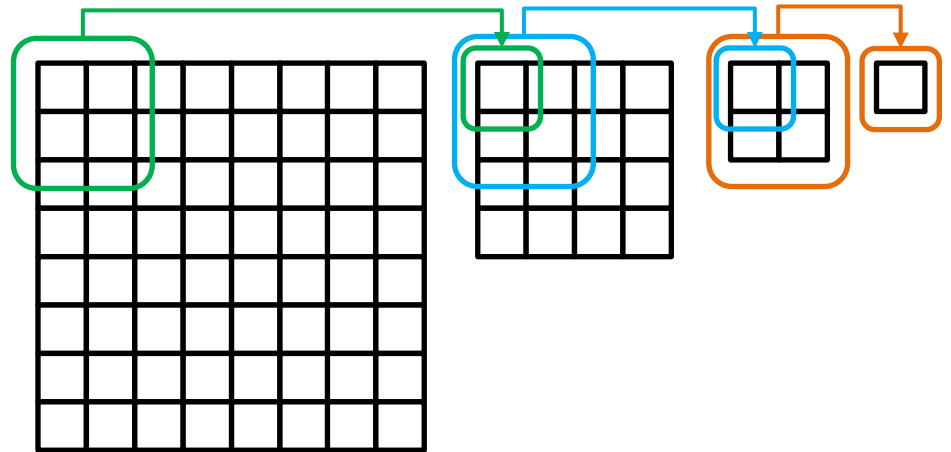
Bez MIP mappingu



Mip mapping



This simple MIP map holds four copies of a chequered texture, all at different scales



MIP Mapping

- OpenGL może sam wygenerować MIP-mapy tekstury.
- Aby to wykonać należy umieścić poniższe polecenie po `glTexImage2D`

```
glGenerateMipmap(GL_TEXTURE_2D);
```

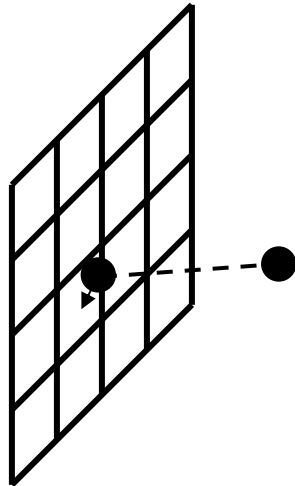
- Polecenie działa na teksturze przypisanej do wybranej jednostki teksturującej (`glActiveTexture+glBindTexture`)

MIP Mapping

- Aby OpenGL używał mip map, konieczne jest wybranie odpowiedniego algorytmu pomniejszania teksteli.
- Istnieją 4 algorytmy wykorzystujące mip mapy:
 - GL_NEAREST_MIPMAP_NEAREST
 - GL_LINEAR_MIPMAP_NEAREST
 - GL_NEAREST_MIPMAP_LINEAR
 - GL_LINEAR_MIPMAP_LINEAR

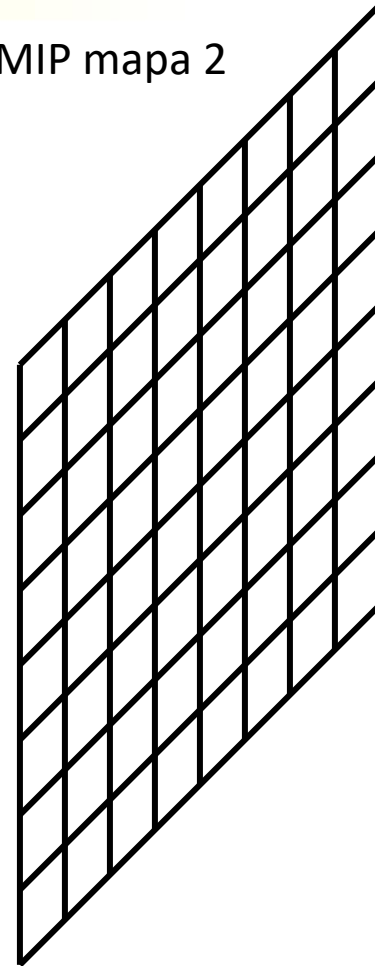
GL_NEAREST_MIPMAP_NEAREST

MIP mapa 1



Próbkowanie GL_NEAREST

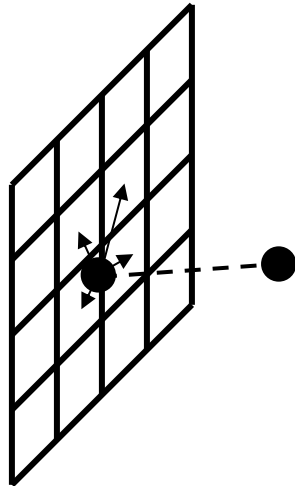
MIP mapa 2



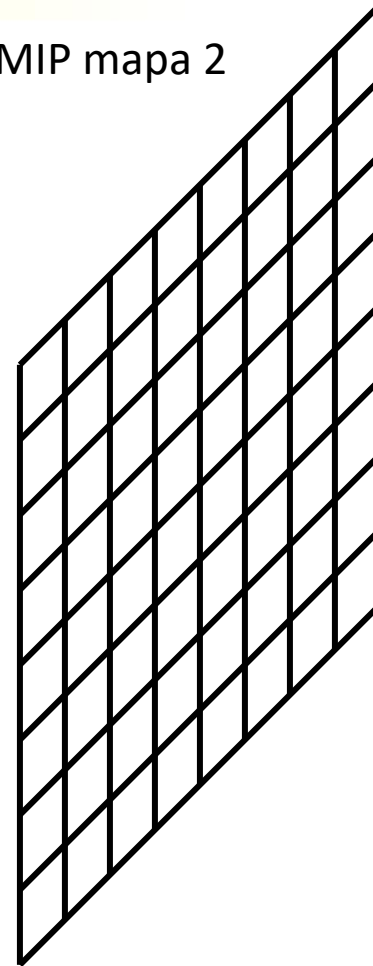
```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,  
                 GL_NEAREST_MIPMAP_NEAREST);
```

GL_LINEAR_MIPMAP_NEAREST

MIP mapa 1



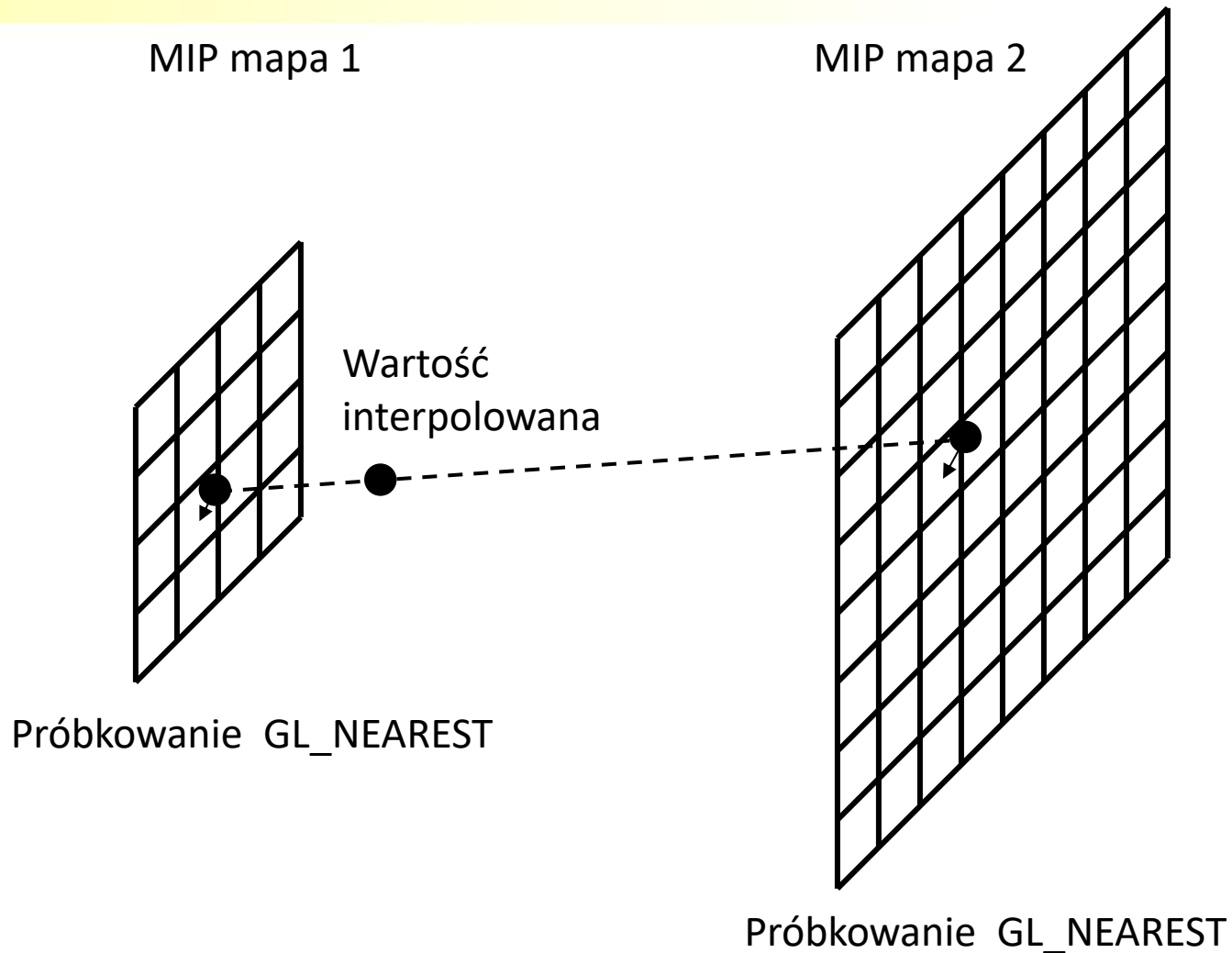
MIP mapa 2



Próbkowanie GL_LINEAR

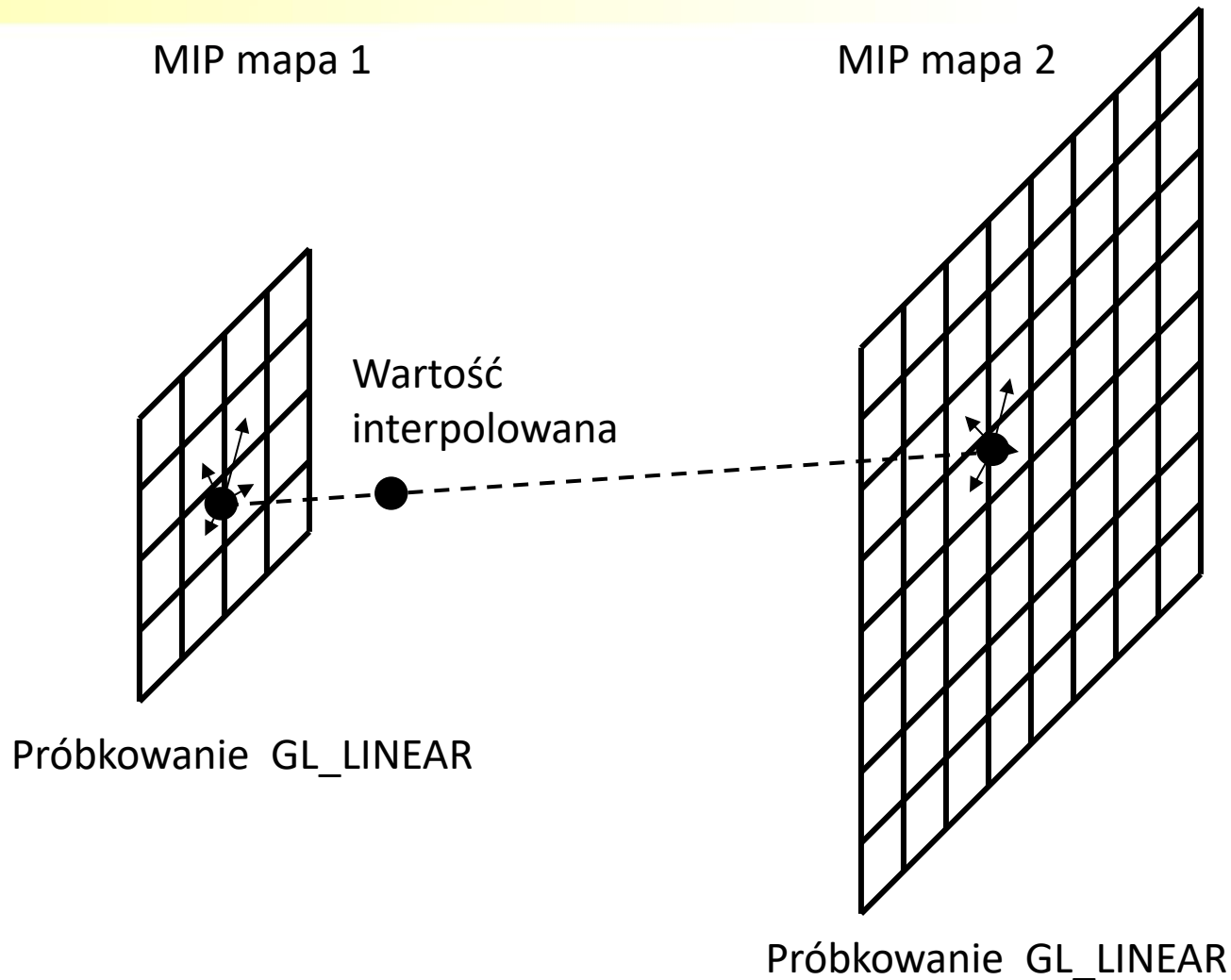
```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,  
                 GL_LINEAR_MIPMAP_NEAREST);
```

GL_NEAREST_MIPMAP_LINEAR



```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,  
                GL_NEAREST_MIPMAP_LINEAR);
```


GL_LINEAR_MIPMAP_LINEAR



```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,  
                GL_LINEAR_MIPMAP_LINEAR);
```

Zadanie

- Zmodyfikuj program:
 - Uzupełnij wczytywania o generowanie mip map.
 - Ustaw adresowanie repeat bądź mirror (zarówno na S jak i T).
 - Zmodyfikuj współrzędne teksturowania zamieniając każdą 3 na 20 tekseli. Dzięki temu będzie dominował algorytm pomniejszania.
 - Oprogramuj 4 klawisze tak aby pozwalały wybrać jedną z 4 metod uwzględniania mip map.
- Postaraj się dostrzec na rysunku granice pomiędzy mip mapami dla GL_NEAREST_MIPMAP_NEAREST i GL_LINEAR_MIPMAP_NEAREST
- Zaobserwuj jak one znikają jeżeli użyć ich odpowiedników GL_NEAREST_MIPMAP_LINEAR i GL_LINEAR_MIPMAP_LINEAR

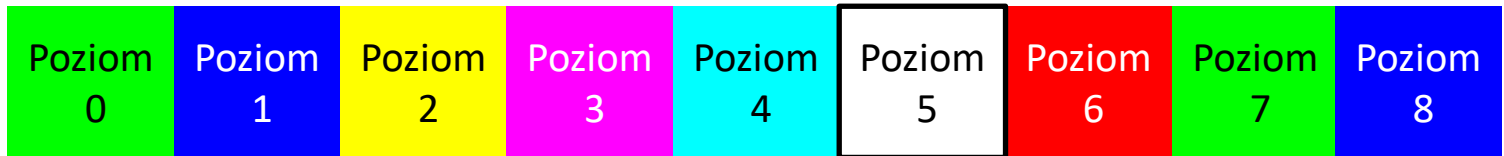
Zadanie (1/2)

- Wklej procedurę generateColorMIPMaps()

```
GLuint generateColorMIPMaps() {
    GLuint tex;
    glActiveTexture(GL_TEXTURE0);
    std::vector<unsigned char> reds(256 * 256 * 3, 0);
    std::vector<unsigned char> greens(256 * 256 * 3, 0);
    std::vector<unsigned char> blues(256 * 256 * 3, 0);
    std::vector<unsigned char> yellows(256 * 256 * 3, 0);
    std::vector<unsigned char> magentas(256 * 256 * 3, 0);
    std::vector<unsigned char> cyans(256 * 256 * 3, 0);
    std::vector<unsigned char> whites(256 * 256 * 3, 0);
    for (int i = 0; i < 256 * 256 * 3; i += 3) {
        reds[i + 0] = 0xff; greens[i + 1] = 0xff; blues[i + 2] = 0xff;
        yellows[i + 0] = 0xff; yellows[i + 1] = 0xff;
        magentas[i + 0] = 0xff; magentas[i + 2] = 0xff;
        cyans[i + 1] = 0xff; cyans[i + 2] = 0xff;
        whites[i + 0] = 0xff; whites[i + 1] = 0xff; whites[i + 2] = 0xff;
    }
    glGenTextures(1, &tex); glBindTexture(GL_TEXTURE_2D, tex);
    unsigned char* pointers[] = {
        reds.data(), greens.data(), blues.data(), yellows.data(),
        magentas.data(), cyans.data(), whites.data()};
    int dim = 256;
    for (int level = 0; level <= 8; level++) {
        glTexImage2D(GL_TEXTURE_2D, level, GL_RGB8, dim, dim, 0,
                     GL_RGB, GL_UNSIGNED_BYTE, pointers[(level+1)%7]);
        dim /= 2;
    }
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_NEAREST);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_NEAREST);
    return tex;
}
```

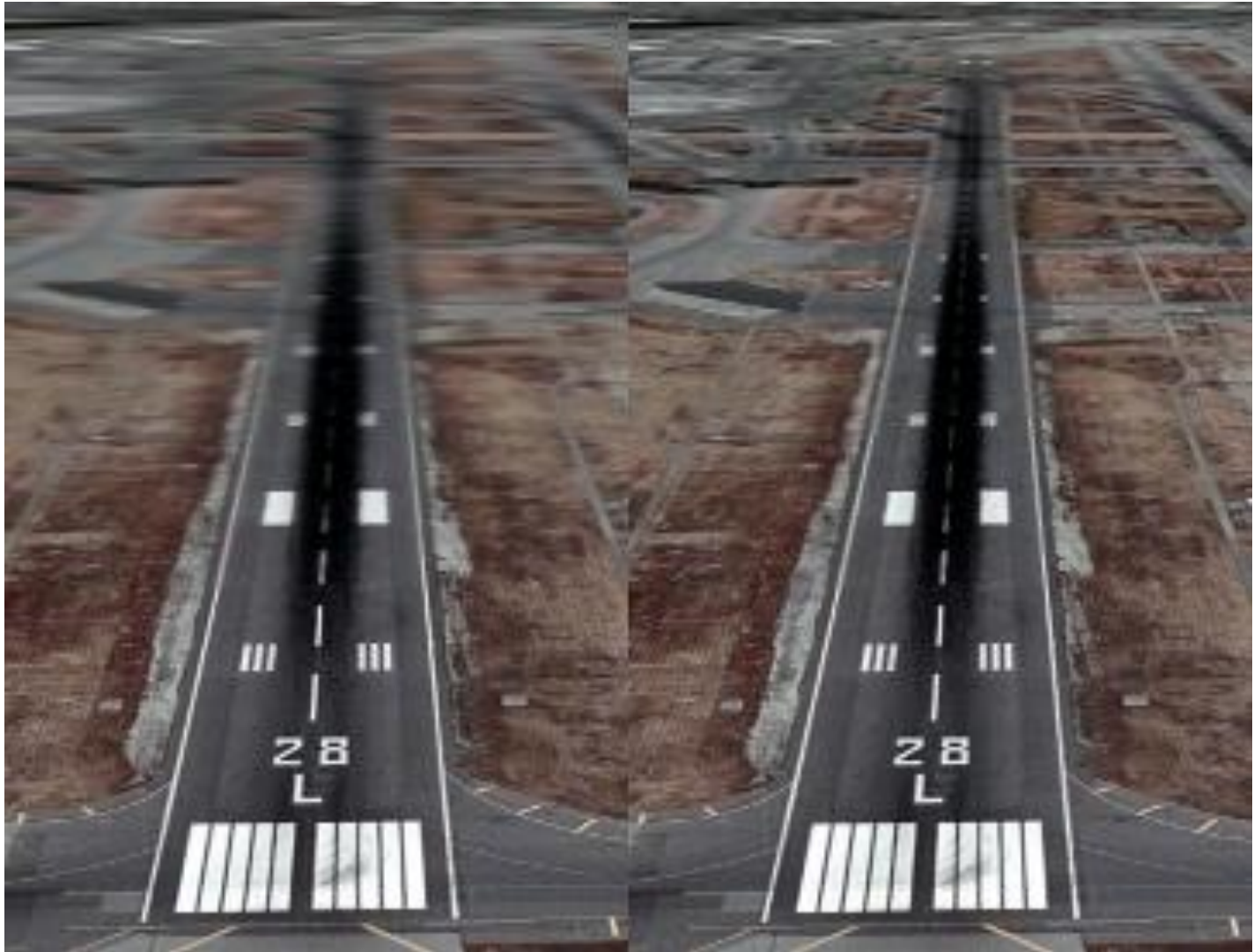
Zadanie (2/2)

- Zamiast wczytywać stone-wall.png, wygeneruj teksturę za pomocą `generateColorMIPMaps()`
- Tworzy ona teksturę 256x256 z MIPmapami o różnych kolorach

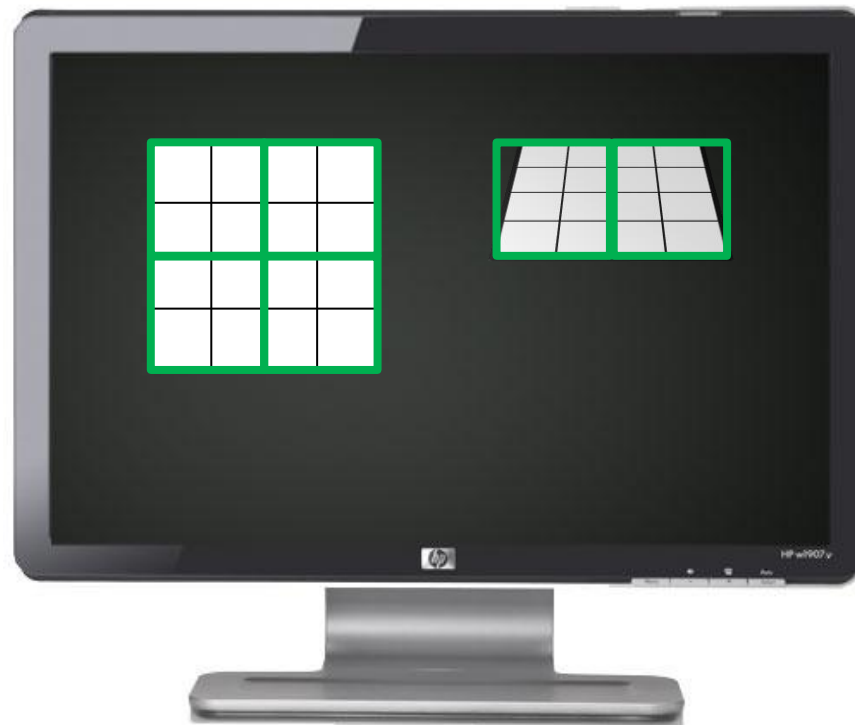
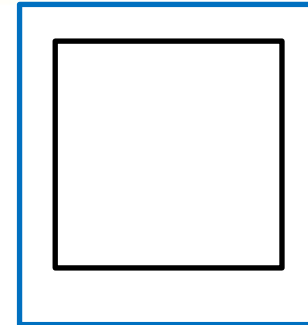
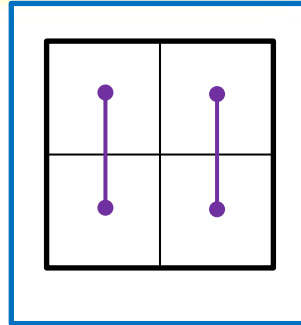
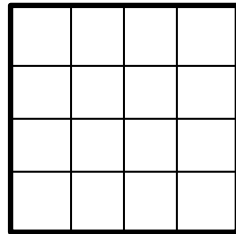


- Zaobserwuj rozmycie/brak rozmycia na granicach pomiędzy MIPmapami dla wariantów `GL_*_LINEAR` i `GL_*_NEAREST`.

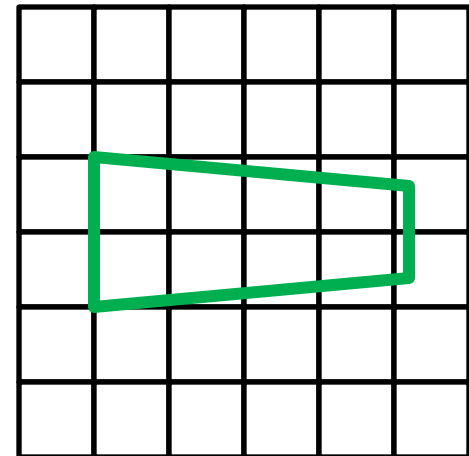
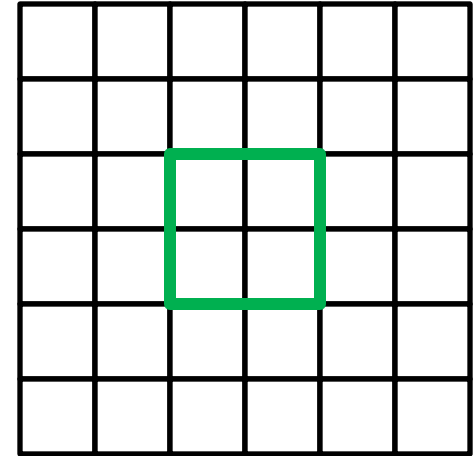
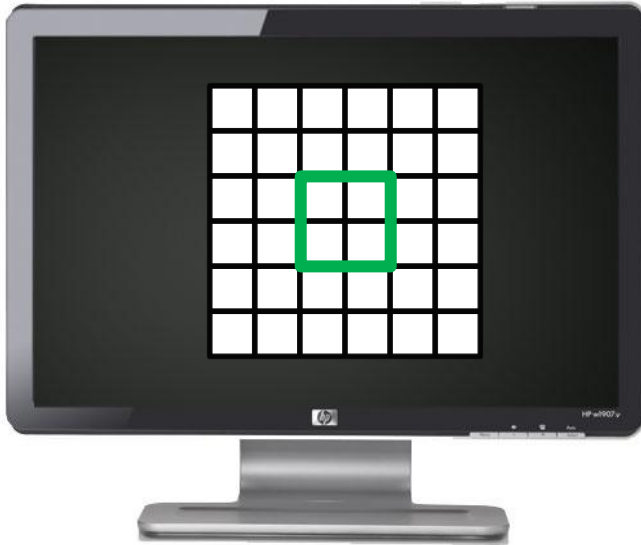
Filtrowanie anizotropowe



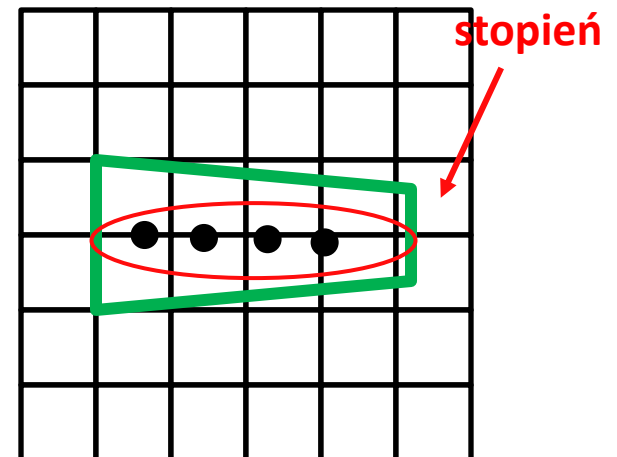
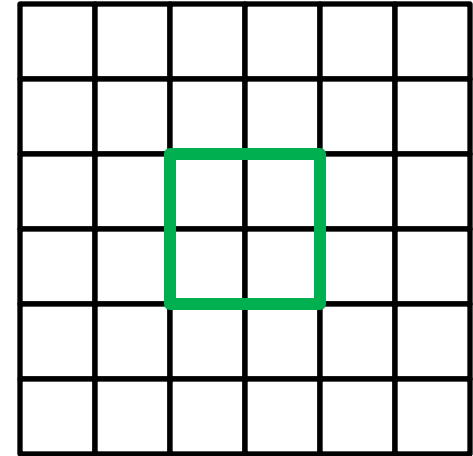
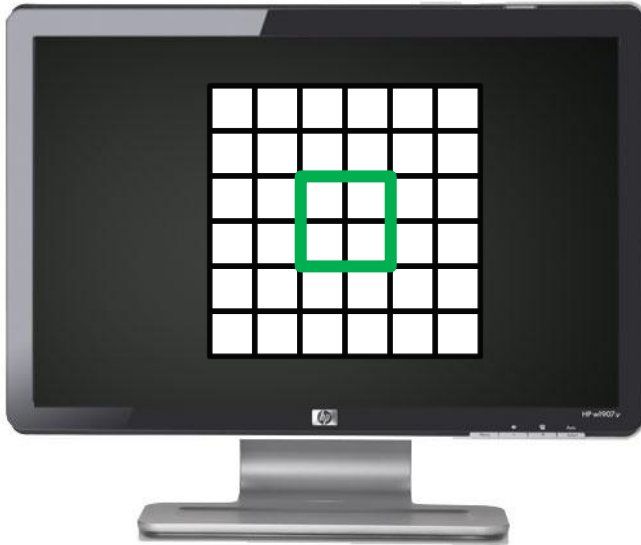
Filtrowanie anizotropowe



Filtrowanie anizotropowe



Filtrowanie anizotropowe



Filtrowanie anizotropowe

- Maksymalny stopień anizotropii

```
float maxAnisotropy;  
glGetFloatv(GL_MAX_TEXTURE_MAX_ANISOTROPY_EXT, &maxAnisotropy);
```

- Ustawienie stopnia anizotropii

```
glTexParameteri(GL_TEXTURE_2D,  
                GL_TEXTURE_MAG_FILTER,  
                GL_LINEAR);  
  
glTexParameteri(GL_TEXTURE_2D,  
                GL_TEXTURE_MIN_FILTER,  
                GL_LINEAR_MIPMAP_LINEAR);  
  
glTexParameterf(GL_TEXTURE_2D,  
                GL_TEXTURE_MAX_ANISOTROPY_EXT, 16.0f);
```

stopień



- _EXT – jeżeli wersja OpenGL <4.6

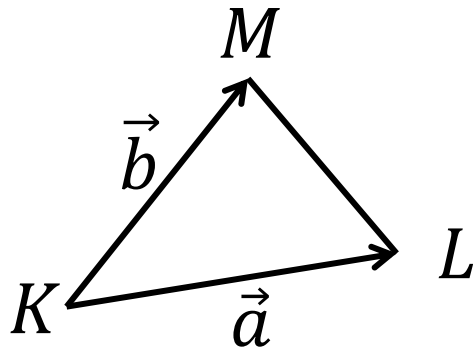
Zadanie (opcjonalne)

- Ustaw kamerę w pozycji $[0, 1.1, -1.5]$
- Przetestowuj filtrowanie anizotropowe dla stopnia 1.0f i maksymalnego stopnia dostępnego na karcie graficznej.

Cieniowanie (1)

- Aby OpenGL poprawnie obliczał ile światła przypada na każdą ścianę lub każdy wierzchołek bryły, musi znać normalne do powierzchni.
- Normalna jest wektorem który jest prostopadły do powierzchni. Normalną można obliczyć znając trzy punkty definiujące tę powierzchnię (tak długo jak nie są współliniowe).

Wektor normalny



$$a_x = L_x - K_x$$

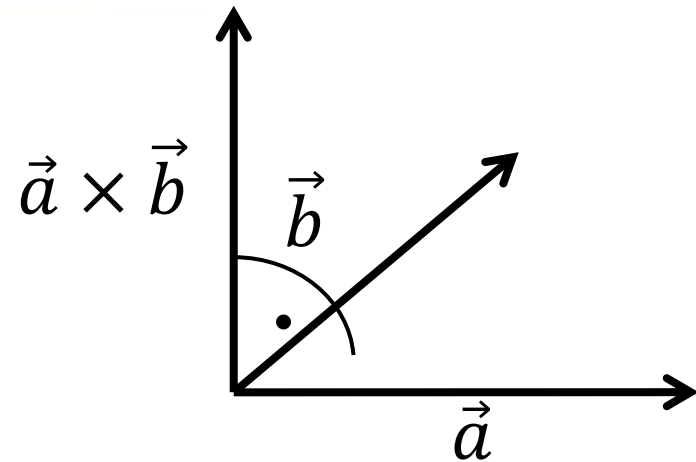
$$a_y = L_y - K_y$$

$$a_z = L_z - K_z$$

$$b_x = M_x - K_x$$

$$b_y = M_y - K_y$$

$$b_z = M_z - K_z$$



$$\vec{a} = [a_x \quad a_y \quad a_z]^T$$

$$\vec{b} = [b_x \quad b_y \quad b_z]^T$$

$$\vec{a} \times \vec{b} = \begin{vmatrix} \vec{i} & \vec{j} & \vec{k} \\ a_x & a_y & a_z \\ b_x & b_y & b_z \end{vmatrix}$$

Cieniowanie (3)

- Współrzędne wektora normalnego podajemy podobnie jak współrzędne tekstur, kolory, i współrzędne wierzchołków – potrzebna jest tablica z odpowiednimi danymi.
- Tablicę wiążemy z poleceniem `glDrawArrays` lub `glDrawElements` za pomocą `glVertexAttribPointer`.

Przykład

```
float verts[]={//Vertex coordinate array
    1.0f,-1.0f,0.0f,1.0f, //A
    -1.0f, 1.0f,0.0f,1.0f, //B
    -1.0f,-1.0f,0.0f,1.0f, //C

    1.0f,-1.0f,0.0f,1.0f, //A
    1.0f, 1.0f,0.0f,1.0f, //D
    -1.0f, 1.0f,0.0f,1.0f}; //B
float texCoords[]={//Texture coord. array
    1.0f, 0.0f, 0.0f, 1.0f, 0.0f, 0.0f, //ABC
    1.0f, 0.0f, 1.0f, 1.0f, 0.0f, 1.0f, //ADB
};
```

```
//Normal vector array
float normals[]={
    0.0f,0.0f,1.0f,0.0f, //A
    0.0f,0.0f,1.0f,0.0f, //B
    0.0f,0.0f,1.0f,0.0f, //C

    0.0f,0.0f,1.0f,0.0f, //A
    0.0f,0.0f,1.0f,0.0f, //D
    0.0f,0.0f,1.0f,0.0f, //B
};
```

```
int vertexCount=6; //Num. of vertices
```

```
spLambertTextured->use();
glUniformMatrix4fv(spLambertTextured->u("P"),1,false,glm::value_ptr(P));
glUniformMatrix4fv(spLambertTextured->u("V"),1,false,glm::value_ptr(V));
glUniformMatrix4fv(spLambertTextured->u("M"),1,false,glm::value_ptr(M));

glEnableVertexAttribArray(spLambertTextured->a("vertex"));
glVertexAttribPointer(spLambertTextured->a("vertex"),4,GL_FLOAT,false,0,verts);

glEnableVertexAttribArray(spLambertTextured->a("normal"));
glVertexAttribPointer(spLambertTextured->a("normal"),4,GL_FLOAT,false,0,normals);

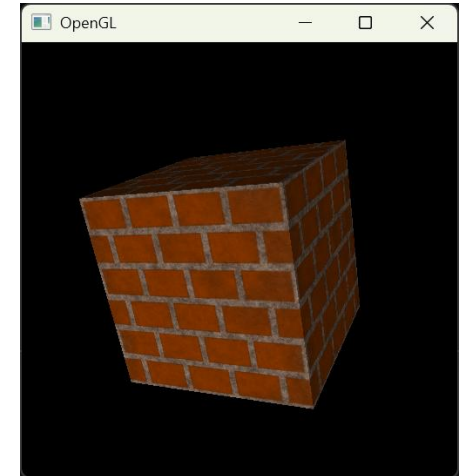
glEnableVertexAttribArray(spLambertTextured->a("texCoord"));
glVertexAttribPointer(spLambertTextured->a("texCoord"),2,GL_FLOAT,false,0,texCoords);

glActiveTexture(GL_TEXTURE0); glBindTexture(GL_TEXTURE_2D,tex);
glUniform1i(spLambertTextured->u("tex"),0);

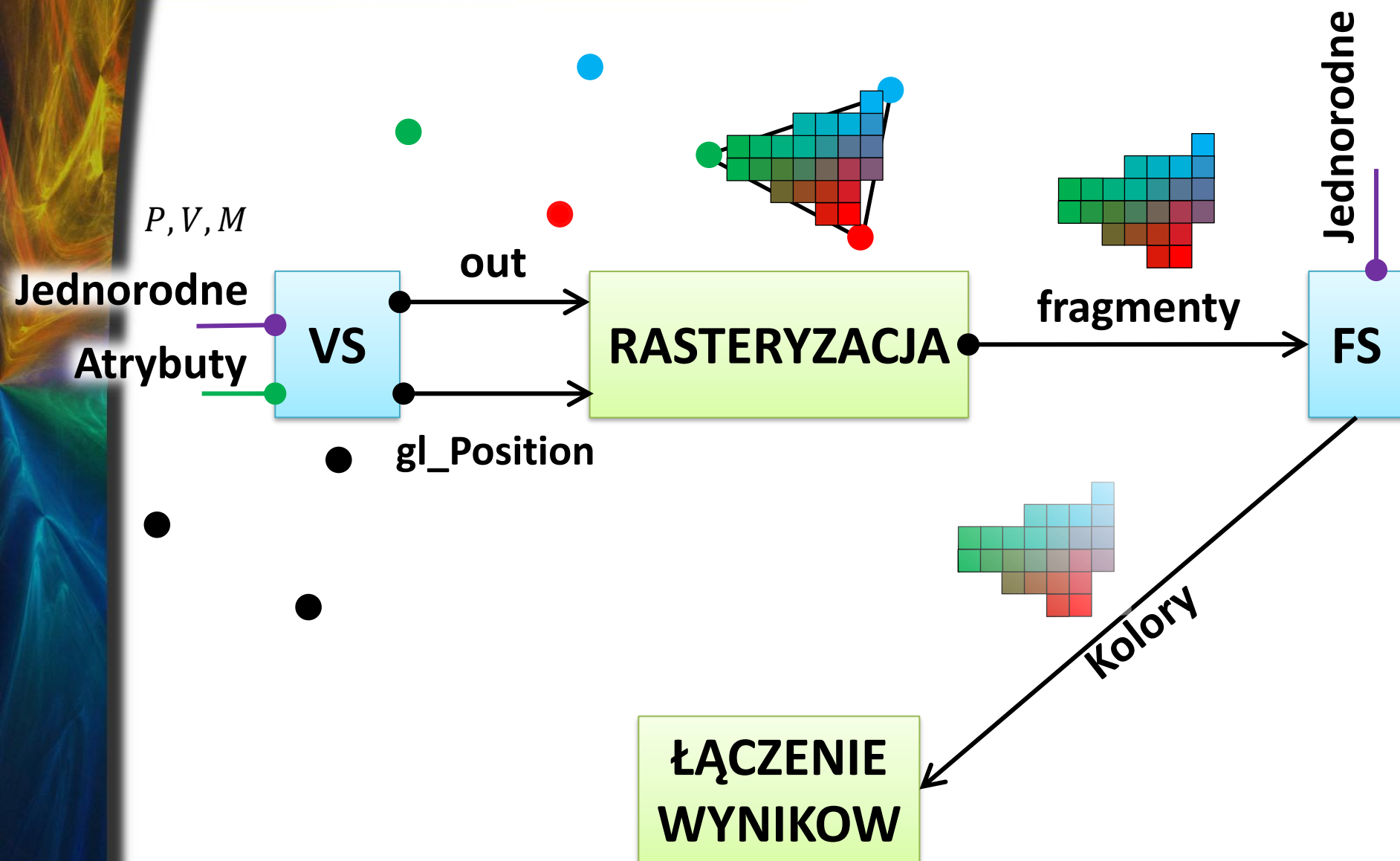
glDrawArrays( GL_TRIANGLES, 0, vertexCount );
glDisableVertexAttribArray(spLambertTextured->a("vertex"));
glDisableVertexAttribArray(spLambertTextured->a("normal"));
glDisableVertexAttribArray(spLambertTextured->a("texCoord"));
```


Zadanie

- Przywróć stan projektu, w którym widać całą kostkę.
- Dodaj wektory normalne do każdej ściany kostki z poprzedniego ćwiczenia.
 - Sciana 1: $\begin{bmatrix} 0 & 0 & -1 & 0 \end{bmatrix}$
 - Sciana 2: $\begin{bmatrix} 0 & 0 & 1 & 0 \end{bmatrix}$
 - Sciana 3: $\begin{bmatrix} 0 & -1 & 0 & 0 \end{bmatrix}$
 - Sciana 4: $\begin{bmatrix} 0 & 1 & 0 & 0 \end{bmatrix}$
 - Sciana 5: $\begin{bmatrix} -1 & 0 & 0 & 0 \end{bmatrix}$
 - Sciana 6: $\begin{bmatrix} 1 & 0 & 0 & 0 \end{bmatrix}$
- Pamiętaj, że wektory normalne podaje się dla każdego wierzchołka osobno.



Podstawowe działanie programu cieniującego



Shadery

- Vertex Shader – programik uruchamiany współbieżnie i niezależnie dla każdego wierzchołka. MUSI przenieść współrzędne z przestrzeni modelu do przestrzeni przycięcia.
- Fragment shader – programik uruchamiany współbieżnie i niezależnie dla każdego fragmentu (~piksela). Generuje jego kolor.

Zmienne

- Zmienne jednorodne – parametry VS i FS. Wartość stała dla jednego rysowania modelu.
- Atrybuty – parametry VS, inna wartość dla każdej instancji.
- Zmienne interpolowane – wartości związane z wierzchołkami, interpolowane dla każdego fragmentu (dane wejściowe FS).
- Fragment – wartości zmiennych interpolowanych, interpolowane dla każdego potencjalnego piksela.

GLSL (1)

- Podstawowe typy

```
float skalar=5;
```

```
vec2 wektor2=vec2(1,2); //Można podawać składowe
```

```
vec3 wektor3=vec3(wektor2,3); //Można „doklejać”
```

```
vec4 wektor4=vec4(wektor2,wektor2); //Można „sklejać”
```

```
mat3 macierz3x3=mat3(
```

```
    vec3(1.0f,2.0f,3.0f),
```

```
    vec3(4.0f,5.0f,6.0f),
```

```
    vec3(7.0f,8.0f,9.0f),
```

```
);
```

```
mat4 macierz4x4=mat4(1.0f);
```

$$\begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & 9 \end{bmatrix}$$
$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

GLSL (2)

- Dostęp do składowych wektora

Aliaszy
składowych

$$a = \begin{bmatrix} 10 \\ 20 \\ 30 \\ 40 \end{bmatrix} \begin{matrix} x & r & s \\ y & g & t \\ z & b & p \\ w & a & q \end{matrix}$$

```
vec4 a=vec4(10,20,30,40);  
float k=a.y //==a.g, ==a.t  
vec2 l=a.xy //[10,20]  
vec3 m=a.zxy //[30,10,20]
```

```
m.zxy=a.yyz;  
m.xxz=a.xyz; //błąd
```

GLSL (3)

- Podstawowe operacje
 - $+$, $-$, $*$, $/$ działa na skalarach, wektorach i macierzach (uwaga na kolejność mnożenia wektorów i macierzy!)

Ćwiczenie 1

- Napisz „flatten” shader.
 - Zmodyfikuj współrzedną wierzchołka w przestrzeni modelu ograniczając ją do płaszczyzny xz .

Ćwiczenie 2

- Zapewnij przekazanie danych z atrybutu color do fragment shadera
 - VS: dodaj atrybut „color”
 - main_file.cpp: uzupełnij o przekazywanie danych z myCubeColors do tego atrybutu
 - VS: Dodaj zmienną interpolowaną „iC”:

```
out vec4 iC;
```
 - Przypisz do iC wartość atrybutu z kolorem.
 - FS: Dodaj zmienną interpolowaną „iC”:

```
in vec4 iC;
```
 - FS: jako wynik FS przypisz iC

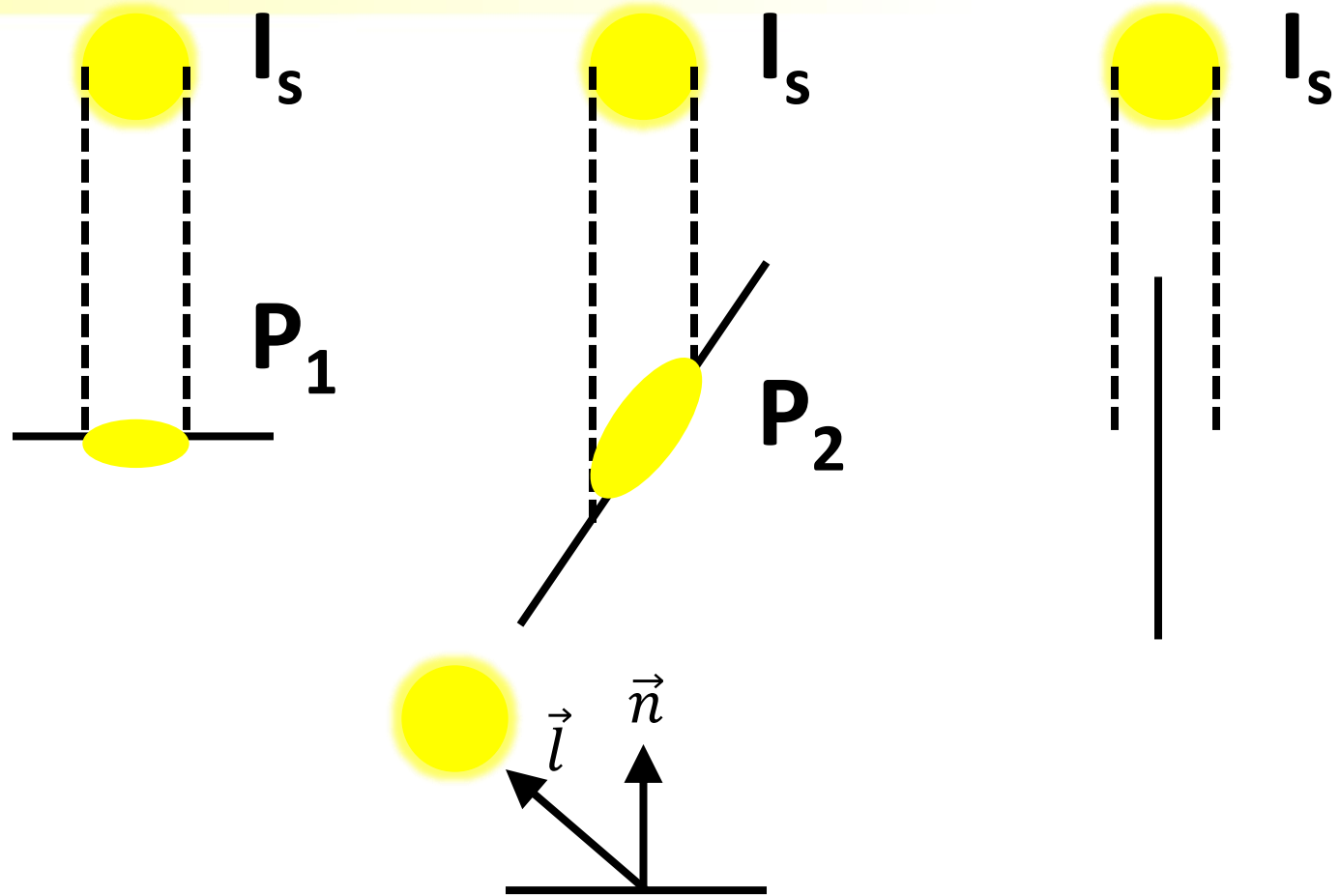
Funkcja distance

```
vec4 a;  
vec4 b;  
//albo dowolny inny typ wektorowy  
  
float d=distance(a,b); //odległość Euklidesowa
```

Ćwiczenie 3

- Distance shader.
 - Im wierzchołek jest bliżej tym jaśniejszy powinien być powiązany z nim kolor.
 - Znając położenie kamery i rozmiary kostki zastanów się w jakim przedziale mogą się pojawić wartości odległości wierzchołka od obserwatora.
 - VS:
 - oblicz odległość wierzchołka od obserwatora (w której przestrzeni najłatwiej to zrobić?)
 - przelicz tę odległość do przedziału $[0,1]$
 - Wykorzystaj obliczoną wartość do zmiany jasności koloru wierzchołka.

Światło rozproszone (1)



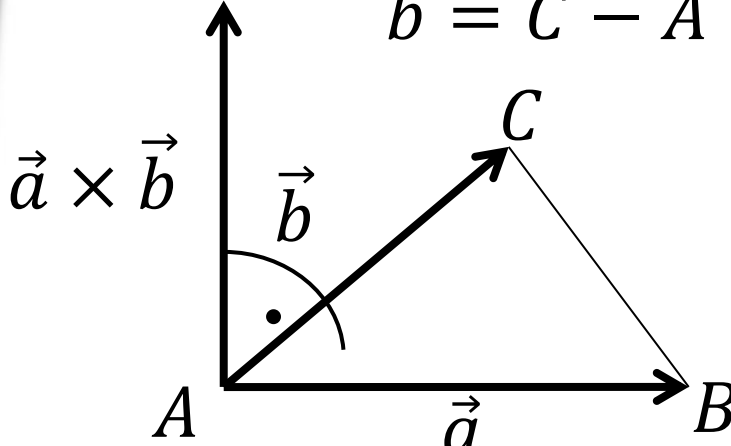
$$E \sim \cos(\vec{n}, \vec{l}) = \vec{n} \cdot \vec{l}$$

Światło rozproszone (2)

$$L = k_a L_a + k_d L_d (\vec{n} \cdot \vec{l})$$

- k_a – kolor materiału dla światła otoczenia
- k_d – kolor materiału dla światła rozpr.
- L_a – kolor światła otoczenia
- L_d – kolor światła rozproszonego
- $\vec{n}$ – wektor normalny (jednostkowy)
- $\vec{l}$ – wektor do światła (jednostkowy)

Wektor normalny

$$\begin{aligned}\vec{a} &= B - A \\ \vec{b} &= C - A\end{aligned}$$

$$\vec{a} \times \vec{b} = \begin{vmatrix} \vec{i} & \vec{j} & \vec{k} \\ a_x & a_y & a_z \\ b_x & b_y & b_z \end{vmatrix}$$
$$\vec{i} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \quad \vec{j} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \quad \vec{k} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

```
#define GLM_FORCE_SWIZZLE //na początku pr.
glm::vec4 A,B,C; //wsp. Wierzchołków

glm::vec4 a=B-A;
glm::vec4 b=C-A;
//Wektor normalny (wynik cross)
//skracamy do długości jednostkowej (normalize)
glm::vec4 n=glm::vec4(
    glm::normalize(glm::cross(a.xyz(),b.xyz())) ,0);
```


Przydatne funkcje

- Iloczyn skalarny dwóch wektorów:

```
vec3 a=vec3(1,2,3);  
vec3 b=vec3(2,4,5);  
float c=dot(a,b); //c=25
```

- Normalizacja wektora:

```
vec3 a=vec3(3,4,0);  
vec3 b;  
b=normalize(a); //b=[0.6,0.8,0]
```

- Ograniczenie wartości do zadanego zakresu

```
float a;  
a=clamp(-1,0,1) //a=0  
a=clamp(2,0,1) //a=1  
a=clamp(0.5,0,1) //a=0.5
```

Ćwiczenie 4a

- Zaimplementuj model oświetlenia ze światłem rozproszonym.
 - VS: dodaj następującą linijkę:

```
//współrzędne światła w przestrzeni świata  
vec4 lp=vec4(0,0,-6,1);
```

- VS i OpenGL: Dodaj atrybut normal, poprzez który przekazane zostaną do VS wektory normalne w przestrzeni modelu.
- Dane do atrybutu normal znajdują się w pliku myCube.h w tablicy myCubeNormals.

Ćwiczenie 4b

- Zaimplementuj model oświetlenia ze światłem rozproszonym (cd).
 - VS: Oblicz:
 - Znormalizowany wektor do światła w przestrzeni oka $\vec{l}$
 - Znormalizowany wektor normalny w przestrzeni oka $\vec{n}$
 - Iloczyn skalarny powyższych wektorów $\vec{n} \cdot \vec{l}$
 - Ogranicz ten iloczyn skalarny do wartości dodatnich (wykorzystaj funkcję *max lub clamp*).
 - Wykorzystaj obliczony powyżej iloczyn skalarny do obliczenia wzoru omawianego wcześniej. Dla uproszczenia załóż, że $L_a = [0 \ 0 \ 0 \ 1]^T$, $L_d = [1 \ 1 \ 1 \ 1]^T$, a k_d to kolor wierzchołka przekazany przez atrybut.
 - Wynik wzoru to ostateczny kolor wierzchołka.

glUniform*

- dla skalarów i wektorów różnych rozmiarów

```
glUniform{1,2,3,4}{f,i}(GLint location, float/int x, ...);
```

```
//uniform vec3 zm;
```

```
glUniform3f(sp->u("zm"), 1, 2, 3);
```

```
//uniform vec4 pos;
```

```
glUniform4f(sp->u("pos"), 1, 2, 3, 4);
```

```
//uniform int x;
```

```
glUniform1i(sp->u("x"), 1);
```

glUniform*v

- Dla skalarów, wektorów i tablic

```
glUniform{1,2,3,4}{f,i}v(GLint location,  
    int count, float/int* pointer);
```

```
//uniform vec3 zm;  
float data[]={1,2,3};  
glUniform3fv(sp->u("zm"),1,data);  
  
glm::vec3 data=glm::vec3(1,2,3);  
glUniform3fv(sp->u("zm"),1,glm::value_ptr(data));
```

```
//uniform vec2 tab[3];  
float data[]={1,2, 3,4, 5,6};  
glUniform2fv(sp->u("tab"), 3,data);  
  
glm::vec2 data[]={  
    glm::vec2(1,2), glm::vec2(3,4), glm::vec2(5,6)};  
glUniform2fv(sp->u("tab"), 3,glm::value_ptr(data[0]));
```

glUniformMatrix*

- dla macierzy i tablic macierzy

```
glUniformMatrix{2,3,4}fv(  
    GLint location,  
    GLsizei count, //liczba macierzy do wysłania  
    GLboolean transpose, //transponować macierze?  
    GLfloat *v); //tablica z danymi macierzy
```

```
//uniform mat4 P;  
glm::mat4 A=glm::mat4(...);  
glUniformMatrix4fv(sp->u("P"),1, false,  
    glm::value_ptr(A));
```

```
//uniform mat4 tab[3];  
glm::mat4 data[3]={  
    glm::mat4(...), glm::mat4(...), glm::mat4(...)};  
glUniformMatrix4fv(sp->u("tab"),3, false,  
    glm::value_ptr(data[0]));
```

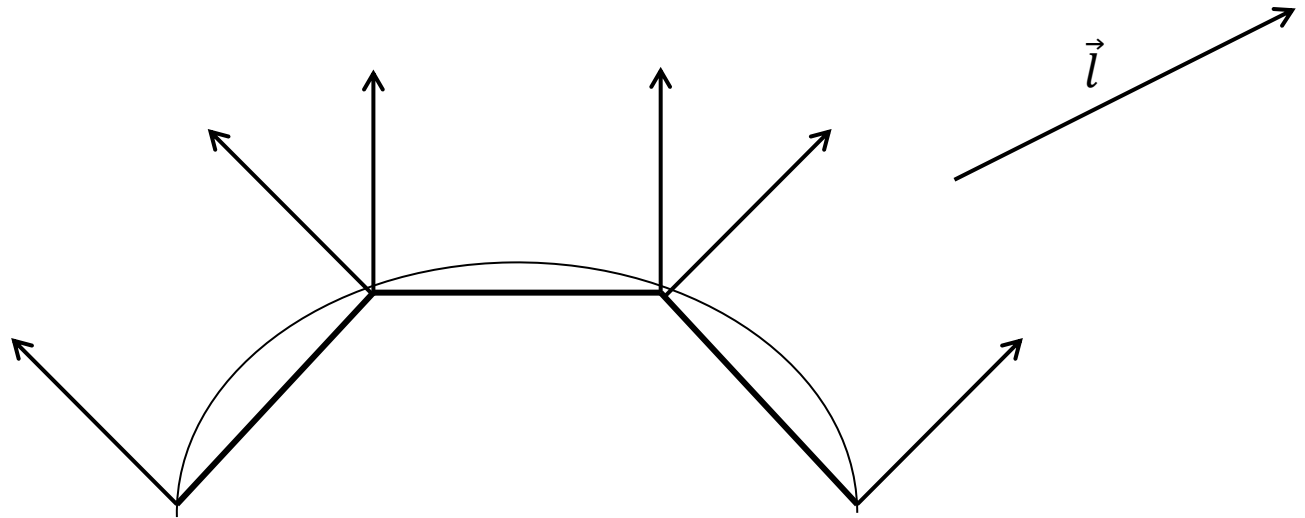
Ćwiczenie 5

- Zastąp zmienną `lp` przechowującą współrzędne źródła światła zmienną jednorodną
- Dodaj kod przekazujący współrzędne źródła światła do zmiennej jednorodnej `lp` za pomocą odpowiedniego wariantu polecenia `glUniform*`.

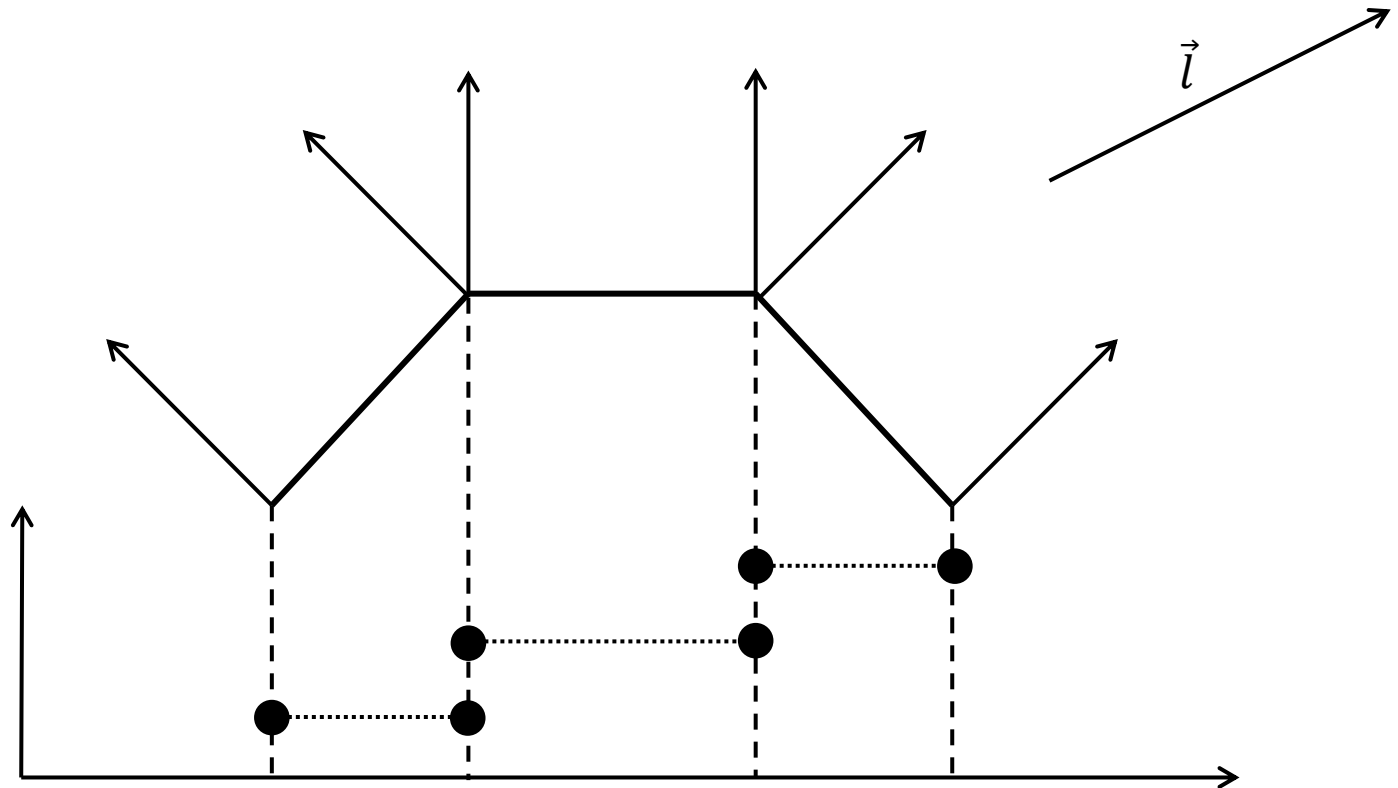
Ćwiczenie 6

- Narysuj czajnik wykorzystując swój shader
- Dane do czajnika znajdują się w pliku `myTeapot.h` w tablicach `myTeapotVertices` i `myTeapotNormals`.
- Liczba wierzchołków znajduje się w zmiennej `myTeapotVertexCount`
- Dlaczego wygląda tak sobie?

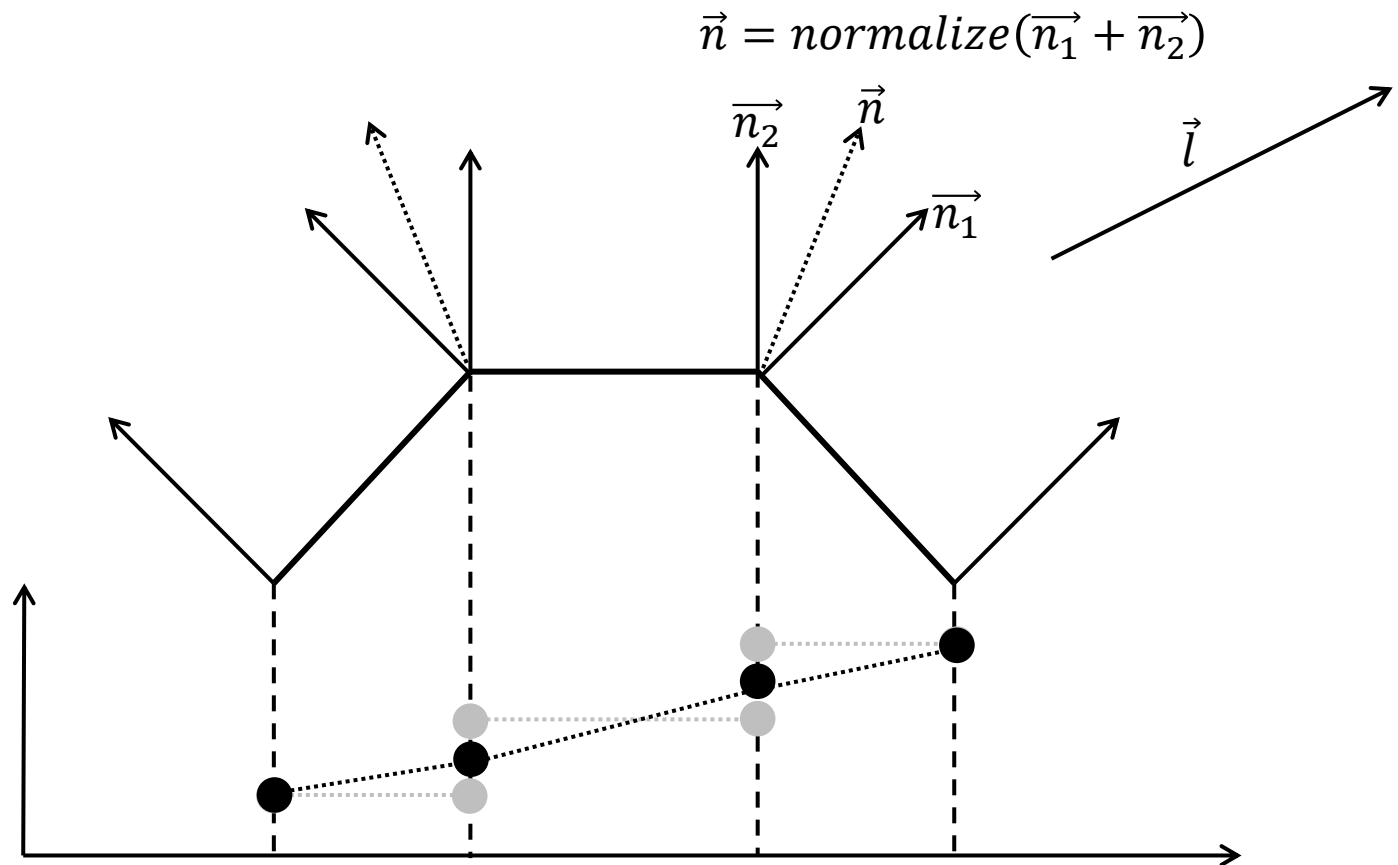
Wektor normalny wierzchołka



Wektor normalny wierzchołka



Wektor normalny wierzchołka



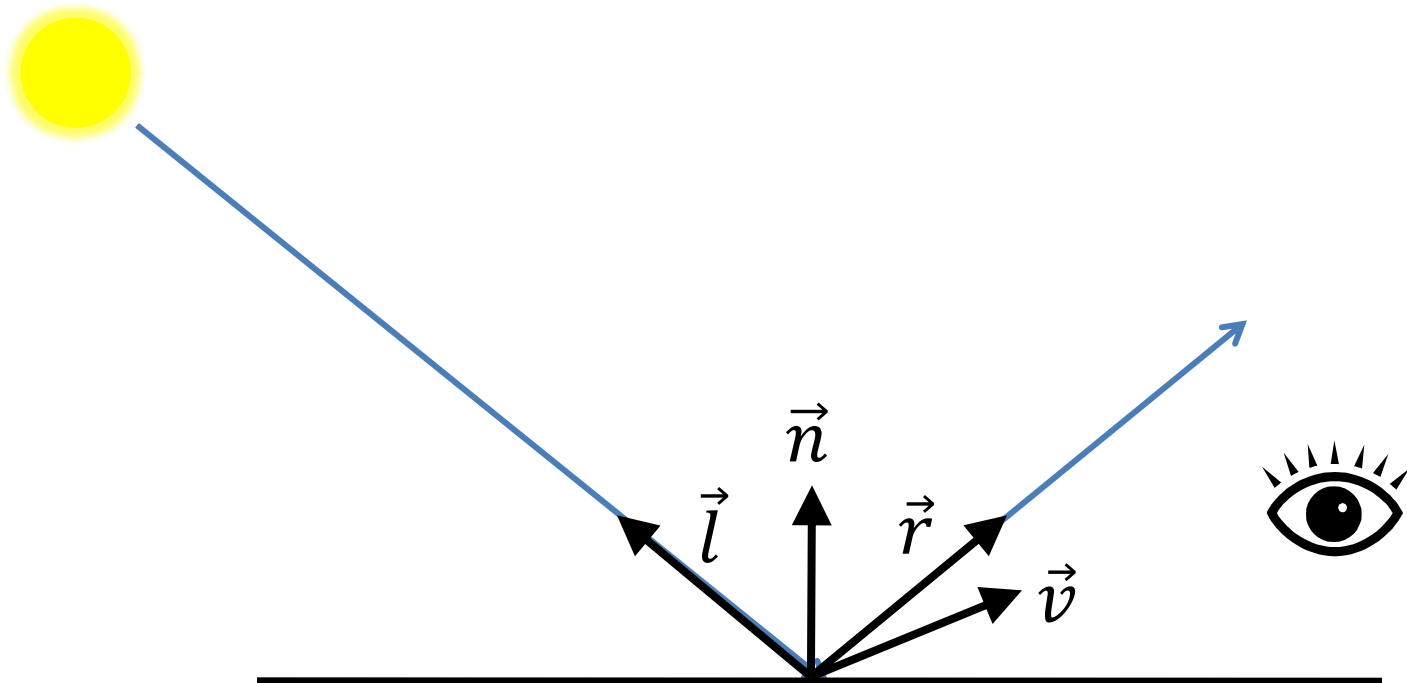
Wektor normalny wierzchołka

- Matematycznie nie istnieje coś takiego!
- Wektor normalny wierzchołka to znormalizowana suma znormalizowanych wektorów normalnych sąsiednich ścian.
- Przeczytaj powyższy punkt szybko 10 razy.

Zadanie

- Użyj tablicy myCubeVertexNormals lub myTeapotVertexNormals zamiast tablic z wektorami używanych poprzednio.
- Odpowiednie tablice są podane w plikach myCube.h i myTeapot.h

Model Phong (1)



Model Phong (2)

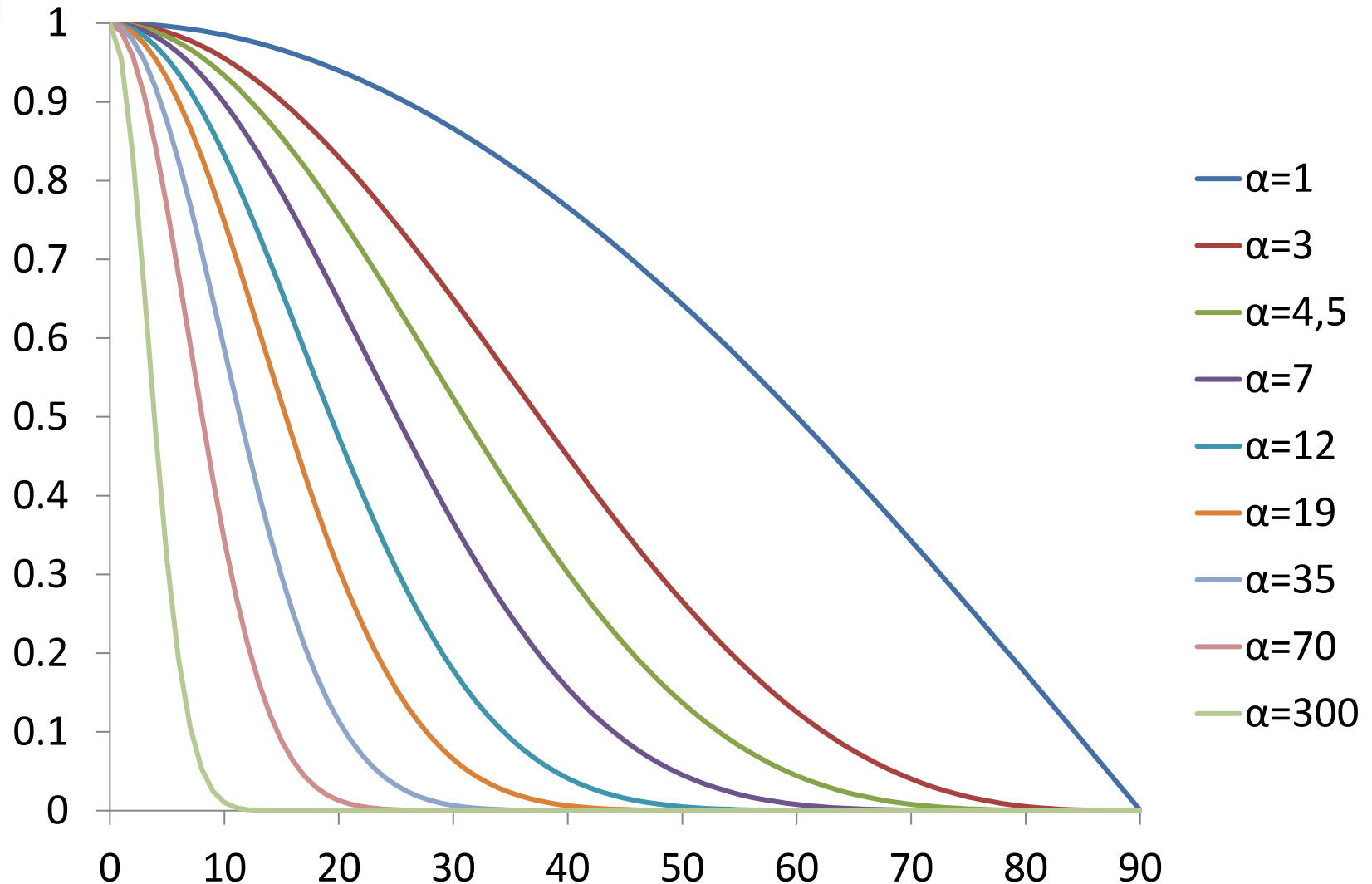
$$L = k_a L_a + k_d L_d (\vec{n} \cdot \vec{l}) + k_s L_s (\vec{r} \cdot \vec{v})^\alpha$$

- L_s – kolor światła odbitego
- k_s – kolor materiału dla światła odbitego
- $\vec{v}$ – wektor do obserwatora (jednostkowy)
- α – wpływa na skupienie odbitego światła
- $\vec{r}$ – wektor odbitego światła (jednostkowy)

$$\vec{r} = 2 (\vec{n} \cdot \vec{l}) \vec{n} - \vec{l}$$

```
vec4 r=reflect(-l,n);
```

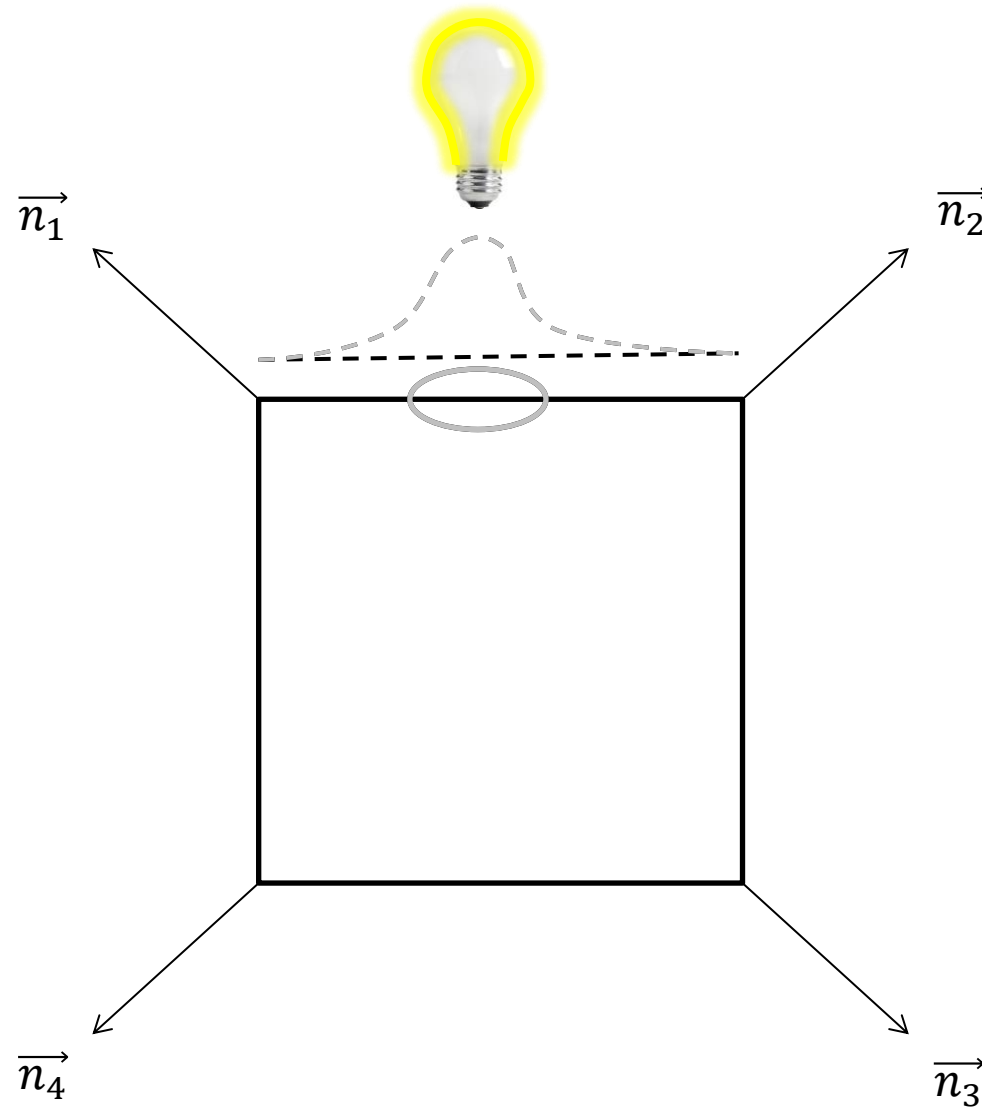
Wykładnik phonga



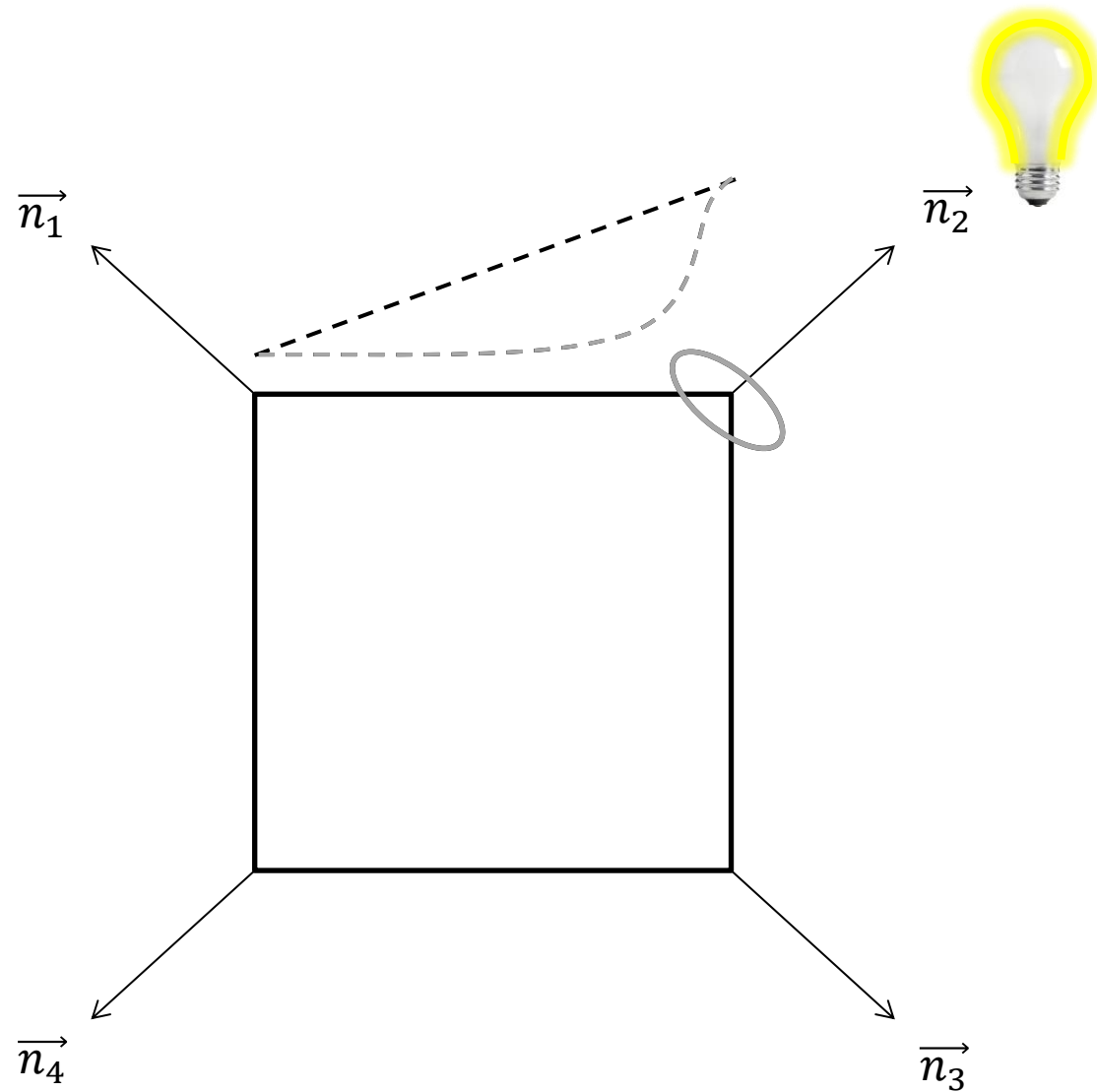
Ćwiczenie 5a

- Uzupełnij poprzednie ćwiczenie o światło odbite:
 - Oblicz znormalizowany wektor $\vec{r}$ w przestrzeni oka.
 - Oblicz znormalizowany wektor $\vec{v}$ w przestrzeni oka.
 - Oblicz iloczyn skalarny. Ogranicz wynik do wartości dodatnich.
 - Podnieś wynik do potęgi α (funkcja *pow*). Przyjmij $\alpha = 25$.
 - Wylicz resztę wzoru i użyj wyniku jako koloru.
 - Załóż, że $L_s = [1 \ 1 \ 1 \ 1]^T$ i $k_s = [1 \ 1 \ 1 \ 1]^T$.
- Dlaczego wynik nie wygląda tak jak powinien?

Problemy z odbiciami światła



Problemy z odbiciami światła



Ćwiczenie 6

- Przenieś obliczenia modelu oświetlenia do fragment shadera.
 - VS: iC przekazuje tylko kolor
 - VS: zakomentuj/usuń liczenie $\vec{r}$, $\vec{n} \cdot \vec{l}$ i $\vec{r} \cdot \vec{v}$
 - VS: zmienne z obliczonymi wektorami $\vec{n}$, $\vec{v}$ i $\vec{l}$ (w przestrzeni oka) zamień na zmienne interpolowane
 - FS: pobierz $\vec{n}$, $\vec{v}$ i $\vec{l}$ ze zmiennych interpolowanych do lokalnych zmiennych i znormalizuj.
 - FS: oblicz $\vec{r}$, $\vec{n} \cdot \vec{l}$ i $\vec{r} \cdot \vec{v}$ oraz wylicz model oświetlenia i przypisz wynik do pixelColor.

Funkcje w GLSL

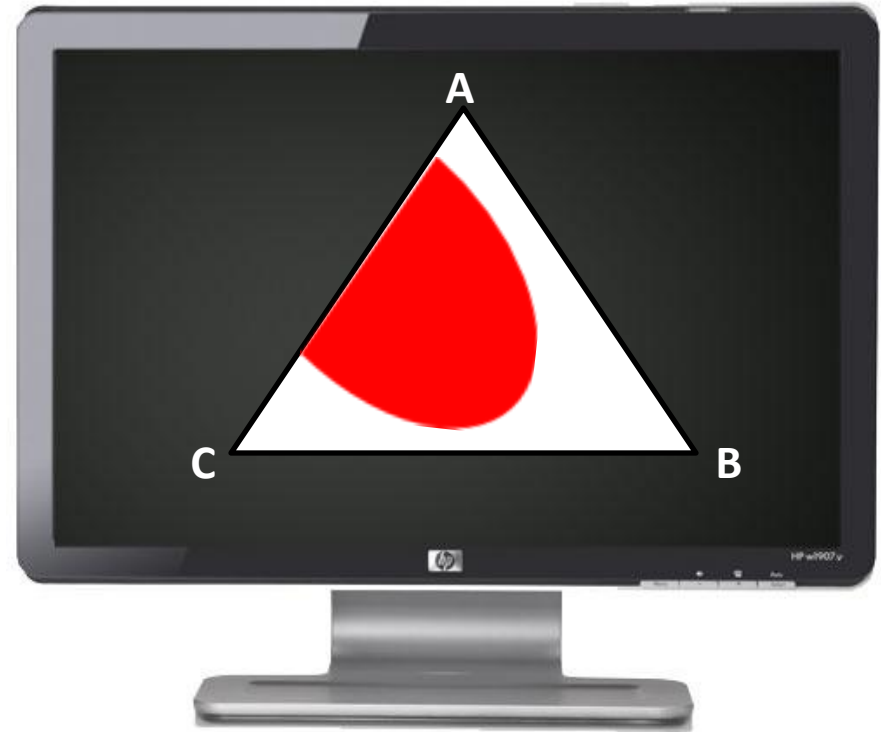
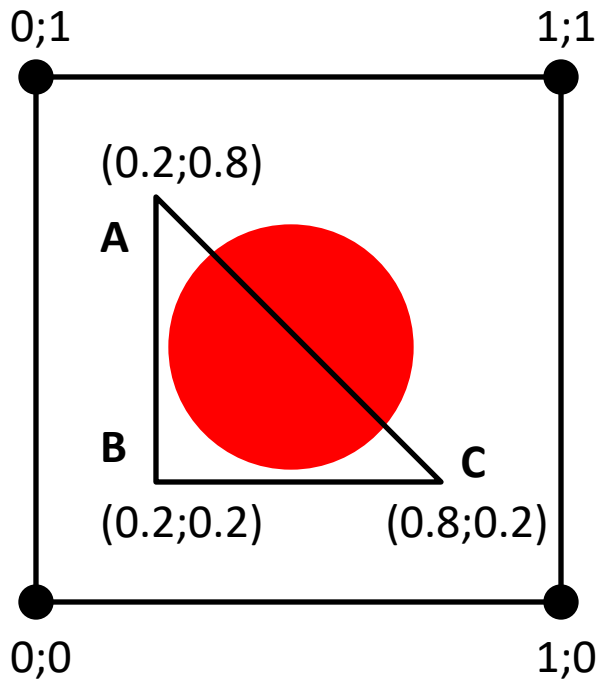
- Tak jak w języku C

```
vec4 convertToMono(vec4 color) {  
    float mono = (0.2125 * color.r) +  
                 (0.7154 * color.g) +  
                 (0.0721 * color.b);  
    return vec4(mono,mono,mono,color.a);  
}
```


Ćwiczenie 7 -Toon shading

- Jasność wyrażoną za pomocą $\vec{n} \cdot \vec{l}$ i $\vec{r} \cdot \vec{v}$ ogranicz do kilku stopni przed ostatecznym wyliczeniem modelu oświetlenia.
- Przykładowo, jeżeli wartości te zmieniają się w przedziale od 0-1 w sposób ciągły, to powinny one po przekształceniu przyjmować tylko wartości ze zbioru: 0, 0.25, 0.5, 0.75, 1.
- Przygotuj funkcję, która wykonuje ograniczenie wartości i użyj jej do wartości $\vec{n} \cdot \vec{l}$ i $\vec{r} \cdot \vec{v}$

Współrzędne teksturowania



Tekstutowanie

- Załadowanie tekstury omawiano wcześniej (glActiveTexture, glGenTextures, glBindTexture, glTexImage2D).
- Konieczne jest napisanie odpowiedniego vertex shadera i fragment shadera.
- Tekstutowaniem zajmują się jednostki teksturujące. Każda jedną teksturą.
- Liczba jednostek teksturujących:

```
GLint textureUnits;  
glGetIntegerv(GL_MAX_TEXTURE_IMAGE_UNITS, &textureUnits);
```

Ćwiczenie 8

- W ćwiczeniu dodasz teksturowanie do Twojego programu.
- Zignoruj kolory przekazywane przez atrybut color. Kolory teraz powinny być pobierane z tekstury.
- Zachowaj cieniowanie (kolor pobrany z tekstury wykorzystaj jako k_d).
- Modyfikacje VS i FS oraz programu OpenGL zostały opisane na kolejnych slajdach.

Ćwiczenie 8, VS

- Przekazanie współrzędnych teksturowania do VS – zwykły atrybut

```
in vec2 texCoord0;
```

- Współrzędne teksturowania muszą być interpolowane

```
out vec2 iTexCoord0; //globalnie  
.....  
iTexCoord0=texCoord0; //w main
```

Ćwiczenie 8, FS

- Zmienna reprezentująca numer jednostki teksturującej – z punktu widzenia OpenGL zwykły int. Ponieważ jest jednorodna, to musi być przypisana z poziomu OpenGL.

```
uniform sampler2D textureMap0; //globalnie
```

- Zmienna interpolowana ze współrzędnymi teksturowania

```
in vec2 iTexCoord0; //globalnie
```

- Pobranie koloru

```
vec4 texColor=texture(textureMap0,iTexCoord0); //main
```

Tekstury w OpenGL („czeklista”)

- Przygotowanie modelu
 - Wczytanie tekstury do pamięci
- Rysowanie modelu
 - Powiązanie tekstury z jednostką teksturującą
 - Przypisanie jednostki teksturującej do zmiennej typu `sampler2D`.
 - Przypisanie tablic ze współrzędnymi teksturowania do odpowiedniego atrybutu.

Ćwiczenie 8, OpenGL(1)

- Przekopiuj poniższą procedurę do programu:

```
GLuint readTexture(const char* filename) {
    GLuint tex;
    glActiveTexture(GL_TEXTURE0);

    //Wczytanie do pamięci komputera
    std::vector<unsigned char> image;    //Alokuj wektor do wczytania obrazka
    unsigned width, height;    //Zmienne do których wczytamy wymiary obrazka
    //Wczytaj obrazek
    unsigned error = lodepng::decode(image, width, height, filename);

    //Import do pamięci karty graficznej
    glGenTextures(1, &tex); //Zainicjuj jeden uchwyt
    glBindTexture(GL_TEXTURE_2D, tex); //Uaktywnij uchwyt
    //Wczytaj obrazek do pamięci KG skojarzonej z uchwytem
    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, width, height, 0,
        GL_RGBA, GL_UNSIGNED_BYTE, (unsigned char*) image.data());

    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);

    return tex;
}
```

Ćwiczenie 8, OpenGL(2)

- Dodaj globalną zmienną reprezentującą uchwyt na teksturę.

```
GLuint tex0;
```

- W `initOpenGLProgram` wczytaj teksturę

```
tex0=readTexture("metal.png");
```

Ćwiczenie 8, OpenGL(3)

- Dodaj tablicę współrzędnych tekstuowania i przekaż ją do atrybutu texCoord0. Zwróć uwagę, na to, że jest to atrybut typu vec2.

```
//W drawScene
glEnableVertexAttribArray(sp->a("texCoord0"));
...
glVertexAttribPointer(sp->a("texCoord0"),
    2, GL_FLOAT, false, 0, texCoords); //odpowiednia tablica
...
glDisableVertexAttribArray(sp->a("texCoord0"));
```

- Teksturowanie będzie teraz działać, ale tylko przez przypadek.

Ćwiczenie 8, OpenGL(4)

- Przypisz do tej zmiennej jednorodnej textureMap0 numer jednostki teksturującej **x** (różnej od 0, ≤ 15) (w drawScene obok innych glUniform*)

```
glUniform1i(sp->u("textureMap0"), x); //drawScene
```

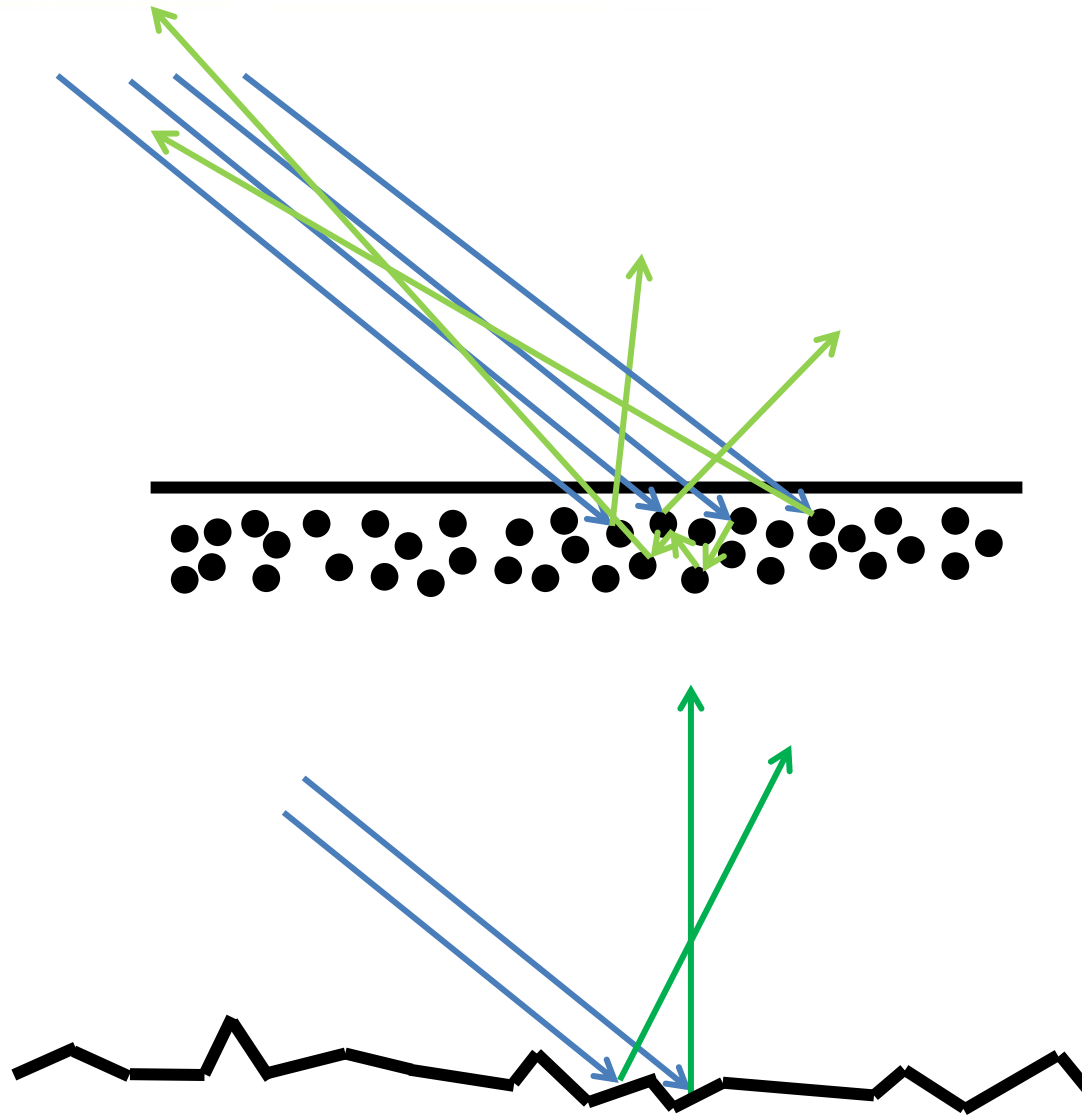
Ćwiczenie 8, OpenGL(5)

- Ostatecznie, przed narysowaniem obiektu przypisz teksturę do jednostki teksturującej.

```
//Przed glDrawArrays w drawScene  
glActiveTexture(GL_TEXTURE0);  
glBindTexture(GL_TEXTURE_2D, tex0);
```

- UWAGA! Nie opisano tutaj usuwania tekstury z pamięci, ale w normalnych programach trzeba to zrobić!

Rozproszenie i odbicie



Ćwiczenie

- Jak wyglądałby czajnik, gdyby kolor odbicia też pochodziłby z tekstury?
- Zmodyfikuj wzór obliczający model oświetlenia tak, aby kolor z tekstury był wykorzystywany zarówno w części Lambertowskiej modelu (k_d) jak i w części Phong'a (k_s), np. $k_d = \text{texture}(\dots)$, $k_s = k_d/3$

Multiteksturowanie (1)

- Aby przypisać teksturę do odpowiedniej jednostki teksturującej można postąpić następująco (powinno to być wykonane przez narysowaniem obiektu):

```
GLuint tex1; //uchwyt na teksturę  
.....  
  
//Uaktywnij pierwszą jednostkę teksturującą  
glActiveTexture(GL_TEXTURE1);  
//Przypisz do niej teksturę  
glBindTexture(GL_TEXTURE_2D, tex1);
```

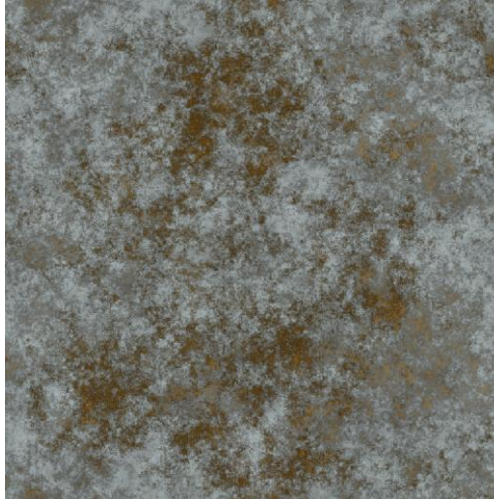
- Można w ten sposób przypisać więcej niż jedną teksturę do wielu jednostek teksturujących.

Multiteksturowanie (2)

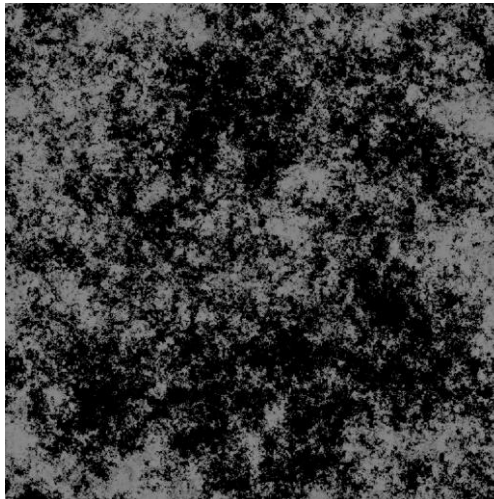
- Korzystanie z wielu tekstur z poziomu VS i FS jest analogiczne.
 - Wiele atrybutów/współrzędnych teksturowania
 - Alternatywnie współrzędne teksturowania generowane przez VS
 - Wszystkie różne współrzędne interpolowane są w osobnych zmiennych interpolowanych.
 - Mamy wiele zmiennych typu sampler2D, każda reprezentuje inną jednostkę teksturującą.
 - Każdą z tych zmiennych odpytujemy niezależnie tak, jak pokazano na poprzednich slajdach

Diffuse i specular map

metal.png



metal_spec.png

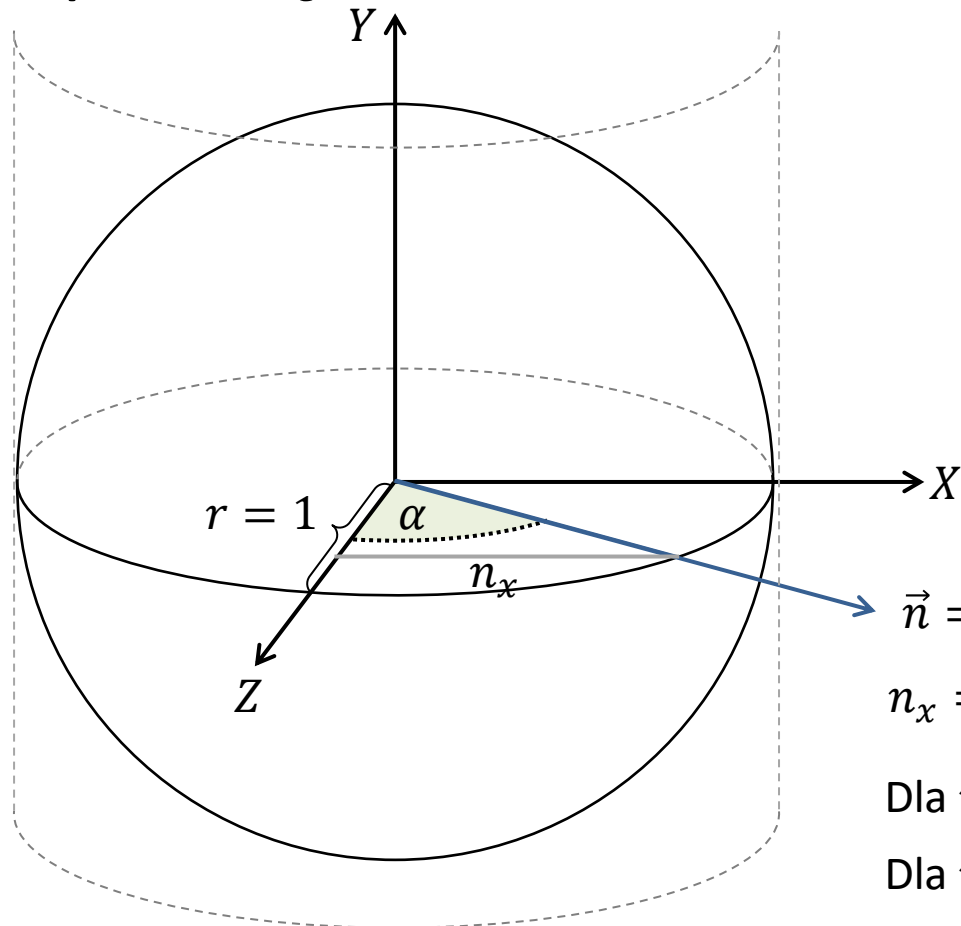


Ćwiczenie 9

- Wczytaj drugą teksturę (metal_spec.png).
- FS: zadeklaruj drugą zmienną jednorodną reprezentującą jednostkę teksturującą.
- OpenGL:
 - Przypisz do zmiennej jednorodnej zadeklarowanej powyżej wartość 1 (glUniform1i(...)).
 - Przed narysowaniem przypisz drugą teksturę do pierwszej jednostki teksturującej.
- Ponieważ nowa tekstura będzie wykorzystywać te same współrzędne teksturowania, nie ma konieczności dodawania nowych zmiennych interpolowanych ani nowych atrybutów.
- We fragment shaderze wykorzystaj kolor pobrany z tekstury jako k_s .

Sphere mapping

- Chcemy rzutować teksturę na widoczną półkulę



$$\vec{n} = [\sin \alpha, 0, \cos \alpha]$$

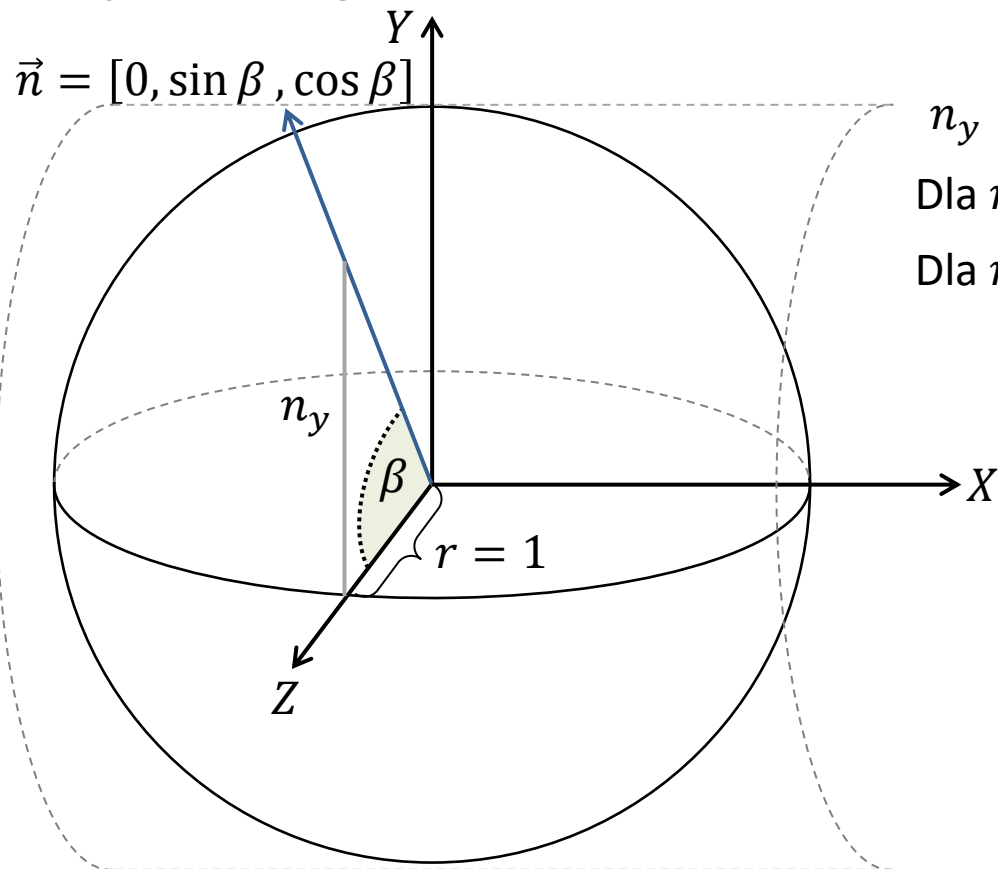
$$n_x = \sin \alpha \Rightarrow \alpha = \arcsin n_x$$

$$\text{Dla } n_x = -1 \text{ zachodzi } \alpha = -\frac{\pi}{2}$$

$$\text{Dla } n_x = 1 \text{ zachodzi } \alpha = \frac{\pi}{2}$$

Sphere mapping

- Chcemy rzutować teksturę na widoczną półkulę



$$n_y = \sin \beta \Rightarrow \beta = \arcsin n_y$$

$$\text{Dla } n_y = -1 \text{ zachodzi } \beta = -\frac{\pi}{2}$$

$$\text{Dla } n_y = 1 \text{ zachodzi } \beta = \frac{\pi}{2}$$

Sphere mapping

- Odwzorowujemy kąt na współrzędną teksturowania

$$s, t \in [0,1]$$

$$\alpha, \beta \in \left[-\frac{\pi}{2}, \frac{\pi}{2}\right]$$

$$\frac{\alpha}{\pi}, \frac{\beta}{\pi} \in \left[-\frac{1}{2}, \frac{1}{2}\right]$$

$$\frac{\alpha}{\pi} + \frac{1}{2}, \frac{\beta}{\pi} + \frac{1}{2} \in [0,1]$$

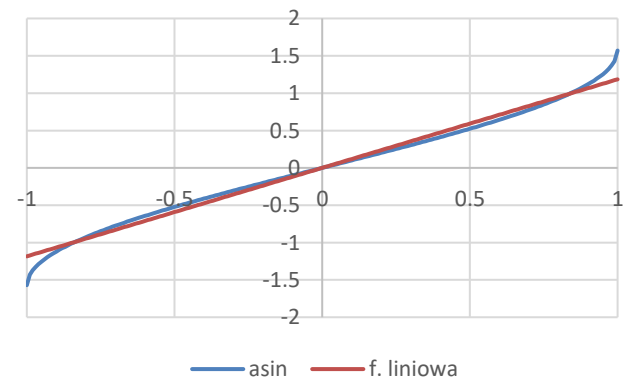
Sphere mapping

- Z powyższych wyliczeń wynika zatem, że:

$$s = \frac{\alpha}{\pi} + \frac{1}{2} = \frac{\operatorname{asin} n_x}{\pi} + \frac{1}{2}$$

$$t = \frac{\beta}{\pi} + \frac{1}{2} = \frac{\operatorname{asin} n_y}{\pi} + \frac{1}{2}$$

- Ponieważ funkcja $\operatorname{asin} x$ jest w przybliżeniu liniowa, możliwe jest uproszczenie powyższych wzorów



Sphere mapping

- Zastąpmy $\text{asin } n_x$ i $\text{asin } n_y$ przez n_x i n_y .

$$s, t \in [0,1]$$

$$n_x, n_y \in [-1,1]$$

$$n_x + 1, n_y + 1 \in [0,2]$$

$$\frac{(n_x + 1)}{2}, \frac{(n_y + 1)}{2} \in [0,1]$$

- Z powyższych wyliczeń wynika zatem, że:

$$s = \frac{(n_x + 1)}{2} \qquad t = \frac{(n_y + 1)}{2}$$

- W ten sposób można rzutować teksturę na dowolny obiekt

Funkcja mix

- Oblicza wynik interpolacji liniowej pomiędzy pierwszymi dwoma argumentami:

$$\text{mix}(a, b, t) = (1 - t)a + tb$$

- Wartość $t \in [0,1]$
- Działa dla dowolnych typów a i b

```
w=mix(a,b,t);
```

Ćwiczenie 10a

- OpenGL: Zamiast `metal_spec.png` wczytaj `sky.png`.
- FS:
 - zadeklaruj zmienną interpolowaną (np. `vec2 iTexCoord1`), która będzie zawierać współrzędne teksturowania dla drugiej tekstury.
 - ostateczny kolor k_d wygeneruj jako wartość pośrednią kolorów metalu i nieba:
`mix(metal, sky, 0.3)`.
 - Niech $k_s = \langle 1, 1, 1, 1 \rangle$ (nie używamy już `metal_spec.png`)

Ćwiczenie 10b

- VS: zadeklaruj zmienną interpolowaną, odpowiednik zadeklarowanej w poprzednim slajdzie (iTexCoord1) która będzie interpolować współrzędne teksturowania dla drugiej tekstury.
- VS: Współrzędne teksturowania dla drugiej tekstury obliczaj w VS na podstawie wektora normalnego w przestrzeni oka w następujący sposób.

```
vec4 n=normalize(V*M*normal);  
iTexCoord1=(n.xy+1)/2;
```

Instancing (1)

- Czasami zachodzi konieczność wielokrotnego narysowania tego samego modelu (np. trawa) z drobnymi zmianami.
- OpenGL udostępnia dwie funkcje, analogiczne do `glDrawArrays` i `glDrawElements`, pozwalające na coś takiego:

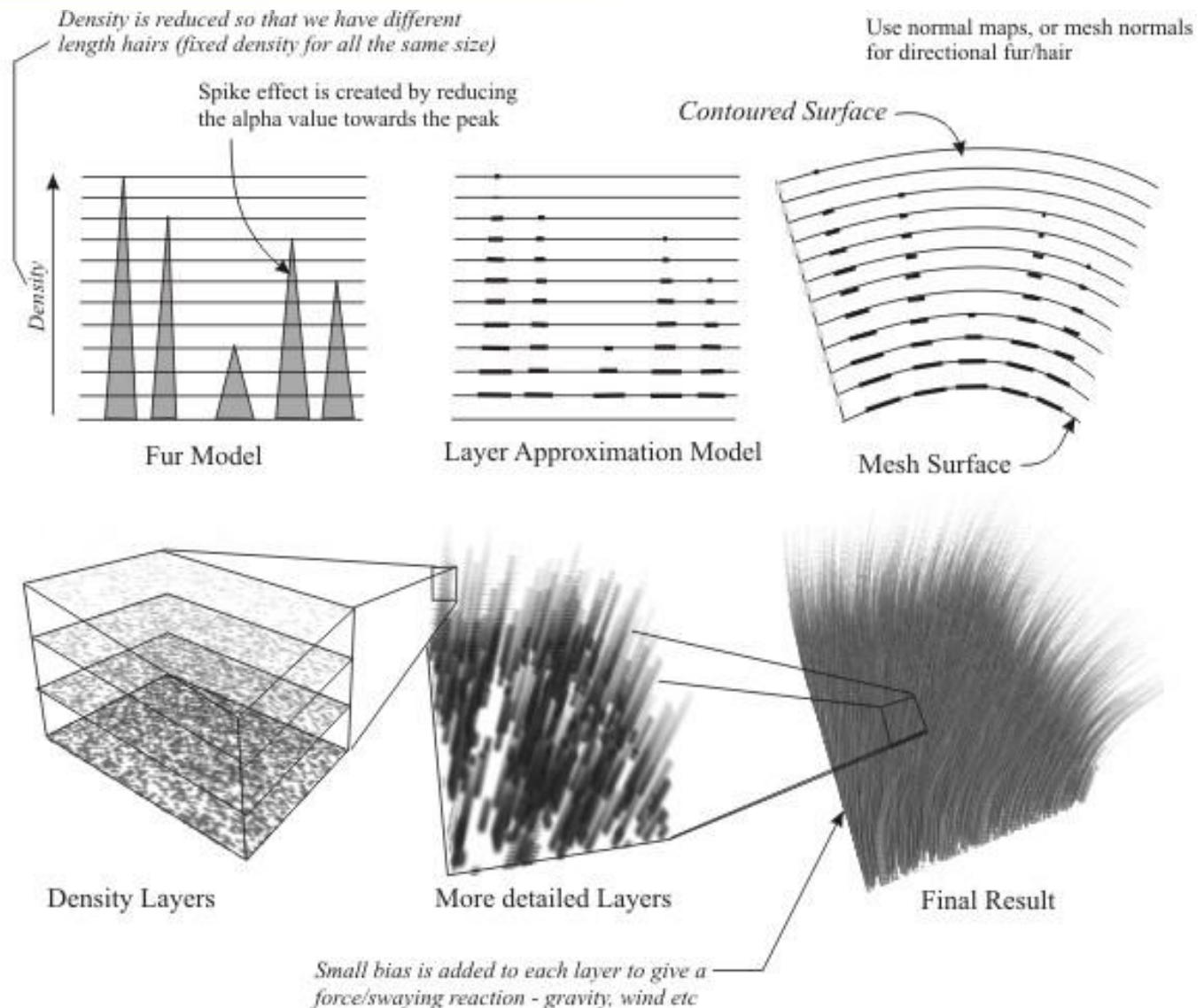
```
void glDrawArraysInstanced(GLenum mode, GLint first,
GLsizei count, GLsizei primcount);
```

```
void glDrawElementsInstanced(GLenum mode, GLsizei
count, GLenum type, const void * indices, GLsizei
primcount);
```

Instancing (2)

- Procedury te różnią się od swoich odpowiedników jednym (ostatnim) parametrem określającym liczbę kopii obiektu.
- Aby istniała możliwość modyfikacji poszczególnych kopii, vertex shader posiada predefiniowaną zmienną `gl_InstanceID` która zawiera numer kopii z której pochodzi wierzchołek.

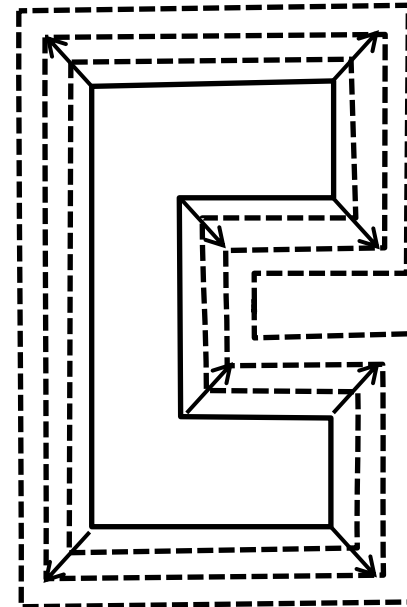
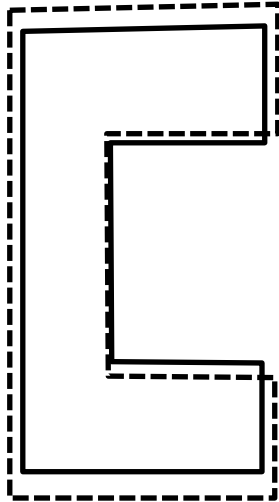
Efekt futra (1)



Efekt futra (2)

- Efekt futra powstaje poprzez wielokrotne rysowanie tego samego modelu coraz większego.
- Powiększenie obiektu nie odbywa się za pomocą skalowania, ale...
- Kolejna większa wersja modelu powstaje poprzez dodanie do współrzędnych wierzchołka jego wektora normalnego wymnożonego razy niewielki współczynnik.
- Takie skalowanie daje efekt „balonika”.

Skalowanie modelu

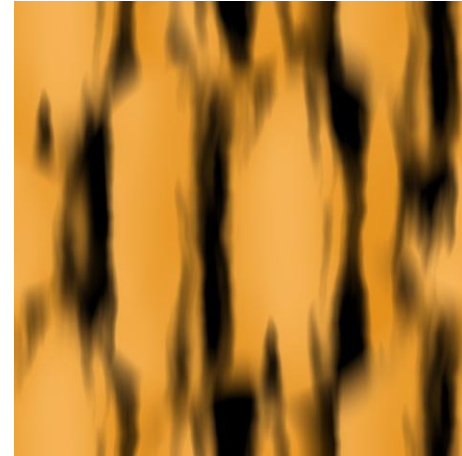


Efekt futra (3)

Tekstura futra
(kanał alfa 0 lub 1)



Tekstura kolorów



Efekt futra (4)

- Najpierw należy narysować oryginalny model pokryty teksturą kolorów.
- Każda kolejna wersja powinna być narysowana w następujący sposób:
 - Pobierz alfa z tekstury futra
 - Jeżeli $\alpha=0$ odrzuć fragment (polecenie **discard;**)
 - W przeciwnym wypadku pobierz kolor futra z tekstury kolorów i zapisz do niego alfa z tekstury futra.
 - Zwróć kolor fragmentu

Ćwiczenie 11

- Niniejsze ćwiczenie demonstruje wykorzystanie mechanizmu instancing do pokrycia obiektu futrem.
- Poszczególne warstwy opisane w algorytmie futra będą kolejnymi kopiami/warstwami modelu.
- W ćwiczeniu wykorzystaj model czajnika. Na kostce nie wygląda to najlepiej (wymagane są uśrednione wektory normalne, a kostka ich nie ma).

Ćwiczenie 11a

- Vertex shader:

- Dodaj 2 zmienne jednorodne typu float:

```
uniform float maxFurLength; //długość futra  
uniform float maxLayer; //liczba rysowanych warstw
```

- Dodaj 1 zmienną interpolowaną typu float (**nie zapomnij również o FS!**):

```
out float layer; //aktualna warstwa
```

- Wylicz nową współrzędną wierzchołka dla konkretnej warstwy i ją wykorzystaj do obliczenia `gl_Position`, `iL` i `iV`

```
layer=gl_InstanceID;//Numer warstwy, to numer kopii  
vec4 nv=vertex+(layer*maxFurLength/maxLayer)*  
    normalize(normal);  
gl_Position=P*V*M*nv;//usuń poprzednie gl_Position!
```

- Dodaj kod potrzebny do interpolacji jednego zestawu współrzędnych teksturowania (atrybut, zmienna interpolowana i przypisanie) – **jeżeli go jeszcze nie ma**.

Ćwiczenie 11b

- Fragment shader:
 - Dodaj kod potrzebny do pobrania z tej samej współrzędnej kolorów z dwóch tekstur (dwie zmienne sampler2D, zmienna interpolowana – para tej z VS, dwa pobrania koloru z tekstur) – **też może już być**.
 - oblicz kolor fragmentu, bądź odrzuć fragment zgodnie z algorytmem opisanym powyżej.
Cieniowanie tylko ze światłem rozproszonym.

```
vec4 fur=texture(textureMap0,iTexCoord0); //futro (jest wlos?)
vec4 furCol=texture(textureMap1,iTexCoord0); //kolor futra
vec4 col=furCol; //domyślny kolor
//Jeżeli nie pierwsza warstwa to dodaj alfa z teks. futra
if (layer!=0) col=vec4(furCol.rgb,fur.a);
// Usuń niewidoczny fragment, Z Bufor pozostanie bez zmian
if (fur.a==0 && layer!=0) discard;
pixelColor=col*vec4(nl,nl,nl,1); //wynikowy kolor
```

Ćwiczenie 11c

- OpenGL:
 - Wczytaj dwie tekstury:
 - tekstura futra: fur.png
 - tekstura koloru futra: tiger.png
 - Odpowiednio skonfiguruj teksturowanie (**pewnie już jest skonfigurowane**)
 - W drawScene przekaż do zmiennych jednorodnych maxFurLength i maxLayer odpowiednie wartości 0.1 i 100 (jak skończysz ćwiczenie to poeksperymentuj).
 - W drawScene rysuj obiekt za pomocą glDrawArraysInstanced. Liczba kopii=maxLayer.
 - Włącz obsługę przeźroczystości (initOpenGLProgram):

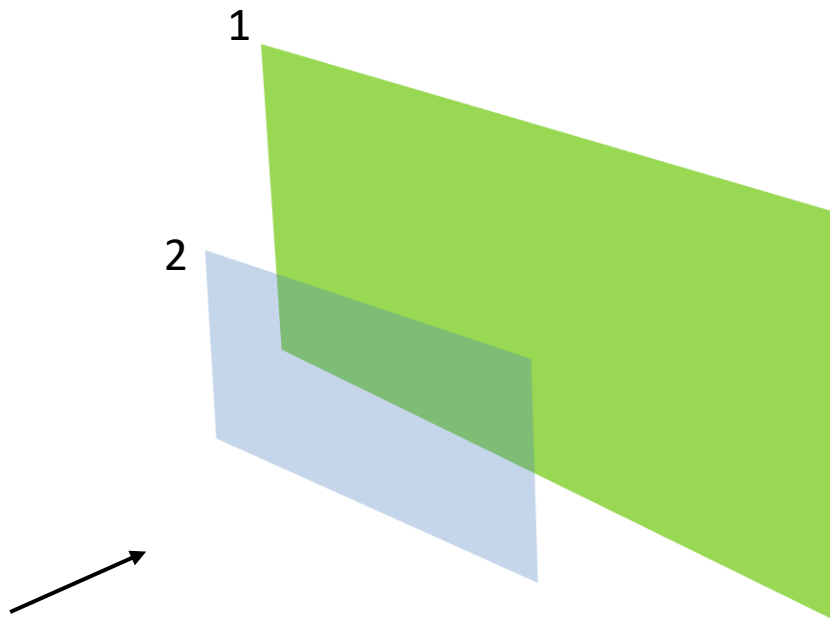
```
glEnable(GL_BLEND);  
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
```

Parę słów o przeźroczystości

- Linijki na ostatnim slajdzie mówią OpenGL, że nowy rysowany kolor ma być łączony z poprzednim za pomocą wzoru:
$$c_{buf}.rgb = c_{new}.a * c_{new}.rgb + (1 - c_{new}.a) * c_{old}.rgb$$
- Gdzie c_{new} to kolor zwracany z fragment shadera, c_{old} to kolor już narysowany, a c_{buf} to nowy kolor, który ma być zapisany do bufora kolorów.
- Nie jest to zatem prawdziwa przeźroczystość.
- Dla wielu półprzeźroczystych warstw oznacza to, że aby ostateczny wynik był poprawny, należy je rysować od najdalszej do najbliższej.

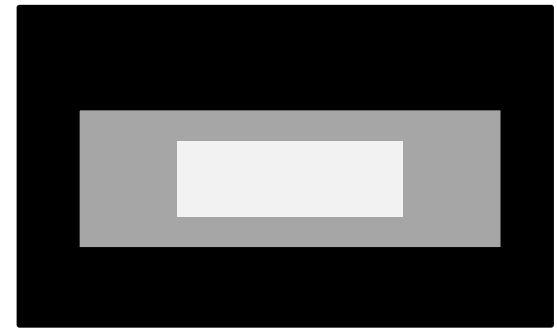
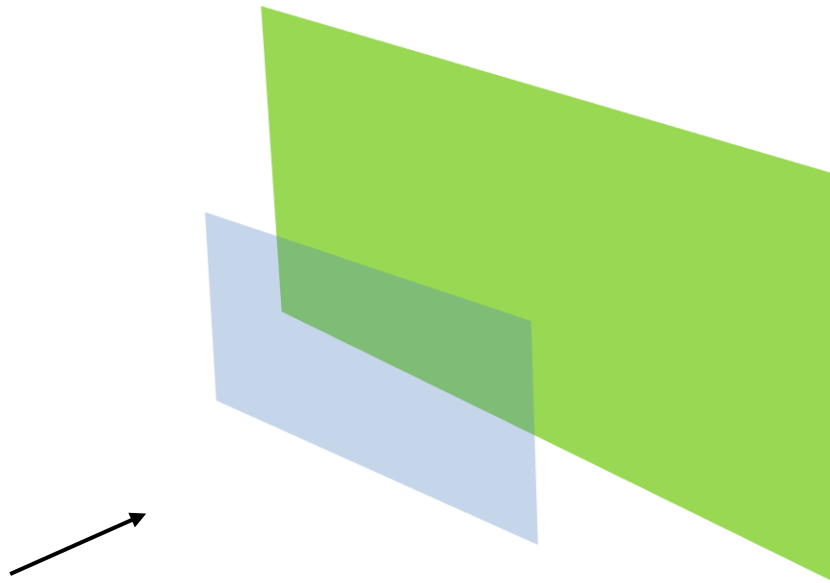
Parę słów o przeźroczystości

- Z bufor w tym przeszkadza a nie pomaga.



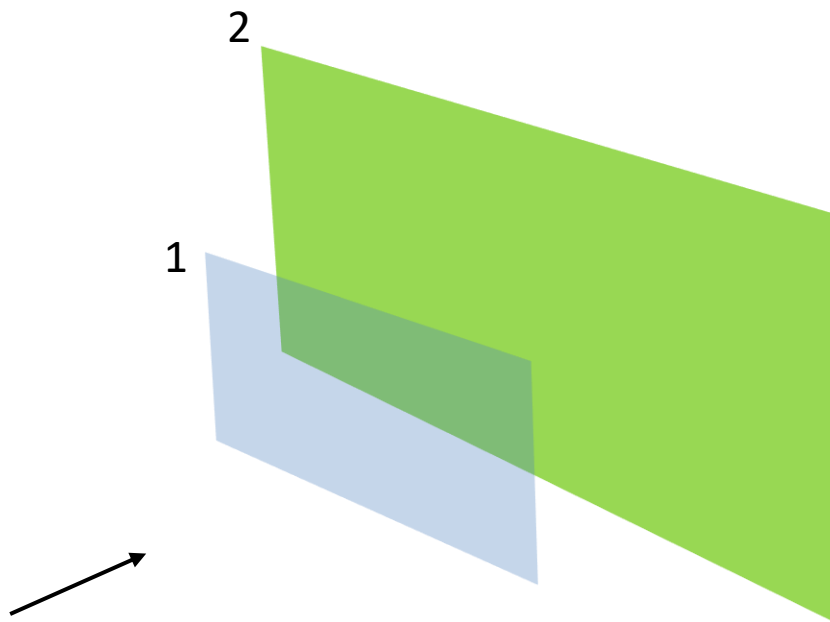
Parę słów o przeźroczystości

- Z bufor w tym przeszkadza a nie pomaga.



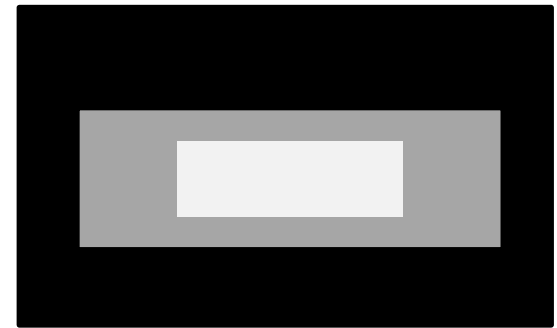
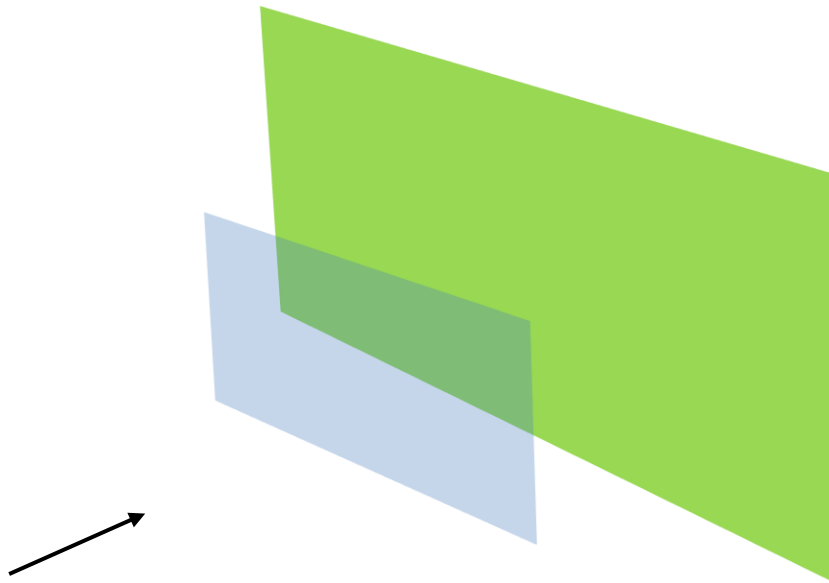
Parę słów o przeźroczystości

- Z bufor w tym przeszkadza a nie pomaga.



Parę słów o przeźroczystości

- Z bufor w tym przeszkadza a nie pomaga.



Parę słów o przeźroczystości

- Niezależnie od poprzednich wniosków dotyczących poprawności, Z-Bufor również wymusza rysowanie od najdalszej do najbliższej powierzchni.
- Jedyny „prawdziwie” przeźroczysty pixel to taki, który został unieważniony za pomocą **discard**. Ale on jest całkowicie przeźroczysty.
- W przypadku futra oznacza to, że niektóre obiekty mogą być rysowane błędnie jeżeli będą się nakładać na futrzany obiekt, bądź będą się za nim znajdować, ale będą narysowane po nim.

Ćwiczenie 12

- Dodaj wpływ grawitacji na futro.
- VS: Dodaj zmienną reprezentującą siłę i kierunek grawitacji w przestrzeni świata:

```
vec4 vGravity=vec4(0,-0.08,0,0);
```

- VS: Przelicz ten wektor do przestrzeni modelu:

```
vGravity=inverse(M)*vGravity;
```

- VS: zmodyfikuj współrzędne wierzchołka warstwy (nv) tak aby uwzględnić wpływ grawitacji na włos (**wzór empiryczny**):

```
nv=nv+vGravity*pow(layer/maxLayer,2);
```



Grafika Komputerowa i Wizualizacja

Geometry shader

Potok przetwarzania danych

P, V, M ●

Jednorodne

Atrybuty

$gl_Position$

VS

GS

RASTERYZACJA

ŁĄCZENIE
WYNIKOW

Jednorodne

FS

out

out

$gl_Position$

fragmenty

Współrzędne

Kolory

Geometry shader

- VS wykonuje się dla każdego wierzchołka niezależnie.
- FS wykonuje się dla każdego fragmentu ($\sim$ piksela) niezależnie.
- GS jest czymś pośrednim wykonywanym przed rasteryzacją.
- GS wykonuje się dla każdego prymitywu (punktu/odcinka/trójkąta) niezależnie.
- GS na podstawie pojedynczego prymitywu może wygenerować wiele prymitywów innego typu.
- GS może pełnić większość funkcji VS.
- GS może odczytać wartości przypisywane do zmiennych „out” w VS i zmienne jednorodne.

Przykładowy VS

```
#version 330

uniform mat4 P;
uniform mat4 V;
uniform mat4 M;

in vec4 vertex;
in vec4 color;

out vec4 gColor;

void main(void) {
    gColor=color;
    gl_Position=P*V*M*vertex;
}
```

PASS-Through GS

```
#version 330

layout (triangles) in;
layout (triangle_strip) out;
layout (max_vertices=3) out;

in vec4 gColor[];
out vec4 iColor;

void main(void) {
    int i;

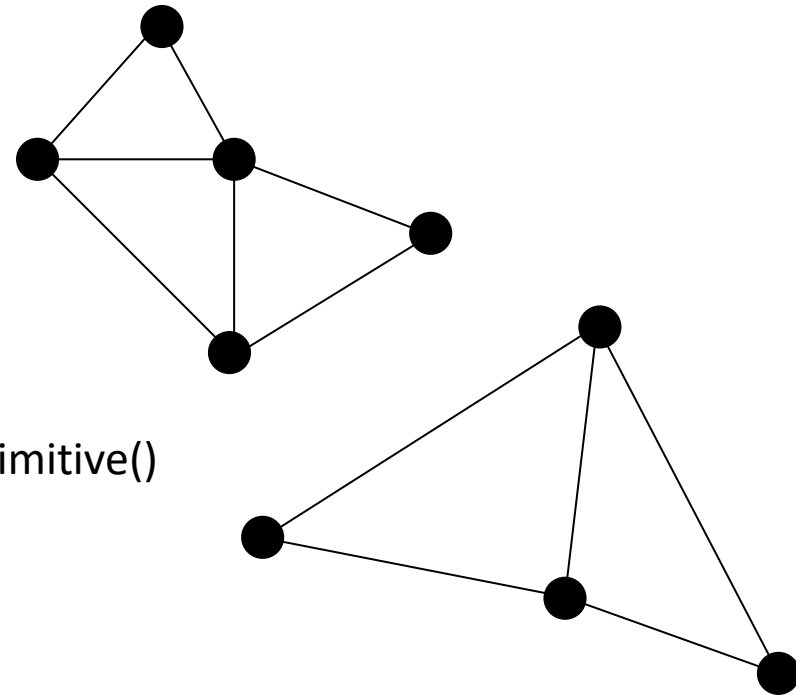
    for (i=0;i<gl_in.length();i++) {
        gl_Position=gl_in[i].gl_Position;
        iColor=gColor[i];
        EmitVertex();
    }
    EndPrimitive();
}
```


Wynik GS

	gl_Position	iColor
1		
2		
3		
4		
5		
6		
7		
8		
9		

EndPrimitive()

EndPrimitive()



Przykładowy FS

```
#version 330

out vec4 pixelColor;
in vec4 iColor;

void main(void) {
    pixelColor=iColor;
}
```

layout

- Dopuszczalne wartości dla input layout:

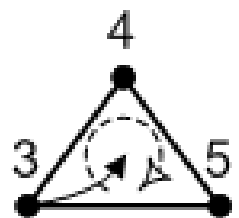
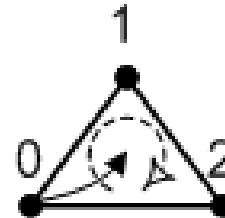
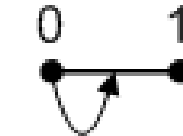
- points 0 1 2 3

- lines

- triangles

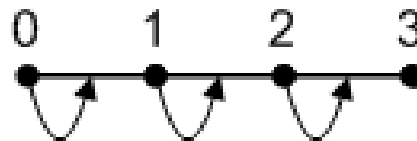
- lines_adjacency

- triangles_adjacency



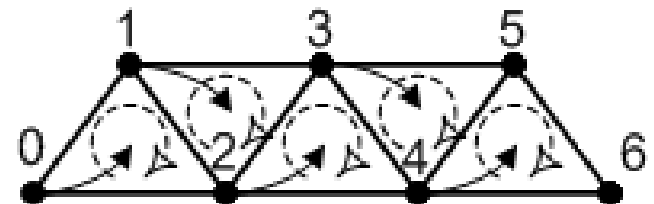
- Dopuszczalne wartości dla output layout:

- points



- line_strip

- triangle_strip



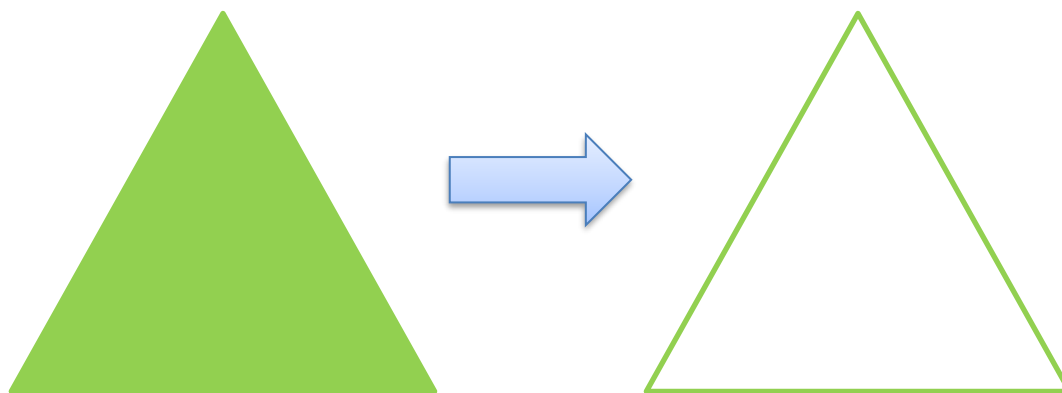
Ładowanie GS w OpenGL

- W przykładowym programie GS może być załadowany za pomocą biblioteki ShaderProgram.
- Wystarczy w procedurze initOpenGLProgram dodać następujące zmiany:

```
//w initOpenGLProgram  
sp=new ShaderProgram(  
    "v_simplest.glsl","g_simplest.glsl","f_simplest.glsl");
```

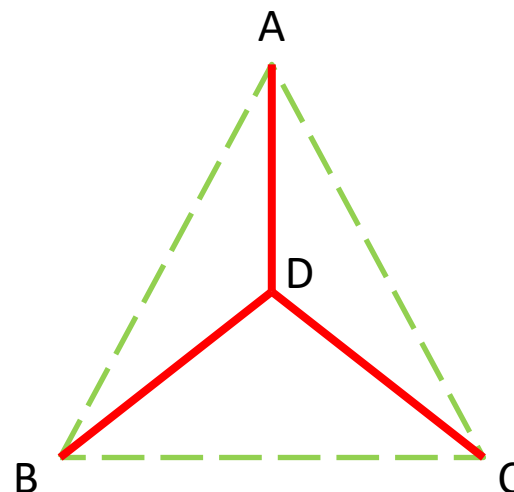
Ćwiczenie 13

- Zmodyfikuj GS tak, żeby trójkąty które dostaje na wejściu konwertował na 3 odcinki reprezentujące jego boki.
- Zadaż o to aby kolory były przekazywane poprawnie.



Ćwiczenie 14

- Zmodyfikuj program tak, aby każdy trójkąt wejściowy był zamieniany na 3 odcinki zgodnie z rysunkiem:



- Gdzie punkt D to średnia z punktów A, B i C:

$$\frac{A + B + C}{3}$$

Ćwiczenie 15a

- Przywróć oryginalny GS.
- Zbuduj shader „eksplodujący” model.
- Eksplozja polega na przesunięciu każdego wierzchołka prymitywu o określoną odległość względem nieuśrednionego wektora normalnego.
- Najwygodniej jest eksplozję wykonać w przestrzeni modelu.
- Aby GS miał dostęp do współrzędnych wierzchołka w przestrzeni modelu przejmie on większość funkcji VS.
- GS będzie sam będzie obliczał nieuśrednione wektory dla każdego prymitywu.

Ćwiczenie 15b

- VS:
 - Nie powinien obliczać `gl_Position`
 - Powinien posiadać zmienną wyjściową np. `gVertex`, poprzez którą przekaże do GS współrzędne wierzchołka w przestrzeni modelu.
- FS:
 - Bez zmian

Ćwiczenie 15c

- GS:
 - Deklaruje zmienne jednorodne P, V i M
 - Deklaruje zmienną jednorodną *amount* typu float. Zmienna ta zawiera wielkość przesunięcia prymitywu wzdłuż wektora.
 - Pobiera wszystkie wierzchołki trójkąta w przestrzeni modelu (gVertex).
 - Oblicza wektor normalny trójkąta. Wykorzystaj funkcje cross, normalize i operator -.
 - Dodaje do każdego wierzchołka iloczyn amount i wektora normalnego (przesunięcie).
 - Przelicza nowe współrzędne do przestrzeni PRZYCIĘCIA.
 - Emituje nowy trójkąt, podając nowe współrzędne jako gl_Position.

Ćwiczenie 15d

- OpenGL:
 - Zapisz do zmiennej jednorodnej *amount* wartość liczbową zmieniającą się w czasie.
 - Wartość ta raczej nie powinna być większa niż 2 i nie mniejsza niż 0.

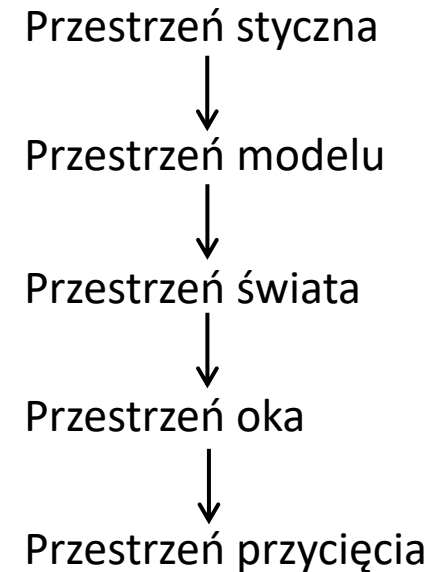
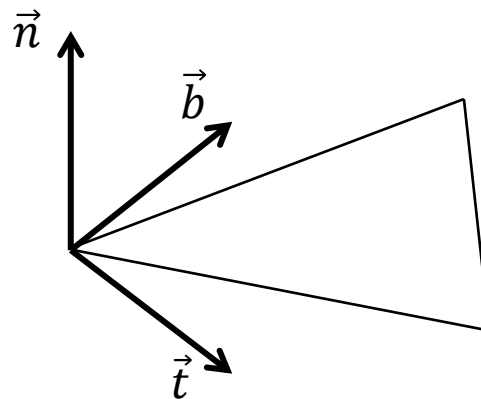


Grafika Komputerowa i Wizualizacja

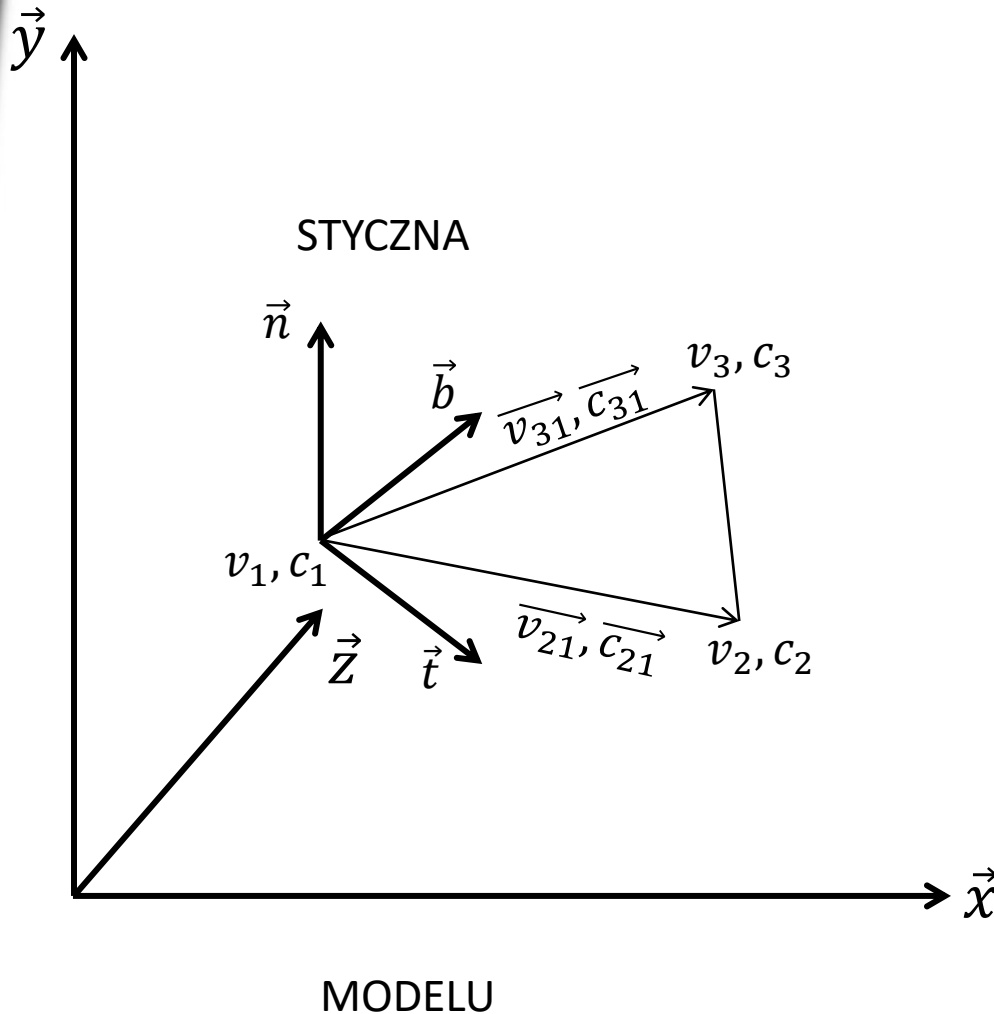
Efekty w przestrzeni stycznej

Przestrzeń styczna

- Przestrzeń związana z pojedynczą ścianą modelu
- Osie układu utworzone są przez dwa wektory styczne: tangent $\vec{t}$, bitangent $\vec{b}$ oraz wektor normalny $\vec{n}$.



Macierz M_{TBN}



$$\begin{aligned} \overrightarrow{c_{21}} &= c_2 - c_1 & \overrightarrow{v_{21}} &= v_2 - v_1 \\ \overrightarrow{c_{31}} &= c_3 - c_1 & \overrightarrow{v_{31}} &= v_3 - v_1 \end{aligned}$$

$$\vec{v}_{21} = \vec{c}_{21}^t \cdot \vec{t} + \vec{c}_{21}^b \cdot \vec{b}$$

$$\vec{v}_{31} = \vec{c}_{31}^t \cdot \vec{t} + \vec{c}_{31}^b \cdot \vec{b}$$

$$\begin{bmatrix} \vec{v}_{21} \\ \vec{v}_{31} \end{bmatrix} = \begin{bmatrix} \vec{c}_{21}^t & \vec{c}_{21}^b \\ \vec{c}_{31}^t & \vec{c}_{31}^b \end{bmatrix} \begin{bmatrix} \vec{t} \\ \vec{b} \end{bmatrix}$$

$$\begin{bmatrix} \vec{t} \\ \vec{b} \end{bmatrix} = \begin{bmatrix} \vec{c}_{21}^t & \vec{c}_{21}^b \\ \vec{c}_{31}^t & \vec{c}_{31}^b \end{bmatrix}^{-1} \begin{bmatrix} \vec{v}_{21} \\ \vec{v}_{31} \end{bmatrix}$$

$$\vec{n} = \vec{t} \times \vec{b}$$

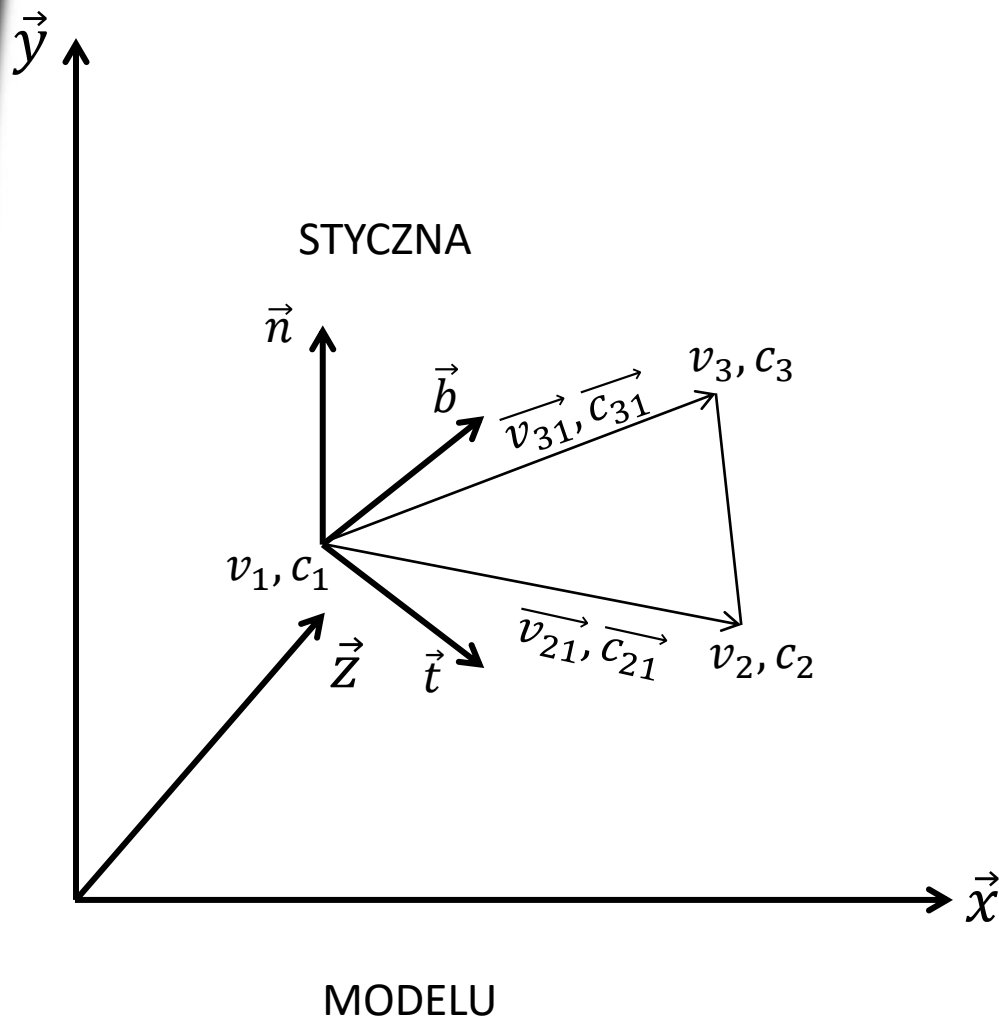
$$\vec{t} = \vec{t} \setminus |\vec{t}|$$

$$\vec{b} = \vec{b} \setminus |\vec{b}|$$

$$\vec{n} = \vec{n} \setminus |\vec{n}|$$

Macierz M_{TBN}

$$M_{TBN} = \begin{bmatrix} \vec{t}^x & \vec{b}^x & \vec{n}^x & 0 \\ \vec{t}^y & \vec{b}^y & \vec{n}^y & 0 \\ \vec{t}^z & \vec{b}^z & \vec{n}^z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



Macierz M_{TBN}

STYCZNA

MODELU

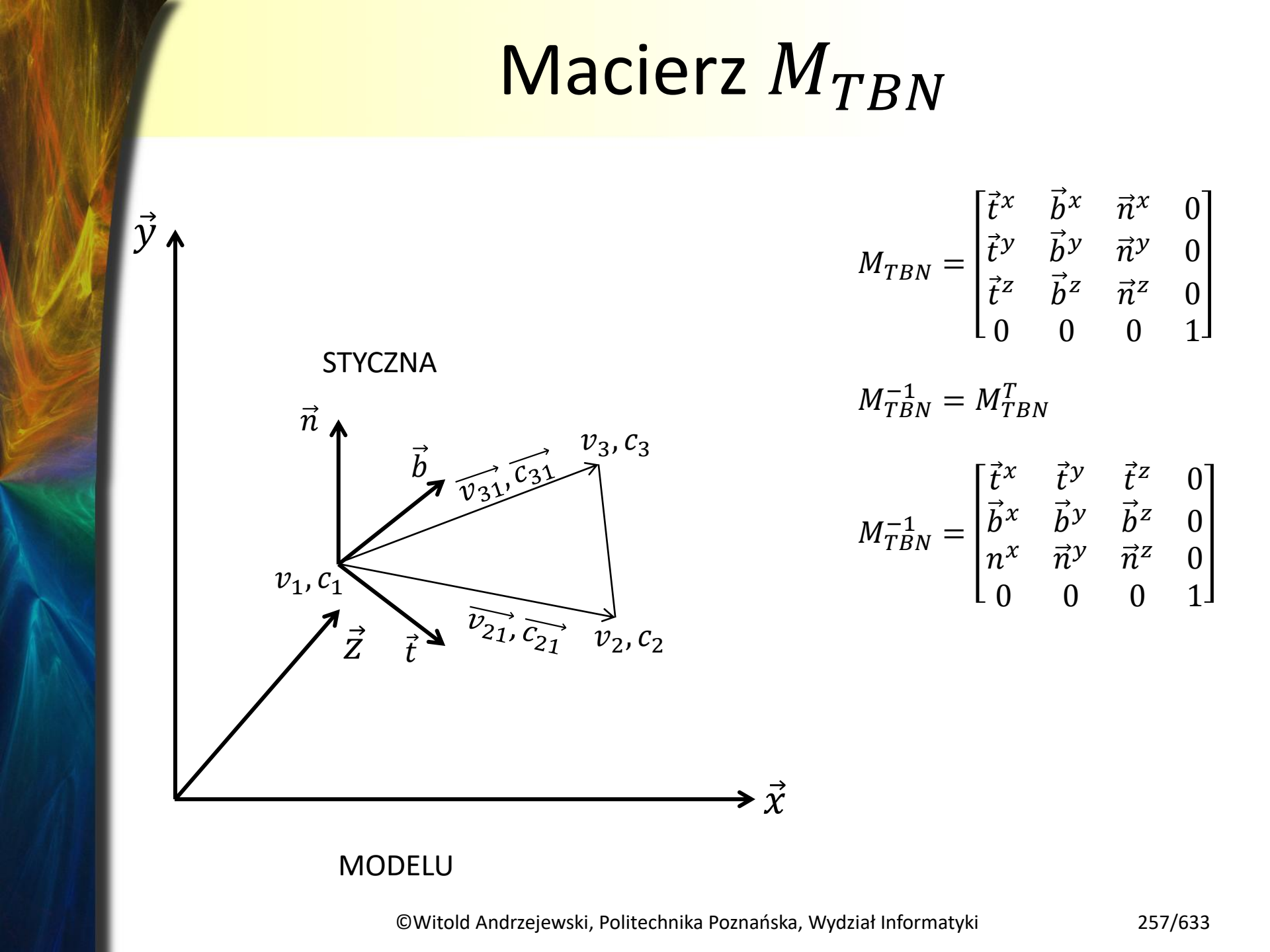
$$M_{TBN} = \begin{bmatrix} \vec{t}^x & \vec{b}^x & \vec{n}^x & 0 \\ \vec{t}^y & \vec{b}^y & \vec{n}^y & 0 \\ \vec{t}^z & \vec{b}^z & \vec{n}^z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_{TBN}^{-1} = M_{TBN}^T$$

$$M_{TBN}^{-1} = \begin{bmatrix} \vec{t}^x & \vec{t}^y & \vec{t}^z & 0 \\ \vec{b}^x & \vec{b}^y & \vec{b}^z & 0 \\ \vec{n}^x & \vec{n}^y & \vec{n}^z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

©Witold Andrzejewski, Politechnika Poznańska, Wydział Informatyki

257/633

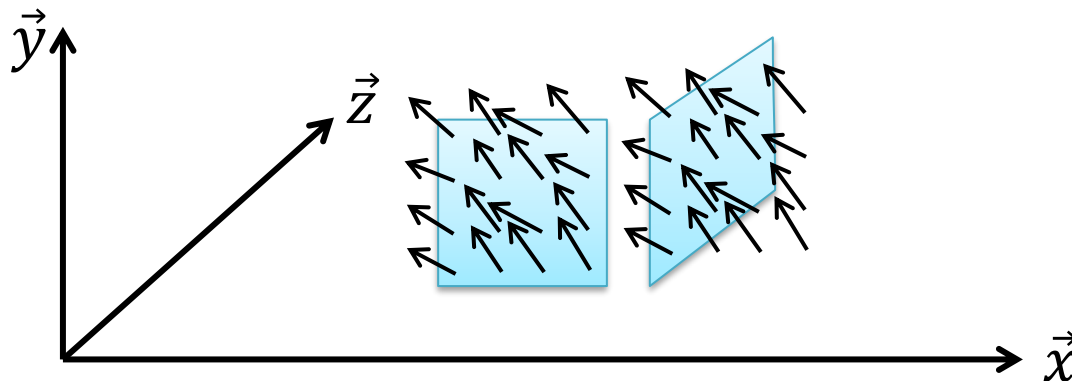


$$M_{TBN}^{-1} = \begin{bmatrix} \vec{t}^x & \vec{t}^y & \vec{t}^z & 0 \\ \vec{b}^x & \vec{b}^y & \vec{b}^z & 0 \\ n^x & \vec{n}^y & \vec{n}^z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Normal Mapping (1)

- W teksturach można przechowywać wektory normalne.
- Problem:

Mamy teksturę z normalnymi zwróconymi mniej więcej wzdłuż osi z. Co będzie jeżeli nałożymy tą samą teksturę na dwie nierównoległe ściany?



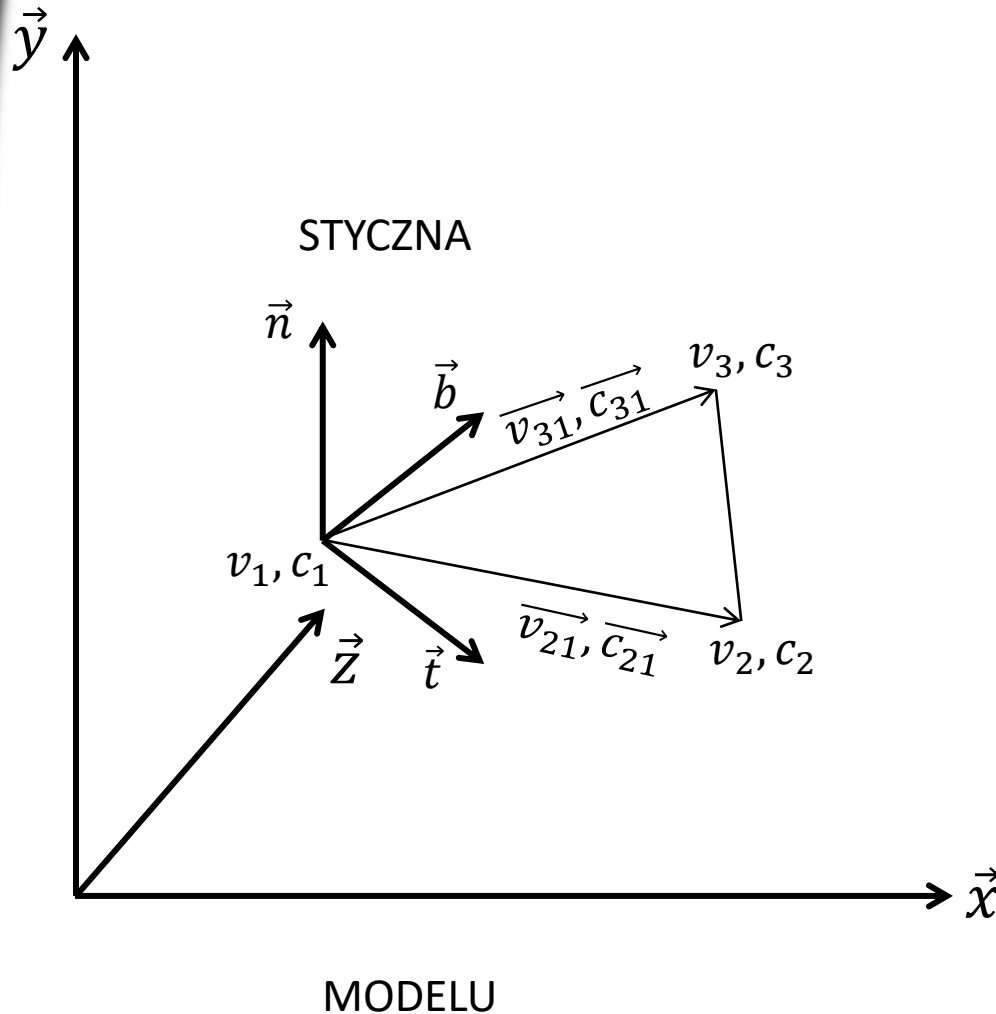
Normal Mapping (2)

- Problem wynika z faktu, że obrót ściany nie wpływa na obrót wektorów normalnych zapisanych w teksturze nałożonej na tą ścianę.
- Wektory w teksturze znajdują się w specjalnym układzie współrzędnych, nazywanym przestrzenią styczną.
- Aby możliwe było cieniowanie, należy:
 - Dla każdego rysowanego piksela przenieść wektor normalny z przestrzeni stycznej do przestrzeni oka, w celu obliczenia modelu oświetlenia.
 - **Albo**: przenieść do przestrzeni stycznej wektory $\vec{l}$ i $\vec{v}$, i interpolować je, a model oświetlenia obliczać właśnie w tej przestrzeni. Jest to rozwiązanie znacznie bardziej wydajne. **W dalszych rozważaniach będzie mowa o tym rozwiązaniu.**

Normal mapping

- Algorytm normal mappingu opiera się na przeniesieniu obliczeń z przestrzeni oka do przestrzeni stycznej:
 1. W VS oblicz współrzędne wektorów $\vec{v}$ i $\vec{l}$ w przestrzeni modelu.
 2. Przenieś je do przestrzeni tangent mnożąc razy macierz M_{TBN}^{-1} .
 3. Interpoluj te wektory dla każdego fragmentu.
 4. We fragment shaderze pobierz z tekstury wektor normalny.
 5. Pozostałe obliczenia wykonuj jak zawsze.

Macierz M_{TBN}

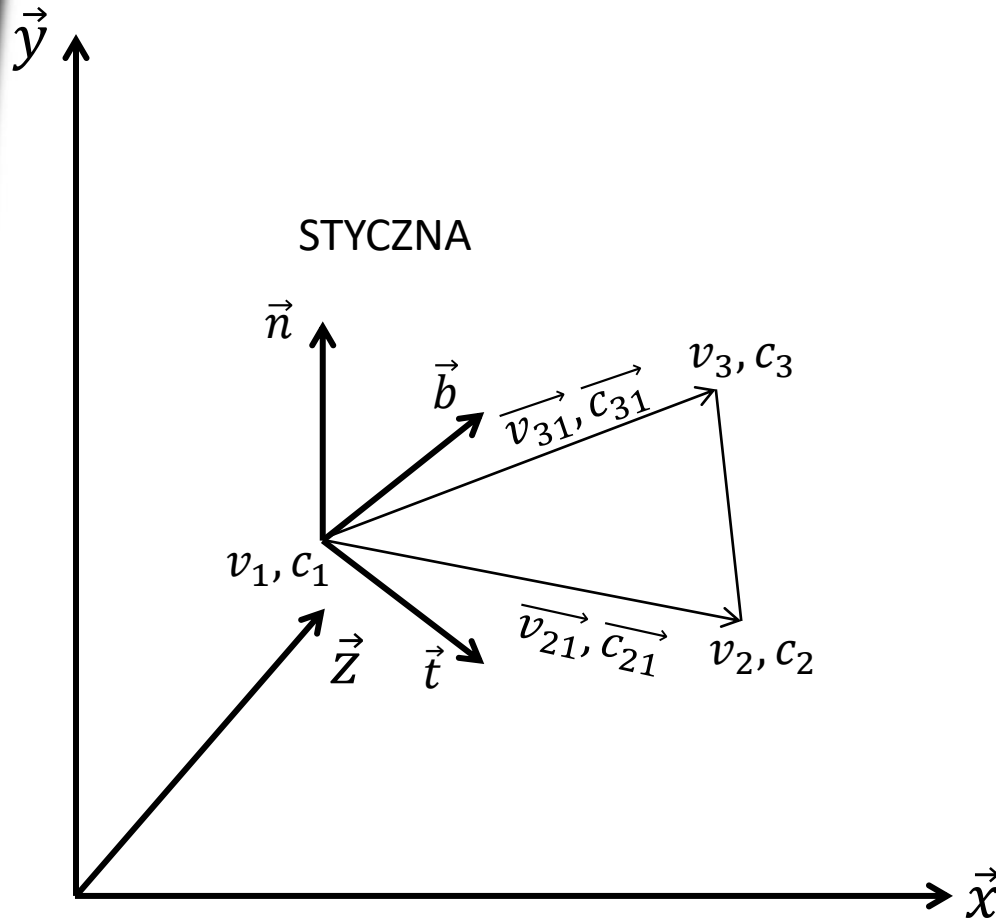


$$M_{TBN} = \begin{bmatrix} \vec{t}^x & \vec{b}^x & \vec{n}^x & 0 \\ \vec{t}^y & \vec{b}^y & \vec{n}^y & 0 \\ \vec{t}^z & \vec{b}^z & \vec{n}^z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_{TBN}^{-1} = M_{TBN}^T$$

$$M_{TBN}^{-1} = \begin{bmatrix} \vec{t}^x & \vec{t}^y & \vec{t}^z & 0 \\ \vec{b}^x & \vec{b}^y & \vec{b}^z & 0 \\ \vec{n}^x & \vec{n}^y & \vec{n}^z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Macierz M_{TBN}



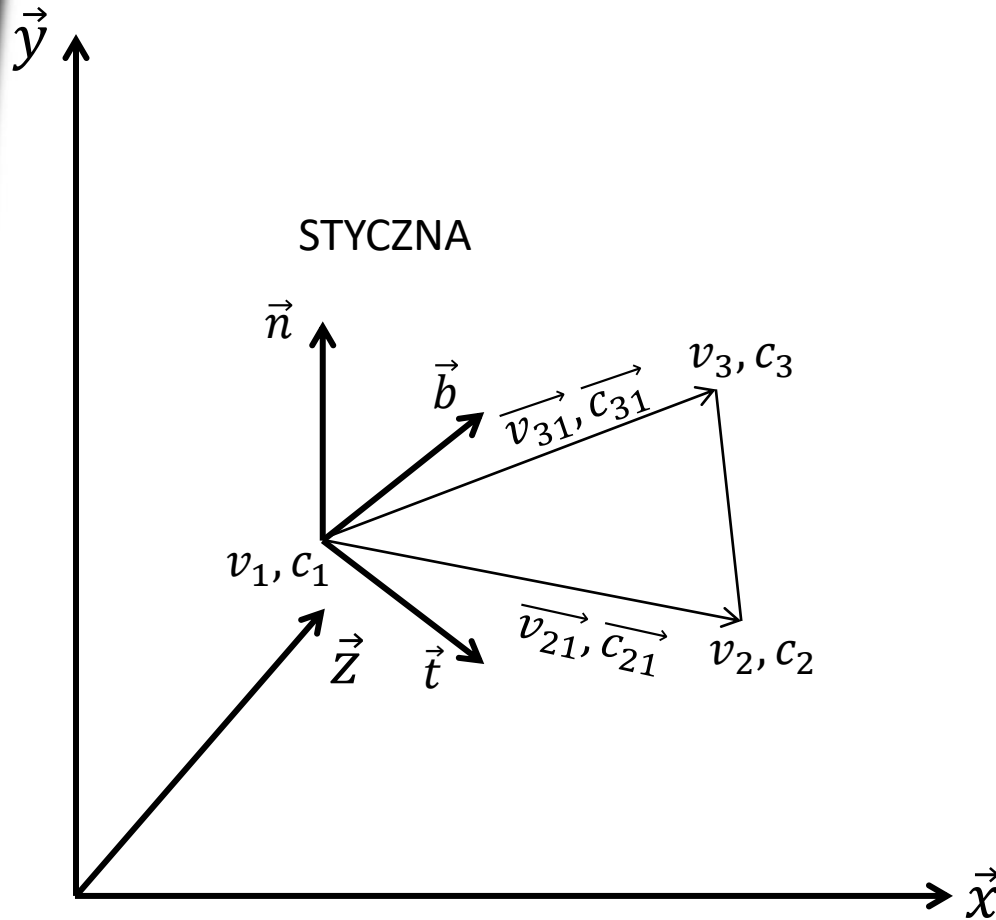
$$M_{TBN} = \begin{bmatrix} \vec{t}^x & \vec{b}^x & \vec{n}^x & 0 \\ \vec{t}^y & \vec{b}^y & \vec{n}^y & 0 \\ \vec{t}^z & \vec{b}^z & \vec{n}^z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_{TBN}^{-1} = M_{TBN}^T$$

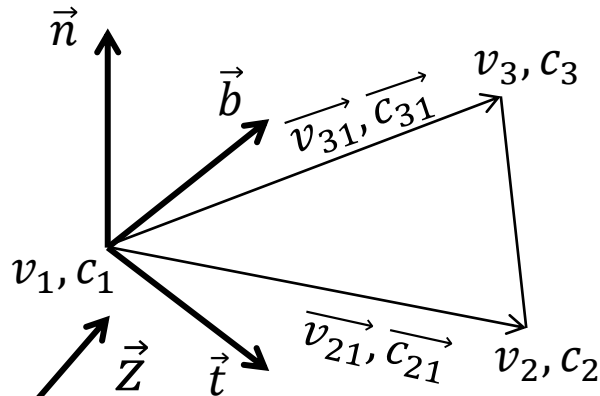
$$M_{TBN}^{-1} = \begin{bmatrix} \vec{t}^x & \vec{t}^y & \vec{t}^z & 0 \\ \vec{b}^x & \vec{b}^y & \vec{b}^z & 0 \\ n^x & n^y & n^z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$c_1 \quad c_2 \quad c_3 \quad c_4$

Macierz M_{TBN}



STYCZNA



MODELU

$$M_{TBN} = \begin{bmatrix} \vec{t}^x & \vec{b}^x & \vec{n}^x & 0 \\ \vec{t}^y & \vec{b}^y & \vec{n}^y & 0 \\ \vec{t}^z & \vec{b}^z & \vec{n}^z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_{TBN}^{-1} = M_{TBN}^T$$

$$M_{TBN}^{-1} = \begin{bmatrix} \vec{t}^x & \vec{b}^x & \vec{n}^x & 0 \\ \vec{t}^y & \vec{b}^y & \vec{n}^y & 0 \\ \vec{t}^z & \vec{b}^z & \vec{n}^z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$c_1 \quad c_2 \quad c_3 \quad c_4$

Wektory normalne w teksturze

- Współrzędne znormalizowanego wektora normalnego mogą należeć do przedziału $[-1,1]$.
- Kolory w teksturze wyrażone są liczbami z przedziału $[0,1]$.
- Konieczna jest konwersja wektora do koloru i na odwrót. Poniżej kod wykonujący konwersję kolor->wektor.

```
vec4 encodedNormal=texture(textureMap1,iTexCoord);  
vec4 decodedNormal=vec4(2*encodedNormal.xyz-1,0);
```

- Ze względu na ograniczoną dokładność kolorów (tylko 4B) warto znormalizować wynik dekodowania.

Ćwiczenie 16a

- Załączone do przykładowego programu modele posiadają tablice reprezentujące kolumny macierzy M_{TBN}^{-1} :
 - myCubeC1, myCubeC2, myCubeC3
 - myTeapotC1, myTeapotC2, myTeapotC3
- Każda z tych kolumn jest wektorem 4 liczb.
- Wykorzystaj je do implementacji normal mappingu.

Ćwiczenie 16b

- OpenGL: zmodyfikuj program tak, aby do zerowej jednostki teksturującej ładowana była tekstura bricks_diffuse.png (kolory), a do pierwszej bricks_normal.png (wektory normalne).
- VS, FS:
 - usuń zmienną interpolowaną odpowiedzialną za wektor normalny, oraz wiersz obliczający jej wartość w VS.
- VS, OpenGL: Dodaj 3 atrybuty przekazujące 3 kolumny macierzy M_{TBN}^{-1} .

Ćwiczenie 16c

- VS:
 - zrekonstruuj macierz M_{TBN}^{-1} korzystając z przekazanych atrybutów (ostatnia kolumna to $[0,0,0,1]$). Wykorzystaj własność konstruktora macierzy `mat4`.
 - zmodyfikuj obliczanie wartości zmiennych interpolowanych odpowiedzialnych za wektory V i L tak, aby wektory te były nie w przestrzeni OKA jak dotychczas, ale w przestrzeni TANGENT. Może się tutaj przydać funkcja GLSL `inverse` obliczająca odwrotność macierzy (przykład):

```
mat4 m; //zawiera jakąś macierz  
mat4 m2=m*inverse(m); //m2 jest macierzą jednostkową.
```

Ćwiczenie 16d

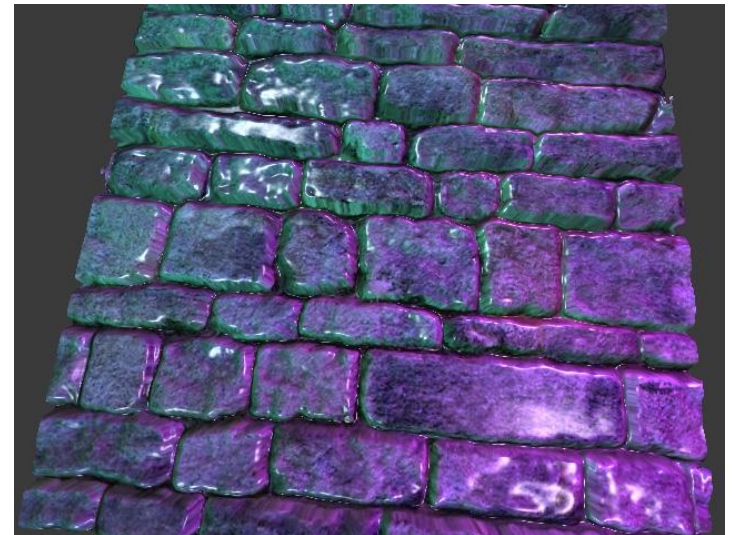
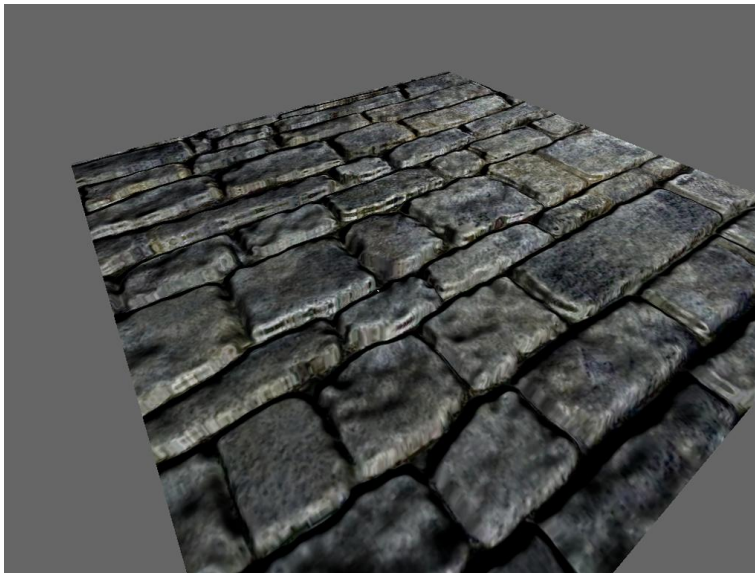
- FS:
 - zamiast pobierać wartość wektora normalnego ze zmiennej interpolowanej, odczytuj go z tekstury umieszczonej w pierwszej jednostce teksturującej (zerowa zawiera kolor).
 - Pamiętaj o odkodowaniu wektora normalnego!
 - Obliczenia modelu oświetlenia wykonaj wykorzystując nowy wektor normalny.
- Wypróbuj nowe cieniowanie zarówno na kostce jak i na czajniku.
- Co jest nie tak z tym ostatnim? Jak można by temu zapobiec?

Ćwiczenie 16e

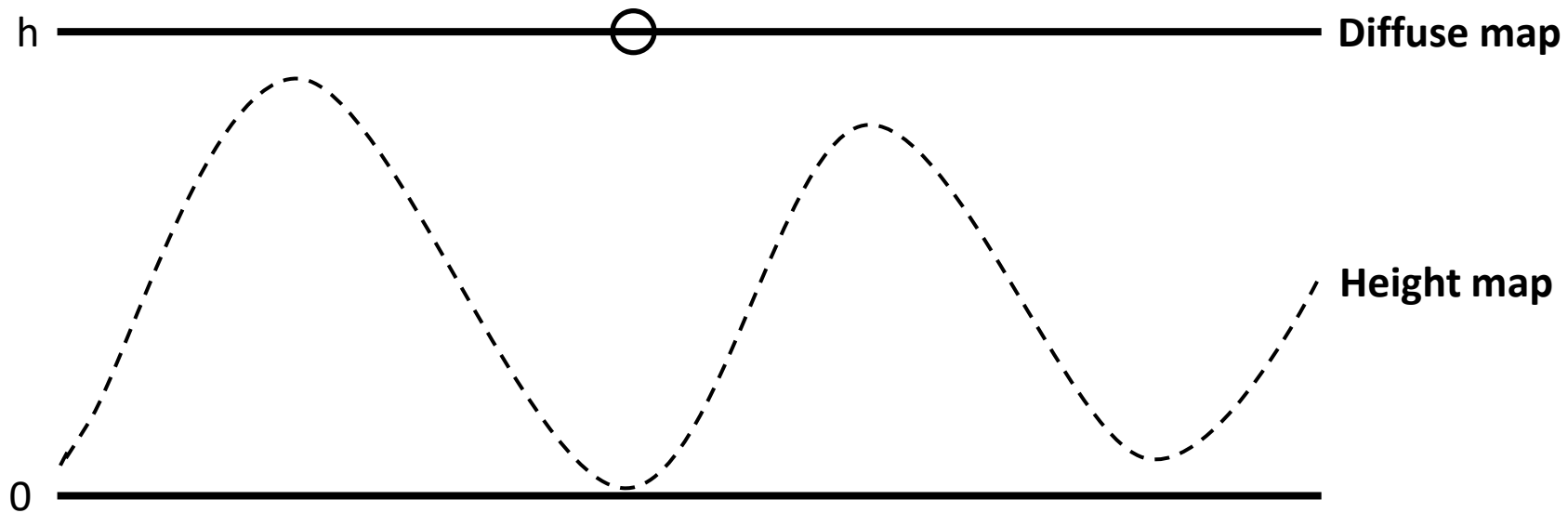
- Nagłówek z modelem czajnika zawiera uśrednione macierze w tablicach: myTeapotC1_2, myTeapotC2_2, myTeapotC3_2
 - Przekaż do atrybutów te macierze zamiast myTeapotC1, myTeapotC2, myTeapotC3
 - Czy tak jest lepiej?
- Przetestuj jeszcze inne tekstury załączone do przykładowego programu:
 - stone_diffuse.png, stone_normal.png

Parallax Mapping

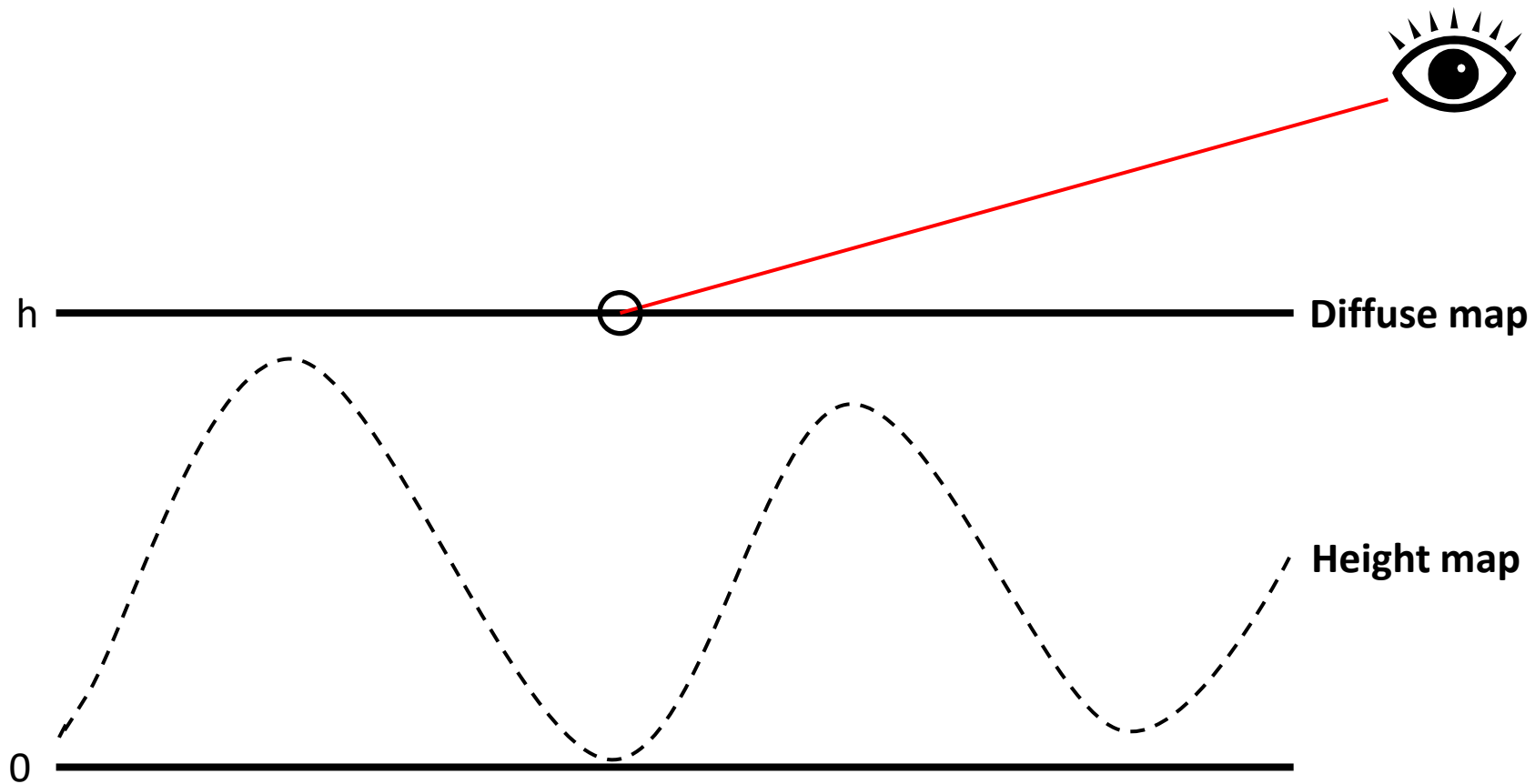
- Wykorzystanie mapy głębokości do uzyskania złudzenia wypukłości.



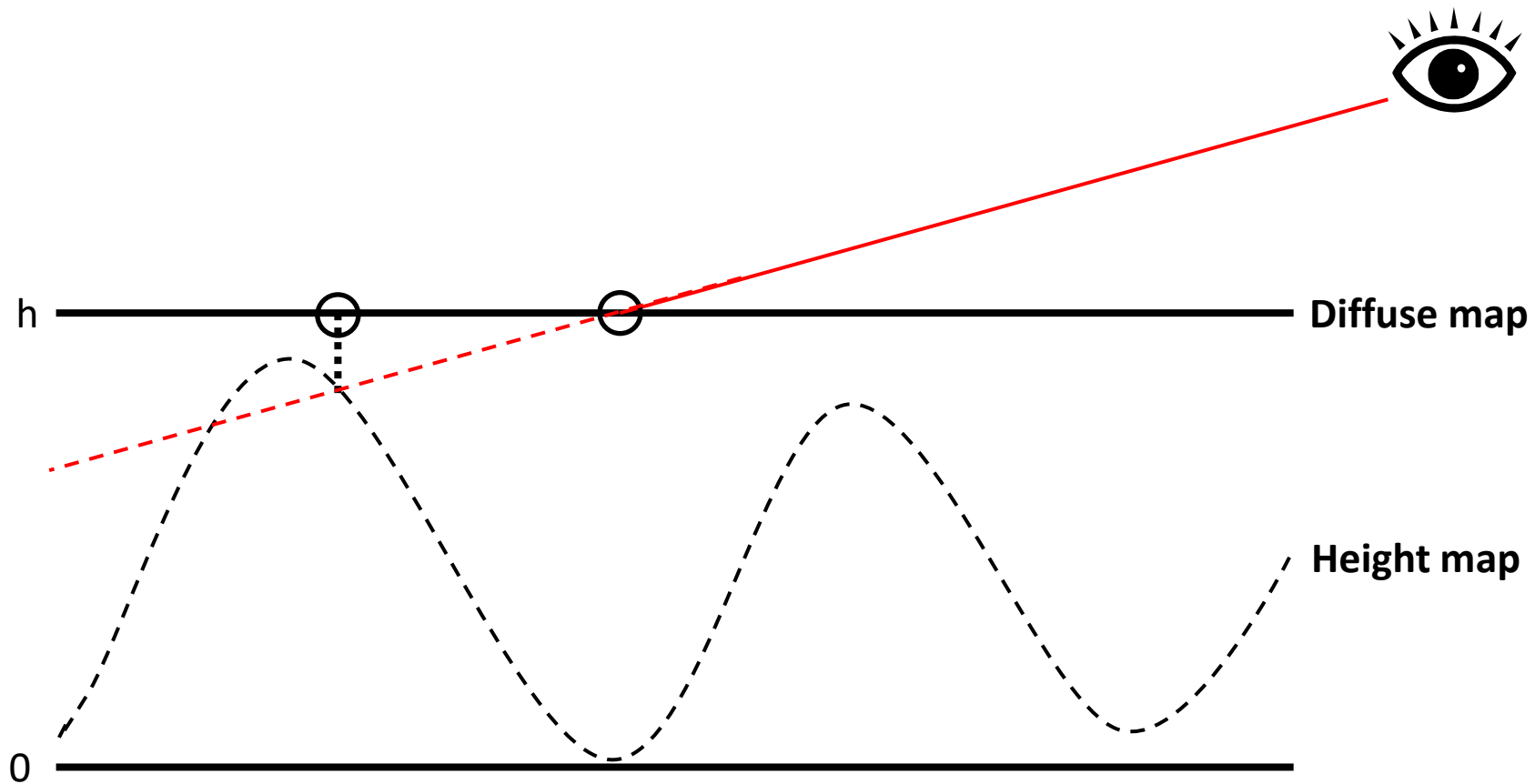
Parallax mapping



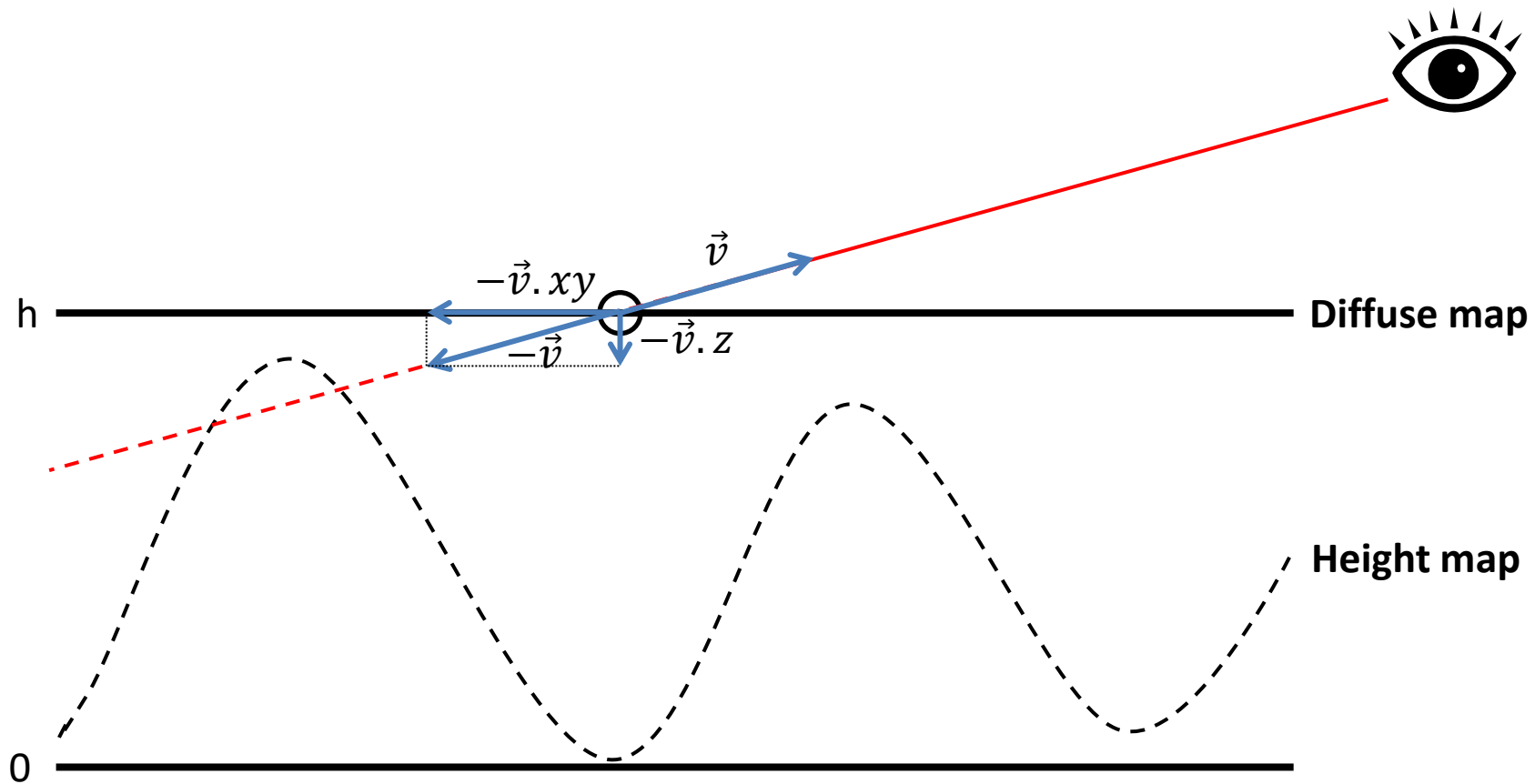
Parallax mapping



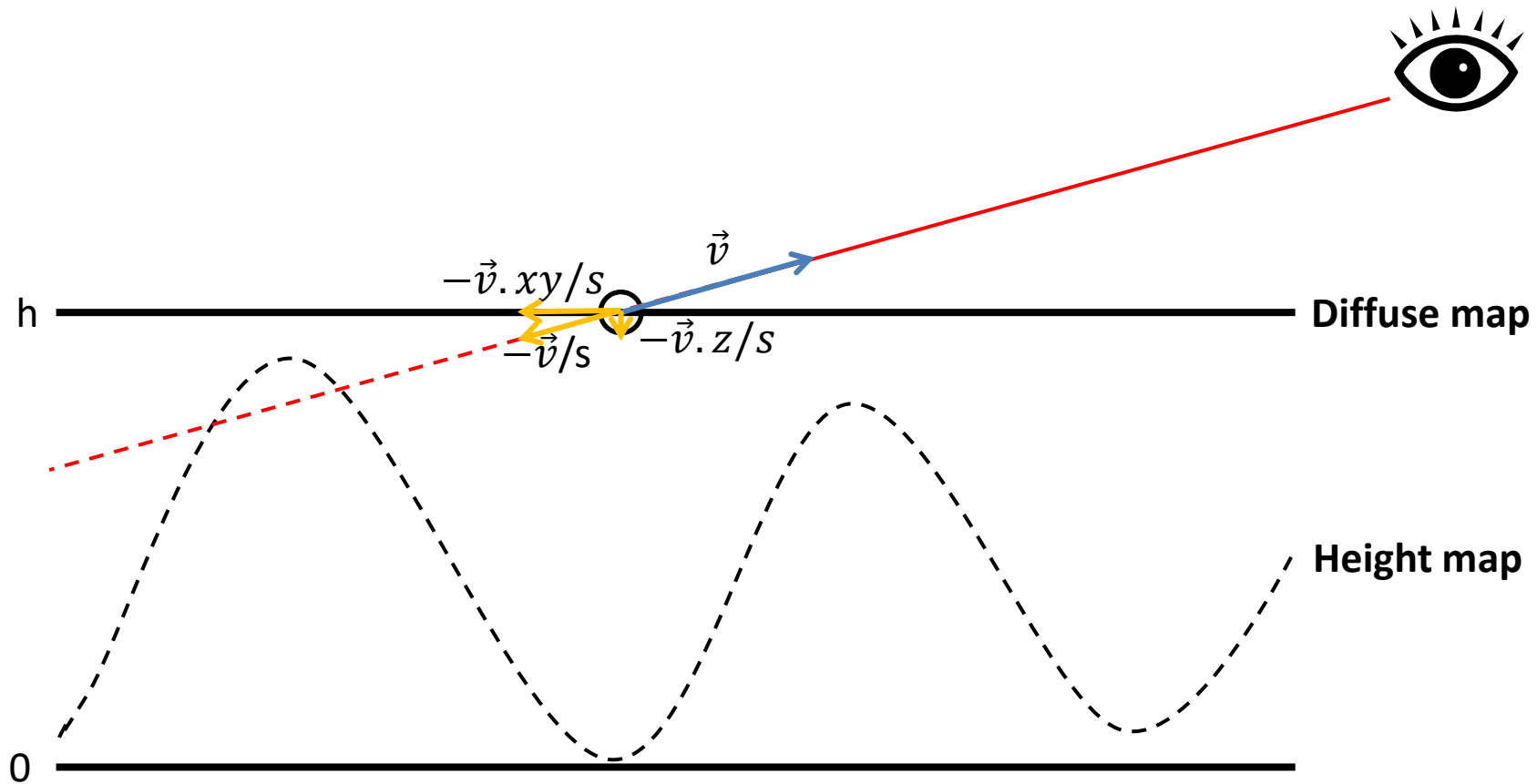
Parallax mapping



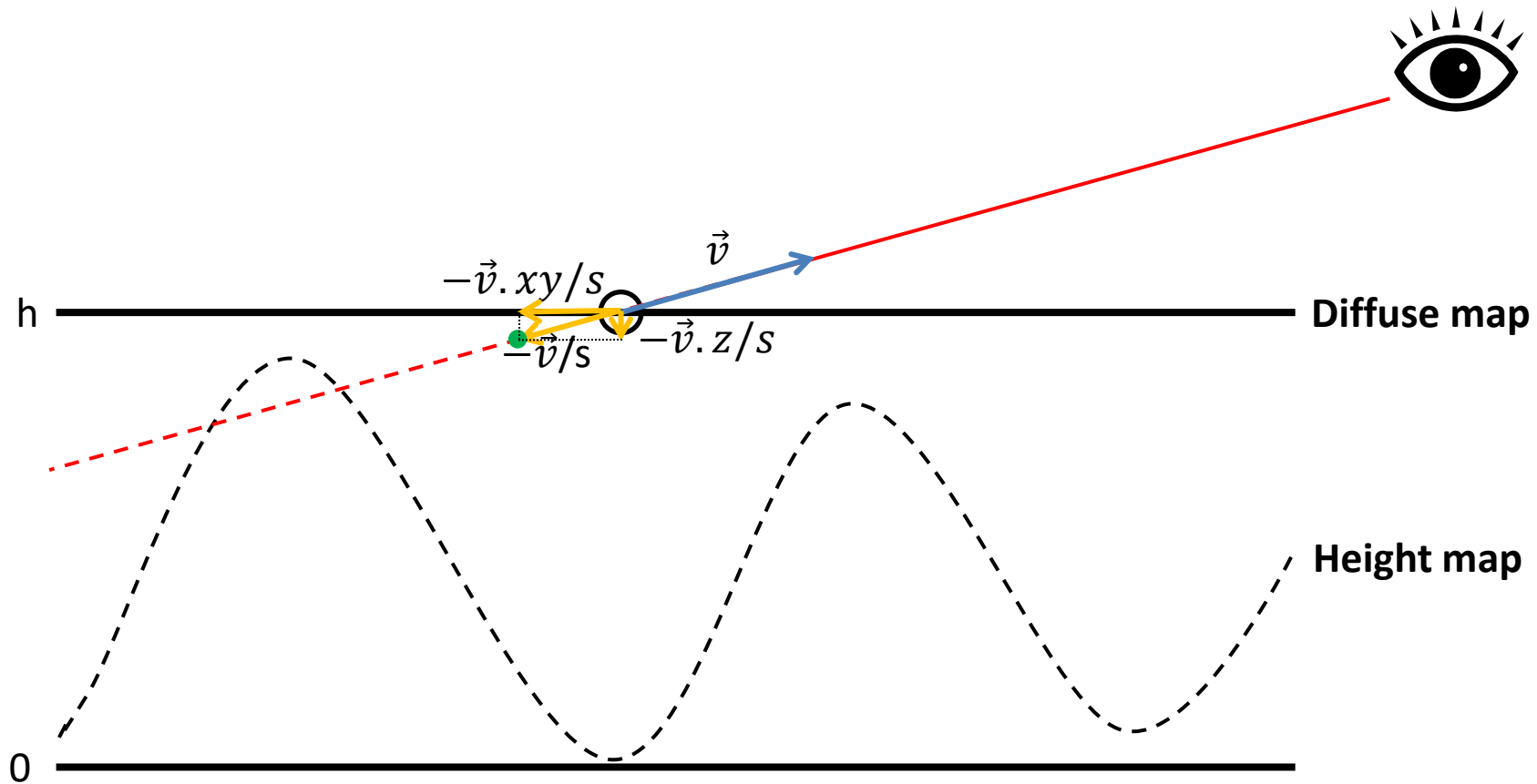
Parallax mapping



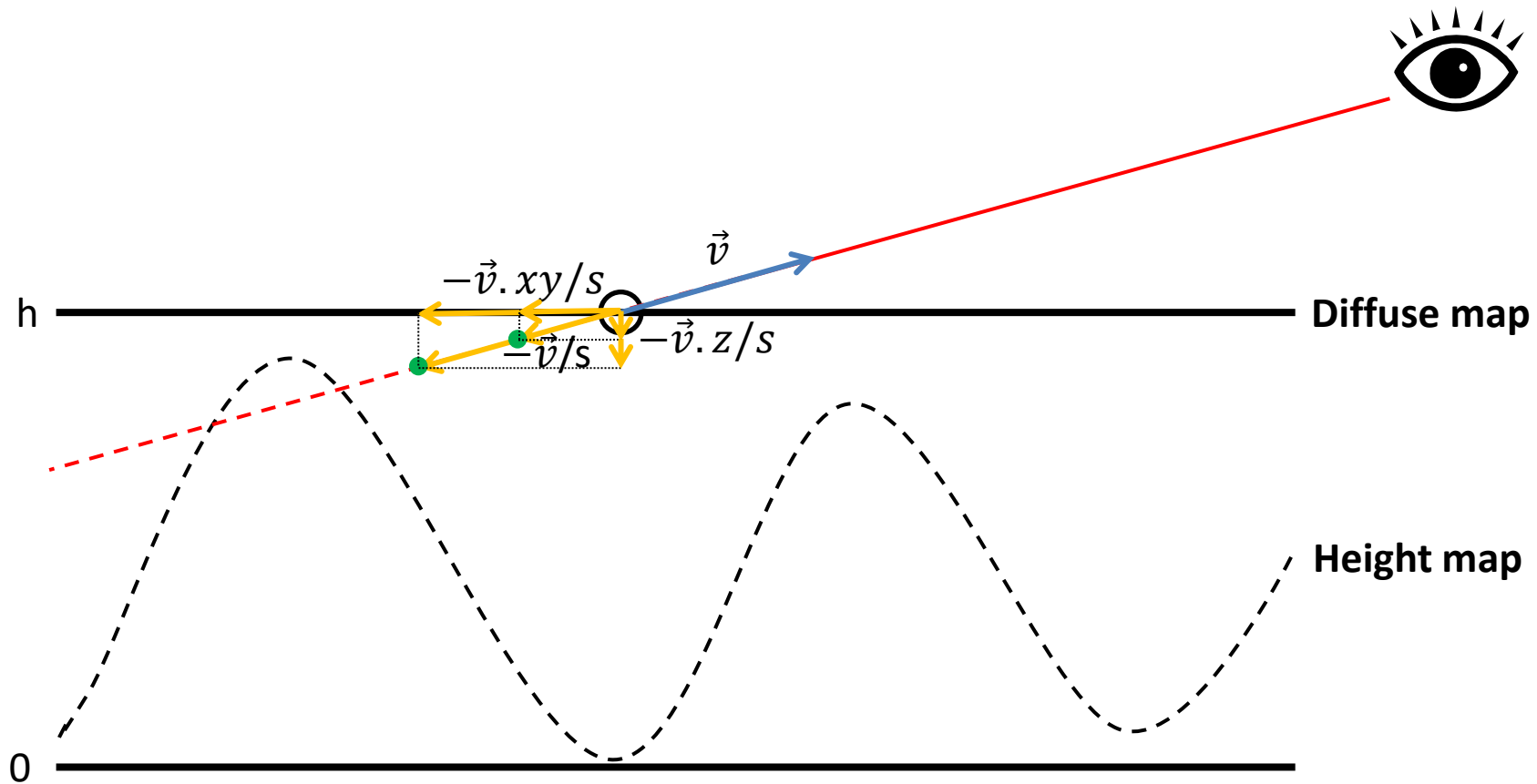
Parallax mapping



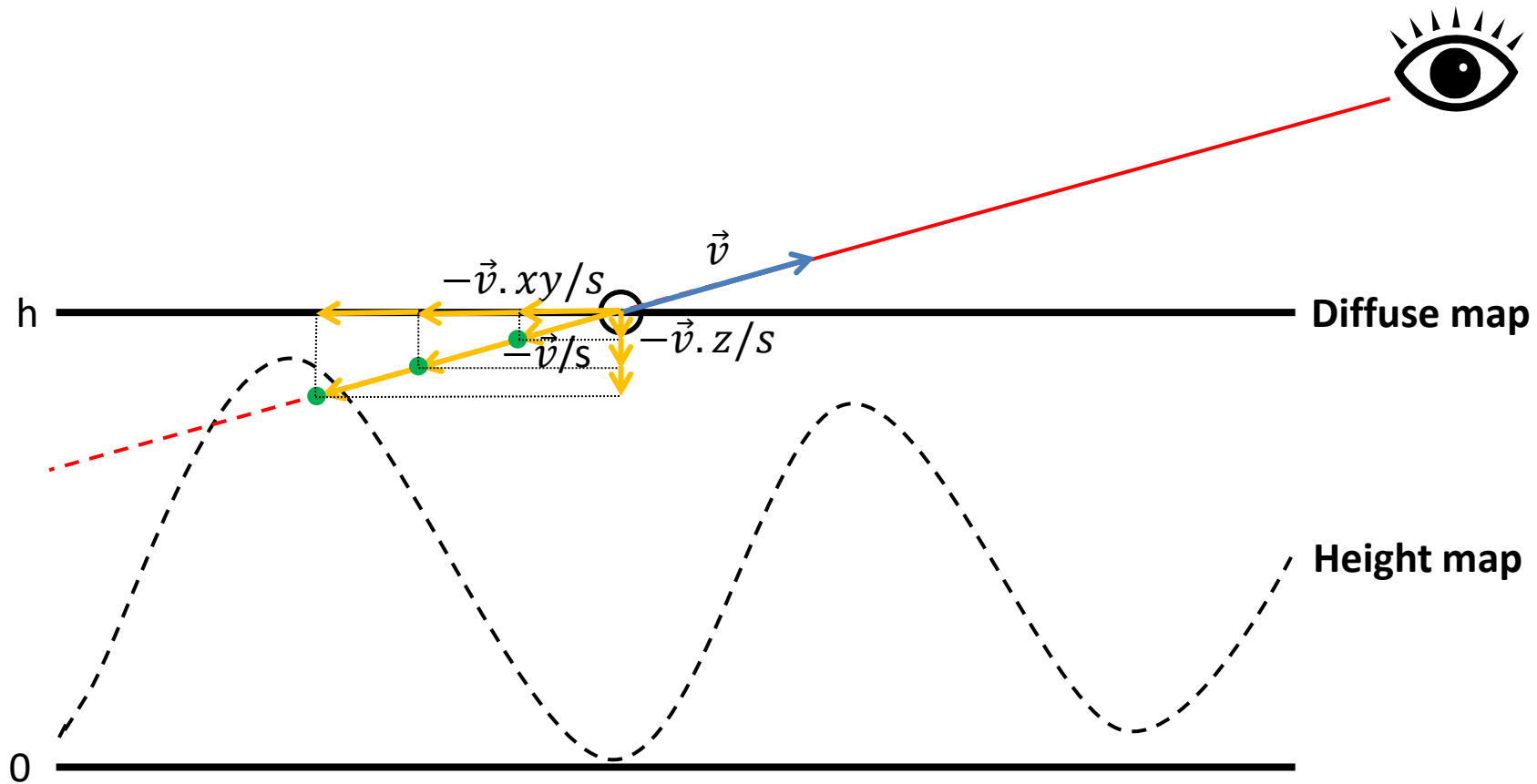
Parallax mapping



Parallax mapping



Parallax mapping



Parallax mapping

h

Diffuse map

Height map

0

$\vec{v}$

$-\vec{v}.x/s$

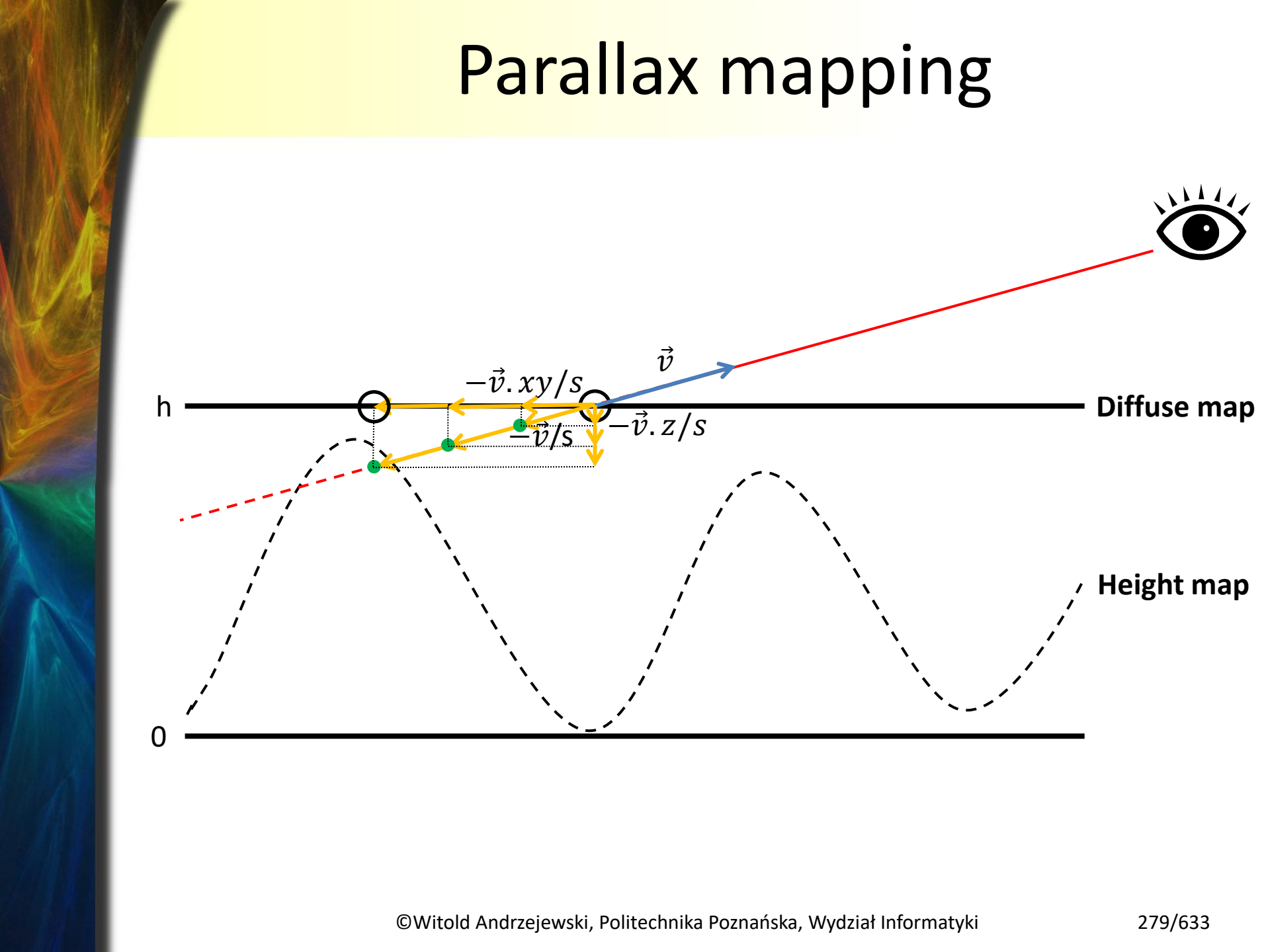
$-\vec{v}.y/s$

$-\vec{v}.z/s$

$\vec{v}/s$

©Witold Andrzejewski, Politechnika Poznańska, Wydział Informatyki

279/633



Algorytm (1/2)

- Wejście:
 - $\vec{v}$ - w przestrzeni stycznej
 - $\vec{t}$ - wsp. teksturowania wyliczone „normalnie”
 - h - maksymalna wysokość
 - s - dokładność
- Inicjowanie:
 - $\vec{t}_i = -\vec{v}.xy/s, \quad h_i = -\vec{v}.z/s$ **//przyrosty**
 - $\vec{t}_c = \vec{t}, \quad h_c = h$ **//aktualne wartości**
 - //wysokość w teksturze**
 - $h_t = h * texture(heightMap, \vec{t}_c).r$

Algorytm (2/2)

if $\vec{v}.z \leq 0$ then *nie rysuj //obs. pod trójkątem*
while $h_c > h_t$: *//dopóki nie dotrzesz pod pow.*
 $\vec{t}_c = \vec{t}_c + \vec{t}_i$ *//zrób krok na teksturze*
 //Nie wyjedź poza teksturę
 if $\vec{t}_c.x \notin [0,1]$ or $\vec{t}_c.y \notin [0,1]$ then *nie rysuj*
 $h_c = h_c + h_i$ *//zrób krok w głąb mapy wysokości*
 //nowa wysokość
 $h_t = h * \text{texture}(\text{heightMap}, \vec{t}_c).r$

Wynik: $\vec{t}_c$

Ćwiczenie 18

- Wczytaj teksturę bricks_height.png z mapą wysokości i udostępni ją fragment shaderowi
- Powinna to być dodatkowa 3 tekstura
- Zaimplementuj funkcję:

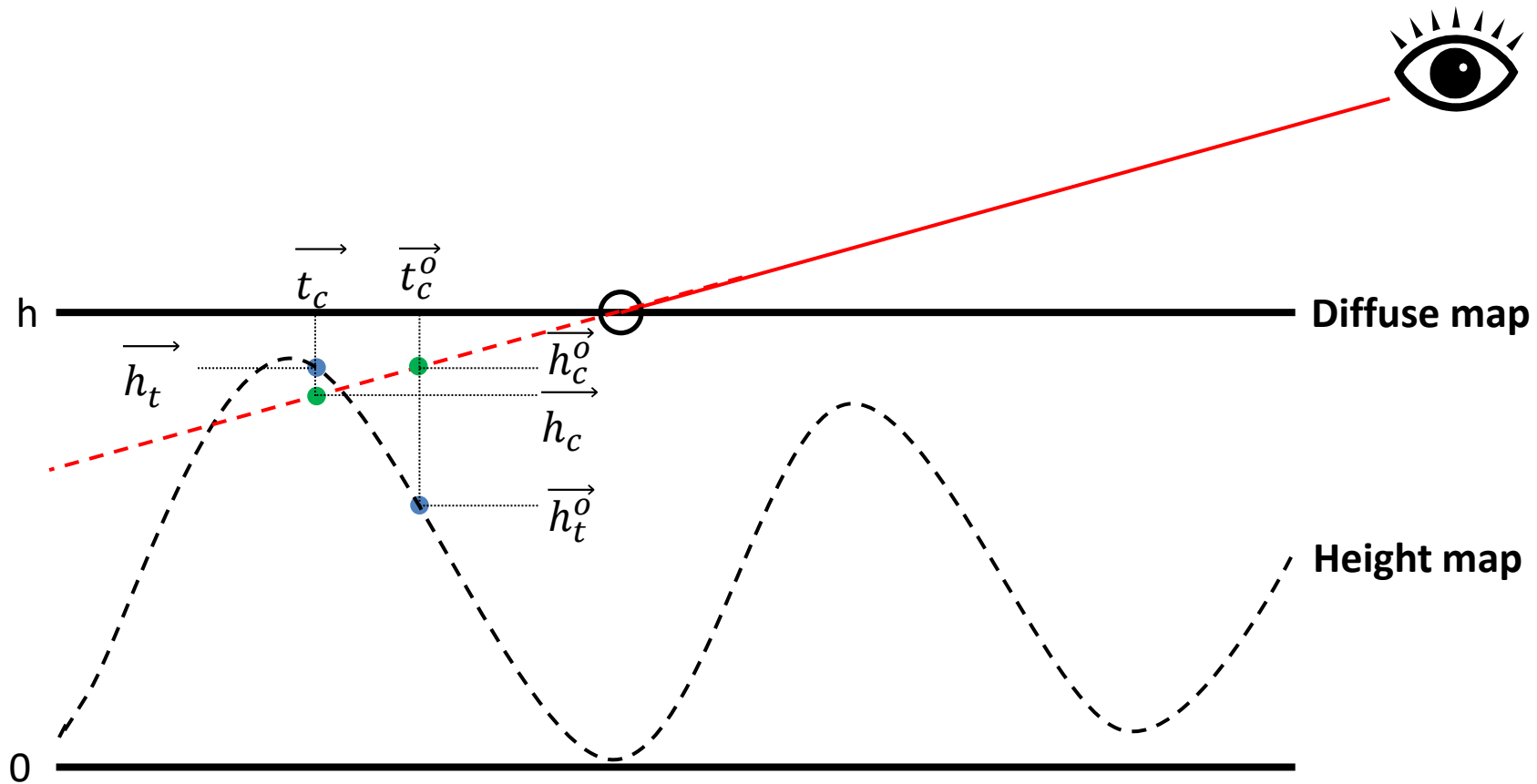
```
vec2 parallaxTexCoords(vec4 v, vec2 t, float h, float s){  
    ...  
    return tc;  
}
```

- Wykorzystaj tą funkcję do obliczenia współrzędnych teksturowania zgodnie z alg. Parallax mapping.
- Wykorzystaj wynik przy pobieraniu z tekstur wartości dla normal mappingu i teksturowaniu (wynik funkcji przypisz do zmiennej, żeby jej nie obliczać dwa razy).
- Przetestuj różne wartości s . Dla jakiej gęstości próbek wynik wygląda dobrze?

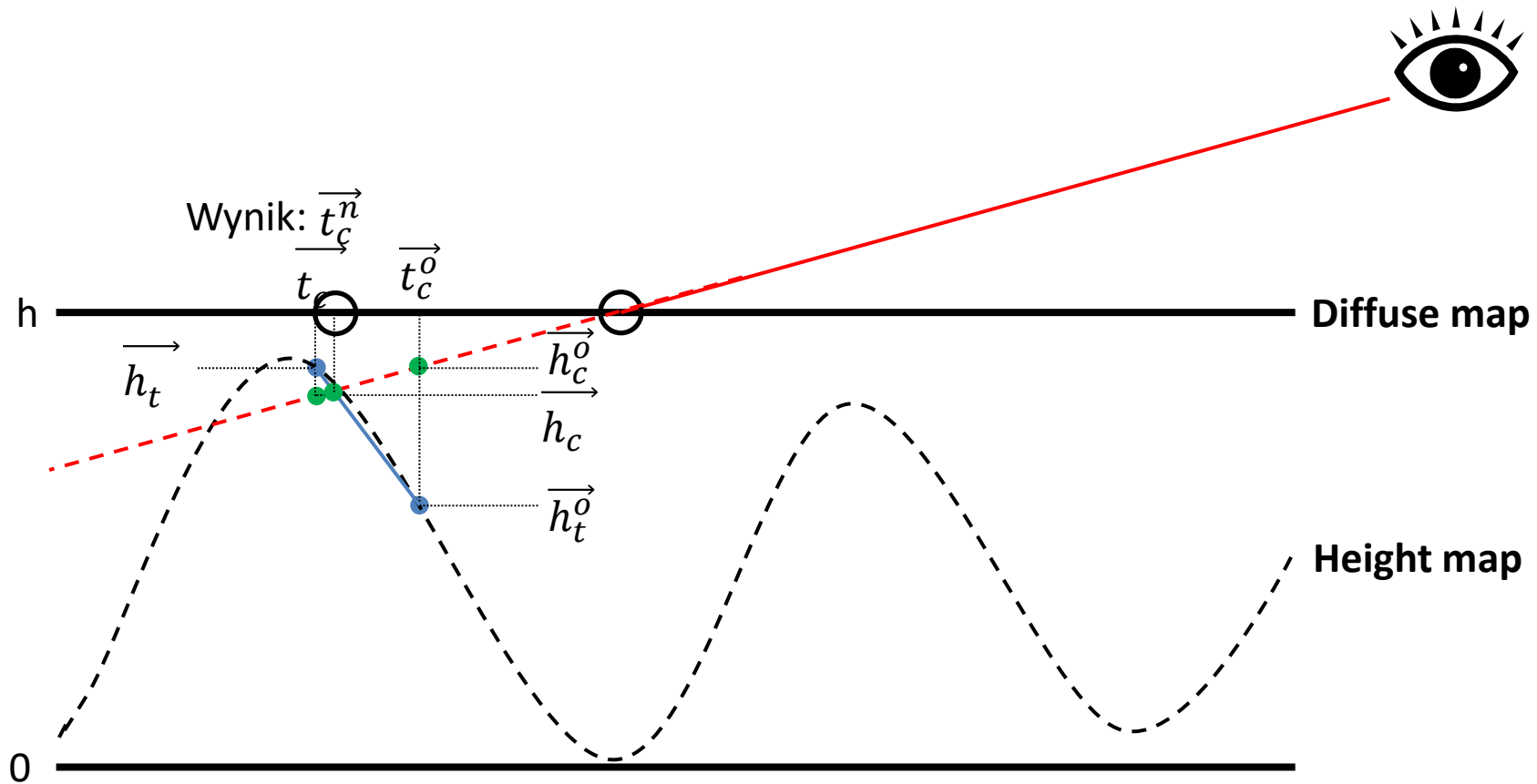
Interpolacja

- Próbkowanie musi być bardzo gęste, żeby mieć wynik dobrej jakości.
- Można próbować odgadnąć pozycję przecięcia pomiędzy dwoma próbkami stosując interpolację.

Parallax mapping



Parallax mapping



Interpolacja pozycji pośredniej

$$\overrightarrow{t_c^n} = (1 - x)\overrightarrow{t_c^o} + x\overrightarrow{t_c}$$

$$h_t^n = (1 - x)h_t^o + xh_t$$

$$h_c^n = (1 - x)h_c^o + xh_c$$

$$h_t^n = h_c^n$$

$$(1 - x)h_t^o + xh_t = (1 - x)h_c^o + xh_c$$

$$h_t^o - xh_t^o + xh_t = h_c^o - xh_c^o + xh_c$$

$$xh_c^o - xh_t^o - x(h_c - h_t) = h_c^o - h_t^o$$

$$x(h_c^o - h_t^o - (h_c - h_t)) = h_c^o - h_t^o$$

$$x = \frac{h_c^o - h_t^o}{h_c^o - h_t^o - (h_c - h_t)}$$

Ćwiczenie 19

- Pamiętaj, że $\vec{t}_c$, h_c i h_t to wartości obliczone w ostatniej iteracji pętli while.
- Zmodyfikuj program tak, aby wykorzystał wzór interpolacyjny (pierwsza i ostatnia linijka poprzedniego slajdu).
- $\vec{t}_c^o$, h_c^o łatwo obliczyć odejmując przyrosty (np. $\vec{t}_c^o = \vec{t}_c - \vec{t}_i$), a h_t^o można odczytać z mapy wysokości w oparciu o $\vec{t}_c^o$
- Zobacz jak dokładne powinno być teraz próbkowanie żeby uzyskać dobrze wyglądające wyniki.

Parallax mapping, optymalizacja (1/2)

- Zmienna liczba iteracji pętli pogarsza wydajność algorytmu.
- Kosztem niewielkiej „niepoprawności” matematycznej można uzyskać stałą liczbę iteracji.
- Załóżmy, że krok „w głąb” jest stały, tj.:

$$h_i = -h/s$$

- zamiast:

$$h_i = -\vec{v} \cdot \vec{z} / s$$

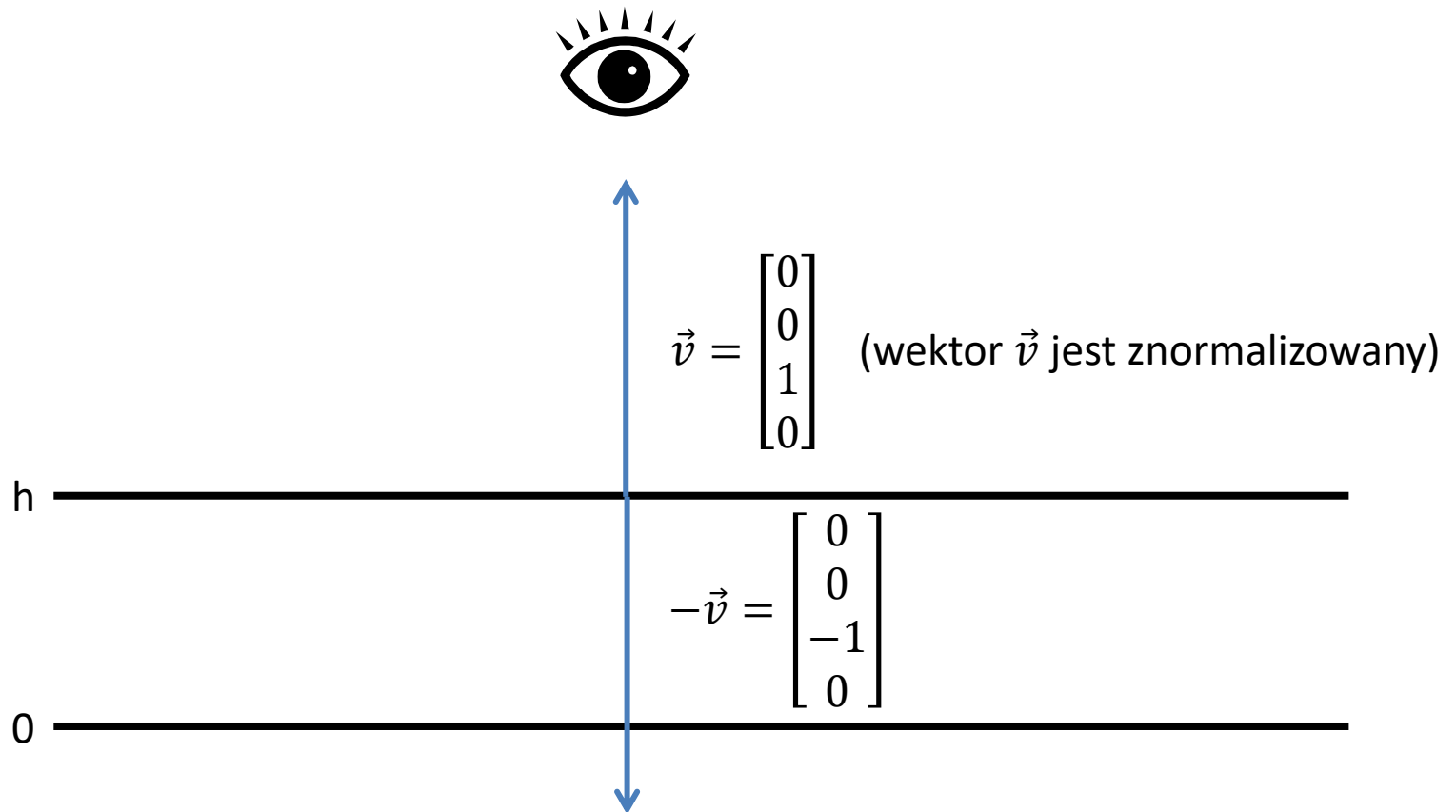
Ćwiczenie 20

- Zmodyfikuj program zgodnie z tym założeniem.
- Zaobserwuj działanie programu. Dlaczego efekt jest zbyt mocny?

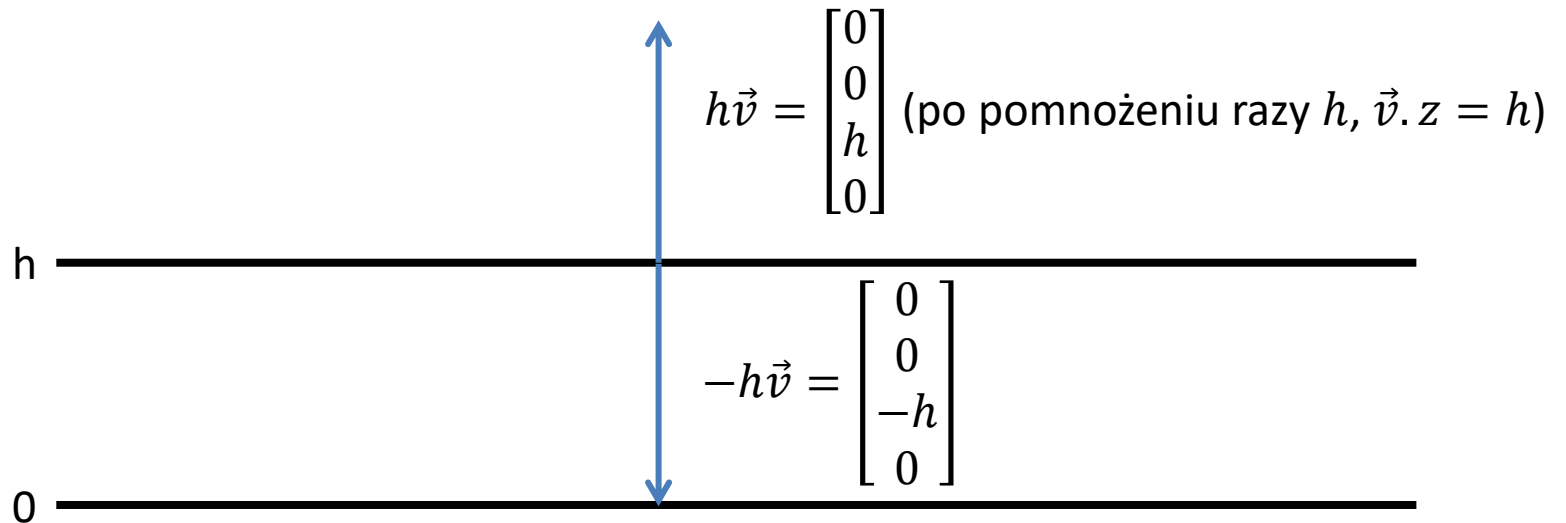
Dlaczego efekt jest zbyt mocny?

- Założenie, że $h_i = -h/s$ zamiast $h_i = -\vec{v} \cdot \vec{z}/s$ oznacza, że przyjmujemy, że $\vec{v} \cdot \vec{z} = h$
- Z oczywistych względów to założenie nie zawsze jest poprawne
- Załóżmy, że ma ono być poprawne wtedy, gdy kierunek do obserwatora jest prostopadły do powierzchni

Dlaczego efekt jest zbyt mocny?



Dlaczego efekt jest zbyt mocny?



Poprawka

- Skoro nasz wektor v powinien być pomnożony razy h przed użyciem, to konieczna jest modyfikacja obliczania $\vec{t}_i$.
- Nowy wzór na $\vec{t}_i$:
$$\vec{t}_i = -h\vec{v} \cdot xy/s$$

Ćwiczenie 21

- Uwzględnij poprawkę z poprzedniego slajdu
- Przetestuj ile warstw trzeba rysować, żeby wynik wyglądał dobrze
- Zaobserwuj błąd wynikający z założenia dającego stały krok „w głąb”
 - Błąd polega na „spłaszczeniu” wypukłości, kiedy patrzymy na ścianę z boku
 - Spłaszczenie wynika z tego, że kiedy patrzymy pod kątem to pętla schodzi „w głąb” za szybko i za szybko dociera do dna.

Ćwiczenie 22

- Mnożenie razy h podczas pobrania głębokości w poleceniu:

$$h_t = h * texture(\overrightarrow{t_c}).r$$

(zarówno podczas inicjowania jak i w pętli)
można wyeliminować jeżeli odpowiednio zmodyfikowane zostaną wyliczenia początkowych wartości h_c i h_i .

- Zastanów się jak to zrobić i zaimplementuj poprawkę

Ćwiczenie 23

- Wykorzystaj następujące tekstury:
 - Diffuse map – stone_diffuse.png
 - Normal map – stone_normal.png
 - Height map – stone_height.png
 - Specular map – stone_specular.png
- Rozszerz ostatnie rozwiązanie o wykorzystanie specular mapy (kolor ks).



OpenGL/GLSL

layout (location = x)

Jawne określanie identyfikatora atrybutu

```
glEnableVertexAttribArray(sp-&gta("vertex"));
glVertexAttribPointer(sp-&gta("vertex"), 4, GL_FLOAT,
    false, 0, vertices);
.....
glDrawArrays( GL_TRIANGLES, 0, geomVertexCount );
.....
glDisableVertexAttribArray(sp-&gta("vertex"));
```

```
layout (location=2) in vec4 vertex;
.....
```

```
printf("%d\n", sp-&gta("vertex")); // Napisze 2
glEnableVertexAttribArray(2);
glVertexAttribPointer(2, 4, GL_FLOAT, false, 0, vertices);
.....
glDrawArrays( GL_TRIANGLES, 0, geomVertexCount );
.....
glDisableVertexAttribArray(2);
```


Jawne określanie identyfikatora zmiennej jednorodnej

```
glUniformMatrix4fv(sp->u("P"), 1, false,  
    glm::value_ptr(P));  
.....  
glDrawArrays( GL_TRIANGLES, 0, geomVertexCount );  
.....
```

```
#version 430  
LUB  
#extension GL_ARB_explicit_uniform_location : enable  
.....  
layout (location=3) uniform mat4 P;  
.....
```

```
printf("%d\n", sp->u("P")); // Napisze 3  
glUniformMatrix4fv(3, 1, false,  
    glm::value_ptr(P));  
.....  
glDrawArrays( GL_TRIANGLES, 0, geomVertexCount );  
.....
```



OpenGL

Vertex Buffer Objects

VBO

- Dane dla VBO i potrzebne deklaracje

```
float geomVertices[]={.....}; //tablica wsp. wierzchołków  
int geomVertexCount=...; //liczba wierzchołków  
GLuint bufGeomVertices; //uchwyt na VBO z wierzchołkami
```

- Utworzenie i uaktywnienie VBO

```
glGenBuffers(1,&bufGeomVertices); //wygenerow. uchwytu  
glBindBuffer(GL_ARRAY_BUFFER,bufGeomVertices);
```

- Wgranie danych do VBO

```
glBufferData(GL_ARRAY_BUFFER,  
             geomVertexCount*sizeof(float)*4, //Liczba bajtów  
             geomVertices, //Tablica z danymi  
             GL_STATIC_DRAW); //Sposób dostępu do danych
```

- Usuwanie VBO

```
glDeleteBuffers(1,&bufGeomVertices);
```


VBO+glDrawArrays

- Rysowanie z użyciem VBO

```
glEnableVertexAttribArray(sp->a("vertex"));
glEnableVertexAttribArray(sp->a("color"));

glBindBuffer(GL_ARRAY_BUFFER, bufGeomVertices);
glVertexAttribPointer(sp->a("vertex"), 4, GL_FLOAT,
                    false, 0, NULL);

glBindBuffer(GL_ARRAY_BUFFER, bufGeomColors);
glVertexAttribPointer(sp->a("color"), 4, GL_FLOAT,
                    false, 0, NULL);

glBindBuffer(GL_ARRAY_BUFFER, 0);

glDrawArrays( GL_TRIANGLES, 0, geomVertexCount );

glDisableVertexAttribArray(sp->a("vertex"));
glDisableVertexAttribArray(sp->a("color"));
```

VBO+glDrawElements

- Tablica indeksów - specjalny bufor VBO:

```
int geomIndexes[]={.....}; //tablica indeksów
int geomIndexCount=...; //liczba indeksów
GLuint bufGeomIndexes; //uchwyt na VBO z indeksami
```

```
glGenBuffers(1, &bufGeomIndexes); //wygenerowanie uchwytu
glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, bufGeomIndexes);
```

```
glBufferData(GL_ELEMENT_ARRAY_BUFFER,
             geomIndexCount*sizeof(int), //Liczba bajtów
             geomIndexes, //Tablica z danymi
             GL_STATIC_DRAW); //Sposób dostępu do danych
```

- Rysowanie (reszta VBO jak dla glDrawArrays)

```
glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, bufGeomIndexes);
glDrawElements(GL_TRIANGLES, geomIndexCount,
              GL_UNSIGNED_INT, NULL );
glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, 0);
```



OpenGL

Vertex Array Object

Tworzenie VAO

```
GLuint vao;  
.....  
glGenVertexArrays(1, &vao)  
glBindVertexArray(vao);  
glEnableVertexAttribArray(sp->a("vertex"));  
glEnableVertexAttribArray(sp->a("color"));  
  
glBindBuffer(GL_ARRAY_BUFFER, bufGeomVertices);  
glVertexAttribPointer(sp->a("vertex"), 4, GL_FLOAT,  
                     false, 0, NULL);  
  
glBindBuffer(GL_ARRAY_BUFFER, bufGeomColors);  
glVertexAttribPointer(sp->a("color"), 4, GL_FLOAT,  
                     false, 0, NULL);  
  
//Poniższa linijka tylko dla glDrawElements!  
glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, bufGeomIndexes);  
  
//glBindBuffer(GL_ARRAY_BUFFER, 0);  
//glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, 0)  
glBindVertexArray(0);
```


Rysowanie

```
glBindVertexArray(vao);  
  
glDrawArrays( GL_TRIANGLES, 0, geomVertexCount );  
//LUB  
glDrawElements(GL_TRIANGLES, geomIndexCount,  
               GL_UNSIGNED_INT, NULL );  
  
glBindVertexArray(0);
```

